

The TILE C Library

libc 1.13.0
January 2004

Tilera Corp.
Steve Chamberlain
Roland Pesch
Red Hat Support
Jeff Johnston

sac@cygnus.com, pesch@cygnus.com, jjohnstn@redhat.com

The TILE C Library

Copyright © 1992, 1993, 1994-2004 Red Hat Inc.

‘libc’ includes software developed by the University of California, Berkeley and its contributors.

‘libc’ includes software developed by Martin Jackson, Graham Haley and Steve Chamberlain of Tadpole Technology and released to Cygnus.

‘libc’ uses floating-point conversion software developed at AT&T, which includes this copyright information:

The author of this software is David M. Gay.

Copyright (c) 1991 by AT&T.

Permission to use, copy, modify, and distribute this software for any purpose without fee is hereby granted, provided that this entire notice is included in all copies of any software which is or includes a copy or modification of this software and in all copies of the supporting documentation for such software.

THIS SOFTWARE IS BEING PROVIDED "AS IS", WITHOUT ANY EXPRESS OR IMPLIED WARRANTY. IN PARTICULAR, NEITHER THE AUTHOR NOR AT&T MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND CONCERNING THE MERCHANTABILITY OF THIS SOFTWARE OR ITS FITNESS FOR ANY PARTICULAR PURPOSE.

Permission is granted to make and distribute verbatim copies of this manual provided the copyright notice and this permission notice are preserved on all copies.

Permission is granted to copy and distribute modified versions of this manual under the conditions for verbatim copying, subject to the terms of the GNU General Public License, which includes the provision that the entire resulting derived work is distributed under the terms of a permission notice identical to this one.

Permission is granted to copy and distribute translations of this manual into another language, under the above conditions for modified versions.

1 Standard Utility Functions (`'stdlib.h'`)

This chapter groups utility functions useful in a variety of programs. The corresponding declarations are in the header file `'stdlib.h'`.

1.1 `_Exit`—end program execution with no cleanup processing

Synopsis

```
#include <stdlib.h>
void _Exit(int code);
```

Description

Use `_Exit` to return control from a program to the host operating environment. Use the argument *code* to pass an exit status to the operating environment: two particular values, `EXIT_SUCCESS` and `EXIT_FAILURE`, are defined in ‘`stdlib.h`’ to indicate success or failure in a portable fashion.

`_Exit` differs from `exit` in that it does not run any application-defined cleanup functions registered with `atexit` and it does not clean up files and streams. It is identical to `_exit`.

Returns

`_Exit` does not return to its caller.

Portability

`_Exit` is defined by the C99 standard.

Supporting OS subroutines required: `_exit`.

1.2 a64l,l64a—convert between radix-64 ascii string and long

Synopsis

```
#include <stdlib.h>
long a64l(const char *input);
char *l64a(long input);
```

Description

Conversion is performed between long and radix-64 characters. The **l64a** routine transforms up to 32-bits of input value starting from least significant bits to the most significant bits. The input value is split up into a maximum of 5 groups of 6-bits and possibly one group of 2 bits (bits 31 and 30).

Each group of 6 bits forms a value from 0-63 which is translated into a character as follows:

0 = '.' 1 = '/' 2-11 = '0' to '9' 12-37 = 'A' to 'Z' 38-63 = 'a' to 'z'

When remaining bits are zero or all bits have been translated, a nul terminator is appended to the string. An input value of 0 results in the empty string.

The **a64l** performs the reverse translation. Each character is used to generate a 6-bit value for up to 30 bits and then a 2-bit value to complete a 32-bit result. The nul terminator means that the remaining digits are 0. An empty input string or NULL string results in 0L. An invalid string results in undefined behavior. If the size of a long is > 32 bits, the result is sign-extended.

Returns

l64a returns a nul-terminated string of 0 to 6 characters. **a64l** returns the 32-bit translated value from the input character string.

Portability

l64a and **a64l** are non-ANSI and are defined by the Single Unix Specification.

Supporting OS subroutines required: None.

1.3 abort—abnormal termination of a program

Synopsis

```
#include <stdlib.h>
void abort(void);
```

Description

Use **abort** to signal that your program has detected a condition it cannot deal with. Normally, **abort** ends your program's execution.

Before terminating your program, **abort** raises the exception **SIGABRT** (using `'raise(SIGABRT)'`). If you have used **signal** to register an exception handler for this condition, that handler has the opportunity to retain control, thereby avoiding program termination.

In this implementation, **abort** does not perform any stream- or file-related cleanup (the host environment may do so; if not, you can arrange for your program to do its own cleanup with a **SIGABRT** exception handler).

Returns

abort does not return to its caller.

Portability

ANSI C requires **abort**.

Supporting OS subroutines required: **_exit** and optionally, **write**.

1.4 abs—integer absolute value (magnitude)

Synopsis

```
#include <stdlib.h>
int abs(int i);
```

Description

abs returns $|x|$, the absolute value of i (also called the magnitude of i). That is, if i is negative, the result is the opposite of i , but if i is nonnegative the result is i .

The similar function **labs** uses and returns **long** rather than **int** values.

Returns

The result is a nonnegative integer.

Portability

abs is ANSI.

No supporting OS subroutines are required.

1.5 `assert`—Macro for Debugging Diagnostics

Synopsis

```
#include <assert.h>
void assert(int expression);
```

Description

Use this macro to embed debuggging diagnostic statements in your programs. The argument *expression* should be an expression which evaluates to true (nonzero) when your program is working as you intended.

When *expression* evaluates to false (zero), `assert` calls `abort`, after first printing a message showing what failed and where:

```
Assertion failed: expression, file filename, line lineno
```

The macro is defined to permit you to turn off all uses of `assert` at compile time by defining `NDEBUG` as a preprocessor variable. If you do this, the `assert` macro expands to

```
(void(0))
```

Returns

`assert` does not return a value.

Portability

The `assert` macro is required by ANSI, as is the behavior when `NDEBUG` is defined.

Supporting OS subroutines required (only if enabled): `close`, `fstat`, `getpid`, `isatty`, `kill`, `lseek`, `read`, `sbrk`, `write`.

1.6 atexit—request execution of functions at program exit

Synopsis

```
#include <stdlib.h>
int atexit (void (*function)(void));
```

Description

You can use **atexit** to enroll functions in a list of functions that will be called when your program terminates normally. The argument is a pointer to a user-defined function (which must not require arguments and must not return a result).

The functions are kept in a LIFO stack; that is, the last function enrolled by **atexit** will be the first to execute when your program exits.

There is no built-in limit to the number of functions you can enroll in this list; however, after every group of 32 functions is enrolled, **atexit** will call **malloc** to get space for the next part of the list. The initial list of 32 functions is statically allocated, so you can always count on at least that many slots available.

Returns

atexit returns 0 if it succeeds in enrolling your function, -1 if it fails (possible only if no space was available for **malloc** to extend the list of functions).

Portability

atexit is required by the ANSI standard, which also specifies that implementations must support enrolling at least 32 functions.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

1.7 atof, atoff—string to double or float

Synopsis

```
#include <stdlib.h>
double atof(const char *s);
float atoff(const char *s);
```

Description

atof converts the initial portion of a string to a **double**. **atoff** converts the initial portion of a string to a **float**.

The functions parse the character string *s*, locating a substring which can be converted to a floating-point value. The substring must match the format:

```
[+|-]digits[.][digits][(e|E)[+|-]digits]
```

The substring converted is the longest initial fragment of *s* that has the expected format, beginning with the first non-whitespace character. The substring is empty if **str** is empty, consists entirely of whitespace, or if the first non-whitespace character is something other than +, -, ., or a digit.

atof(s) is implemented as **strtod(s, NULL)**. **atoff(s)** is implemented as **strtod(s, NULL)**.

Returns

atof returns the converted substring value, if any, as a **double**; or 0.0, if no conversion could be performed. If the correct value is out of the range of representable values, plus or minus **HUGE_VAL** is returned, and **ERANGE** is stored in **errno**. If the correct value would cause underflow, 0.0 is returned and **ERANGE** is stored in **errno**.

atoff obeys the same rules as **atof**, except that it returns a **float**.

Portability

atof is ANSI C. **atof**, **atoi**, and **atol** are subsumed by **strod** and **strol**, but are used extensively in existing code. These functions are less reliable, but may be faster if the argument is verified to be in a valid range.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

1.8 atoi, atol—string to integer

Synopsis

```
#include <stdlib.h>
int atoi(const char *s);
long atol(const char *s);
int _atoi_r(struct _reent *ptr, const char *s);
long _atol_r(struct _reent *ptr, const char *s);
```

Description

`atoi` converts the initial portion of a string to an `int`. `atol` converts the initial portion of a string to a `long`.

`atoi(s)` is implemented as `(int)strtol(s, NULL, 10)`. `atol(s)` is implemented as `strtol(s, NULL, 10)`.

`_atoi_r` and `_atol_r` are reentrant versions of `atoi` and `atol` respectively, passing the reentrancy struct pointer.

Returns

The functions return the converted value, if any. If no conversion was made, 0 is returned.

Portability

`atoi`, `atol` are ANSI.

No supporting OS subroutines are required.

1.9 `atoll`—convert a string to a long long integer

Synopsis

```
#include <stdlib.h>
long long atoll(const char *str);
long long _atoll_r(struct _reent *ptr, const char *str);
```

Description

The function `atoll` converts the initial portion of the string pointed to by **str* to a type `long long`. A call to `atoll(str)` in this implementation is equivalent to `strtoll(str, (char **)NULL, 10)` including behavior on error.

The alternate function `_atoll_r` is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

The converted value.

Portability

`atoll` is ISO 9899 (C99) and POSIX 1003.1-2001 compatible.

No supporting OS subroutines are required.

1.10 calloc—allocate space for arrays

Synopsis

```
#include <stdlib.h>
void *calloc(size_t n, size_t s);
void *calloc_r(void *reent, size_t <n>, <size_t> s);
```

Description

Use `calloc` to request a block of memory sufficient to hold an array of n elements, each of which has size s .

The memory allocated by `calloc` comes out of the same memory pool used by `malloc`, but the memory block is initialized to all zero bytes. (To avoid the overhead of initializing the space, use `malloc` instead.)

The alternate function `_calloc_r` is reentrant. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

If successful, a pointer to the newly allocated space.

If unsuccessful, `NULL`.

Portability

`calloc` is ANSI.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

1.11 `div`—divide two integers

Synopsis

```
#include <stdlib.h>
div_t div(int n, int d);
```

Description

Divide n/d , returning quotient and remainder as two integers in a structure `div_t`.

Returns

The result is represented with the structure

```
typedef struct
{
    int quot;
    int rem;
} div_t;
```

where the `quot` field represents the quotient, and `rem` the remainder. For nonzero d , if ‘ $r = \text{div}(n, d);$ ’ then n equals ‘ $r.\text{rem} + d * r.\text{quot}$ ’.

To divide `long` rather than `int` values, use the similar function `ldiv`.

Portability

`div` is ANSI.

No supporting OS subroutines are required.

1.12 `ecvt, ecvtf, fcvt, fcvtf`—double or float to string

Synopsis

```
#include <stdlib.h>

char *ecvt(double val, int chars, int *decpt, int *sgn);
char *ecvtf(float val, int chars, int *decpt, int *sgn);

char *fcvt(double val, int decimals,
            int *decpt, int *sgn);
char *fcvtf(float val, int decimals,
            int *decpt, int *sgn);
```

Description

`ecvt` and `fcvt` produce (null-terminated) strings of digits representating the double number `val`. `ecvtf` and `fcvtf` produce the corresponding character representations of `float` numbers.

(The `stdlib` functions `ecvtbuf` and `fcvtbuf` are reentrant versions of `ecvt` and `fcvt`.)

The only difference between `ecvt` and `fcvt` is the interpretation of the second argument (*chars* or *decimals*). For `ecvt`, the second argument *chars* specifies the total number of characters to write (which is also the number of significant digits in the formatted string, since these two functions write only digits). For `fcvt`, the second argument *decimals* specifies the number of characters to write after the decimal point; all digits for the integer part of *val* are always included.

Since `ecvt` and `fcvt` write only digits in the output string, they record the location of the decimal point in **decpt*, and the sign of the number in **sgn*. After formatting a number, **decpt* contains the number of digits to the left of the decimal point. **sgn* contains 0 if the number is positive, and 1 if it is negative.

Returns

All four functions return a pointer to the new string containing a character representation of *val*.

Portability

None of these functions are ANSI C.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

1.13 gcvvt, gcvtf—format double or float as string

Synopsis

```
#include <stdlib.h>
```

```
char *gcvt(double val, int precision, char *buf);  
char *gcvtf(float val, int precision, char *buf);
```

Description

gcvt writes a fully formatted number as a null-terminated string in the buffer **buf*. **gcvtf** produces corresponding character representations of **float** numbers.

gcvt uses the same rules as the **printf** format *'%.precisiong'*—only negative values are signed (with '-'), and either exponential or ordinary decimal-fraction format is chosen depending on the number of significant digits (specified by *precision*).

Returns

The result is a pointer to the formatted representation of *val* (the same as the argument *buf*).

Portability

Neither function is ANSI C.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

1.14 ecvtbuf, fcvtbuf—double or float to string

Synopsis

```
#include <stdio.h>

char *ecvtbuf(double val, int chars, int *decpt,
               int *sgn, char *buf);

char *fcvtbuf(double val, int decimals, int *decpt,
               int *sgn, char *buf);
```

Description

`ecvtbuf` and `fcvtbuf` produce (null-terminated) strings of digits representating the **double** number *val*.

The only difference between `ecvtbuf` and `fcvtbuf` is the interpretation of the second argument (*chars* or *decimals*). For `ecvtbuf`, the second argument *chars* specifies the total number of characters to write (which is also the number of significant digits in the formatted string, since these two functions write only digits). For `fcvtbuf`, the second argument *decimals* specifies the number of characters to write after the decimal point; all digits for the integer part of *val* are always included.

Since `ecvtbuf` and `fcvtbuf` write only digits in the output string, they record the location of the decimal point in **decpt*, and the sign of the number in **sgn*. After formatting a number, **decpt* contains the number of digits to the left of the decimal point. **sgn* contains 0 if the number is positive, and 1 if it is negative. For both functions, you supply a pointer *buf* to an area of memory to hold the converted string.

Returns

Both functions return a pointer to *buf*, the string containing a character representation of *val*.

Portability

Neither function is ANSI C.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

1.15 `__env_lock`, `__env_unlock`—lock environ variable

Synopsis

```
#include "envlock.h"
void __env_lock (struct _reent *reent);
void __env_unlock (struct _reent *reent);
```

Description

The `setenv` family of routines call these functions when they need to modify the environ variable. The version of these routines supplied in the library use the lock API defined in `sys/lock.h`. If multiple threads of execution can call `setenv`, or if `setenv` can be called reentrantly, then you need to define your own versions of these functions in order to safely lock the memory pool during a call. If you do not, the memory pool may become corrupted.

A call to `setenv` may call `__env_lock` recursively; that is, the sequence of calls may go `__env_lock`, `__env_lock`, `__env_unlock`, `__env_unlock`. Any implementation of these routines must be careful to avoid causing a thread to wait for a lock that it already holds.

1.16 `exit`—end program execution

Synopsis

```
#include <stdlib.h>
void exit(int code);
```

Description

Use `exit` to return control from a program to the host operating environment. Use the argument *code* to pass an exit status to the operating environment: two particular values, `EXIT_SUCCESS` and `EXIT_FAILURE`, are defined in '`stdlib.h`' to indicate success or failure in a portable fashion.

`exit` does two kinds of cleanup before ending execution of your program. First, it calls all application-defined cleanup functions you have enrolled with `atexit`. Second, files and streams are cleaned up: any pending output is delivered to the host system, each open file or stream is closed, and files created by `tmpfile` are deleted.

Returns

`exit` does not return to its caller.

Portability

ANSI C requires `exit`, and specifies that `EXIT_SUCCESS` and `EXIT_FAILURE` must be defined.

Supporting OS subroutines required: `_exit`.

1.17 `getenv`—look up environment variable

Synopsis

```
#include <stdlib.h>
char *getenv(const char *name);
```

Description

`getenv` searches the list of environment variable names and values (using the global pointer “`char **environ`”) for a variable whose name matches the string at *name*. If a variable name matches, `getenv` returns a pointer to the associated value.

Returns

A pointer to the (string) value of the environment variable, or `NULL` if there is no such environment variable.

Portability

`getenv` is ANSI, but the rules for properly forming names of environment variables vary from one system to another.

`getenv` requires a global pointer `environ`.

1.18 labs—long integer absolute value

Synopsis

```
#include <stdlib.h>
long labs(long i);
```

Description

labs returns $|x|$, the absolute value of i (also called the magnitude of i). That is, if i is negative, the result is the opposite of i , but if i is nonnegative the result is i .

The similar function **abs** uses and returns **int** rather than **long** values.

Returns

The result is a nonnegative long integer.

Portability

labs is ANSI.

No supporting OS subroutine calls are required.

1.19 ldiv—divide two long integers

Synopsis

```
#include <stdlib.h>
ldiv_t ldiv(long n, long d);
```

Description

Divide n/d , returning quotient and remainder as two long integers in a structure `ldiv_t`.

Returns

The result is represented with the structure

```
typedef struct
{
    long quot;
    long rem;
} ldiv_t;
```

where the `quot` field represents the quotient, and `rem` the remainder. For nonzero d , if ‘ $r = \text{ldiv}(n, d)$ ’;’ then n equals ‘ $r.\text{rem} + d * r.\text{quot}$ ’.

To divide `int` rather than `long` values, use the similar function `div`.

Portability

`ldiv` is ANSI.

No supporting OS subroutines are required.

1.20 labs—compute the absolute value of a long integer.

Synopsis

```
#include <stdlib.h>
long labs(long j);
```

Description

The `labs` function computes the absolute value of the long integer argument *j* (also called the magnitude of *j*).

The similar function `labs` uses and returns `long` rather than `long long` values.

Returns

A nonnegative long integer.

Portability

`labs` is ISO 9899 (C99) compatible.

No supporting OS subroutines are required.

1.21 lldiv—divide two long long integers

Synopsis

```
#include <stdlib.h>
lldiv_t lldiv(long long n, long long d);
```

Description

Divide n/d , returning quotient and remainder as two long long integers in a structure `lldiv_t`.

Returns

The result is represented with the structure

```
typedef struct
{
    long long quot;
    long long rem;
} lldiv_t;
```

where the `quot` field represents the quotient, and `rem` the remainder. For nonzero d , if ‘ $r = \text{ldiv}(n, d)$ ’ then n equals ‘ $r.\text{rem} + d * r.\text{quot}$ ’.

To divide `long` rather than `long long` values, use the similar function `ldiv`.

Portability

`lldiv` is ISO 9899 (C99) compatible.

No supporting OS subroutines are required.

1.22 malloc, realloc, free—manage memory

Synopsis

```
#include <stdlib.h>
void *malloc(size_t nbytes);
void *realloc(void *aptr, size_t nbytes);
void free(void *aptr);

void *memalign(size_t align, size_t nbytes);

size_t malloc_usable_size(void *aptr);

void *_malloc_r(void *reent, size_t nbytes);
void *_realloc_r(void *reent,
    void *aptr, size_t nbytes);
void _free_r(void *reent, void *aptr);

void *_memalign_r(void *reent,
    size_t align, size_t nbytes);

size_t _malloc_usable_size_r(void *reent, void *aptr);
```

Description

These functions manage a pool of system memory.

Use **malloc** to request allocation of an object with at least *nbytes* bytes of storage available. If the space is available, **malloc** returns a pointer to a newly allocated block as its result.

If you already have a block of storage allocated by **malloc**, but you no longer need all the space allocated to it, you can make it smaller by calling **realloc** with both the object pointer and the new desired size as arguments. **realloc** guarantees that the contents of the smaller object match the beginning of the original object.

Similarly, if you need more space for an object, use **realloc** to request the larger size; again, **realloc** guarantees that the beginning of the new, larger object matches the contents of the original object.

When you no longer need an object originally allocated by **malloc** or **realloc** (or the related function **calloc**), return it to the memory storage pool by calling **free** with the address of the object as the argument. You can also use **realloc** for this purpose by calling it with 0 as the *nbytes* argument.

The **memalign** function returns a block of size *nbytes* aligned to a *align* boundary. The *align* argument must be a power of two.

The **malloc_usable_size** function takes a pointer to a block allocated by **malloc**. It returns the amount of space that is available in the block. This may or may not be more than the size requested from **malloc**, due to alignment or minimum size constraints.

The alternate functions **_malloc_r**, **_realloc_r**, **_free_r**, **_memalign_r**, and **_malloc_usable_size_r** are reentrant versions. The extra argument *reent* is a pointer to a reentrancy structure.

If you have multiple threads of execution which may call any of these routines, or if any of these routines may be called reentrantly, then you must provide implementations of the `__malloc_lock` and `__malloc_unlock` functions for your system. See the documentation for those functions.

These functions operate by calling the function `_sbrk_r` or `sbrk`, which allocates space. You may need to provide one of these functions for your system. `_sbrk_r` is called with a positive value to allocate more space, and with a negative value to release previously allocated space if it is no longer required. See Section 10.1 [Stubs], page 241.

Returns

`malloc` returns a pointer to the newly allocated space, if successful; otherwise it returns `NULL`. If your application needs to generate empty objects, you may use `malloc(0)` for this purpose.

`realloc` returns a pointer to the new block of memory, or `NULL` if a new block could not be allocated. `NULL` is also the result when you use `'realloc(aptr,0)'` (which has the same effect as `'free(aptr)'`). You should always check the result of `realloc`; successful reallocation is not guaranteed even when you request a smaller object.

`free` does not return a result.

`memalign` returns a pointer to the newly allocated space.

`malloc_usable_size` returns the usable size.

Portability

`malloc`, `realloc`, and `free` are specified by the ANSI C standard, but other conforming implementations of `malloc` may behave differently when *nbytes* is zero.

`memalign` is part of SVR4.

`malloc_usable_size` is not portable.

Supporting OS subroutines required: `sbrk`.

1.23 mallinfo, malloc_stats, mallopt—malloc support

Synopsis

```
#include <malloc.h>
struct mallinfo mallinfo(void);
void malloc_stats(void);
int mallopt(int parameter, value);

struct mallinfo _mallinfo_r(void *reent);
void _malloc_stats_r(void *reent);
int _mallopt_r(void *reent, int parameter, value);
```

Description

mallinfo returns a structure describing the current state of memory allocation. The structure is defined in `malloc.h`. The following fields are defined: **arena** is the total amount of space in the heap; **ordblks** is the number of chunks which are not in use; **uordblks** is the total amount of space allocated by **malloc**; **fordblks** is the total amount of space not in use; **keepcost** is the size of the top most memory block.

malloc_stats print some statistics about memory allocation on standard error.

mallopt takes a parameter and a value. The parameters are defined in `malloc.h`, and may be one of the following: **M_TRIM_THRESHOLD** sets the maximum amount of unused space in the top most block before releasing it back to the system in **free** (the space is released by calling **_sbrk_r** with a negative argument); **M_TOP_PAD** is the amount of padding to allocate whenever **_sbrk_r** is called to allocate more space.

The alternate functions **_mallinfo_r**, **_malloc_stats_r**, and **_mallopt_r** are reentrant versions. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

mallinfo returns a `mallinfo` structure. The structure is defined in `malloc.h`.

malloc_stats does not return a result.

mallopt returns zero if the parameter could not be set, or non-zero if it could be set.

Portability

mallinfo and **mallopt** are provided by SVR4, but **mallopt** takes different parameters on different systems. **malloc_stats** is not portable.

1.24 `__malloc_lock`, `__malloc_unlock`—lock malloc pool

Synopsis

```
#include <malloc.h>
void __malloc_lock (struct _reent *reent);
void __malloc_unlock (struct _reent *reent);
```

Description

The `malloc` family of routines call these functions when they need to lock the memory pool. The version of these routines supplied in the library use the lock API defined in `sys/lock.h`. If multiple threads of execution can call `malloc`, or if `malloc` can be called reentrantly, then you need to define your own versions of these functions in order to safely lock the memory pool during a call. If you do not, the memory pool may become corrupted.

A call to `malloc` may call `__malloc_lock` recursively; that is, the sequence of calls may go `__malloc_lock`, `__malloc_lock`, `__malloc_unlock`, `__malloc_unlock`. Any implementation of these routines must be careful to avoid causing a thread to wait for a lock that it already holds.

1.25 mblen—minimal multibyte length function

Synopsis

```
#include <stdlib.h>
int mblen(const char *s, size_t n);
```

Description

When `_MB_CAPABLE` is not defined, this is a minimal ANSI-conforming implementation of `mblen`. In this case, the only “multi-byte character sequences” recognized are single bytes, and thus 1 is returned unless `s` is the null pointer or has a length of 0 or is the empty string.

When `_MB_CAPABLE` is defined, this routine calls `_mbtowc_r` to perform the conversion, passing a state variable to allow state dependent decoding. The result is based on the locale setting which may be restricted to a defined set of locales.

Returns

This implementation of `mblen` returns 0 if `s` is NULL or the empty string; it returns 1 if not `_MB_CAPABLE` or the character is a single-byte character; it returns -1 if the multi-byte character is invalid; otherwise it returns the number of bytes in the multibyte character.

Portability

`mblen` is required in the ANSI C standard. However, the precise effects vary with the locale. `mblen` requires no supporting OS subroutines.

1.26 `mbstowcs`—minimal multibyte string to wide char converter

Synopsis

```
#include <stdlib.h>
int mbstowcs(wchar_t *pwc, const char *s, size_t n);
```

Description

When `_MB_CAPABLE` is not defined, this is a minimal ANSI-conforming implementation of `mbstowcs`. In this case, the only “multi-byte character sequences” recognized are single bytes, and they are “converted” to wide-char versions simply by byte extension.

When `_MB_CAPABLE` is defined, this routine calls `_mbstowcs_r` to perform the conversion, passing a state variable to allow state dependent decoding. The result is based on the locale setting which may be restricted to a defined set of locales.

Returns

This implementation of `mbstowcs` returns 0 if `s` is `NULL` or is the empty string; it returns -1 if `_MB_CAPABLE` and one of the multi-byte characters is invalid or incomplete; otherwise it returns the minimum of: `n` or the number of multi-byte characters in `s` plus 1 (to compensate for the nul character). If the return value is -1, the state of the `pwc` string is indeterminate. If the input has a length of 0, the output string will be modified to contain a `wchar_t` nul terminator.

Portability

`mbstowcs` is required in the ANSI C standard. However, the precise effects vary with the locale.

`mbstowcs` requires no supporting OS subroutines.

1.27 mbtowc—minimal multibyte to wide char converter

Synopsis

```
#include <stdlib.h>
int mbtowc(wchar_t *pwc, const char *s, size_t n);
```

Description

When `_MB_CAPABLE` is not defined, this is a minimal ANSI-conforming implementation of `mbtowc`. In this case, only “multi-byte character sequences” recognized are single bytes, and they are “converted” to themselves. Each call to `mbtowc` copies one character from `*s` to `*pwc`, unless `s` is a null pointer. The argument `n` is ignored.

When `_MB_CAPABLE` is defined, this routine calls `_mbtowc_r` to perform the conversion, passing a state variable to allow state dependent decoding. The result is based on the locale setting which may be restricted to a defined set of locales.

Returns

This implementation of `mbtowc` returns 0 if `s` is NULL or is the empty string; it returns 1 if not `_MB_CAPABLE` or the character is a single-byte character; it returns -1 if `n` is 0 or the multi-byte character is invalid; otherwise it returns the number of bytes in the multibyte character. If the return value is -1, no changes are made to the `pwc` output string. If the input is the empty string, a `wchar_t` nul is placed in the output string and 0 is returned. If the input has a length of 0, no changes are made to the `pwc` output string.

Portability

`mbtowc` is required in the ANSI C standard. However, the precise effects vary with the locale.

`mbtowc` requires no supporting OS subroutines.

1.28 `on_exit`—request execution of function with argument at program exit

Synopsis

```
#include <stdlib.h>
int on_exit (void (*function)(int, void *), void *arg);
```

Description

You can use `on_exit` to enroll functions in a list of functions that will be called when your program terminates normally. The argument is a pointer to a user-defined function which takes two arguments. The first is the status code passed to `exit` and the second argument is of type pointer to void. The function must not return a result. The value of *arg* is registered and passed as the argument to *function*.

The functions are kept in a LIFO stack; that is, the last function enrolled by `atexit` or `on_exit` will be the first to execute when your program exits. You can intermix functions using `atexit` and `on_exit`.

There is no built-in limit to the number of functions you can enroll in this list; however, after every group of 32 functions is enrolled, `atexit/on_exit` will call `malloc` to get space for the next part of the list. The initial list of 32 functions is statically allocated, so you can always count on at least that many slots available.

Returns

`on_exit` returns 0 if it succeeds in enrolling your function, -1 if it fails (possible only if no space was available for `malloc` to extend the list of functions).

Portability

`on_exit` is a non-standard glibc extension

Supporting OS subroutines required: None

1.29 rand, srand—pseudo-random numbers

Synopsis

```
#include <stdlib.h>
int rand(void);
void srand(unsigned int seed);
int rand_r(unsigned int *seed);
```

Description

rand returns a different integer each time it is called; each integer is chosen by an algorithm designed to be unpredictable, so that you can use **rand** when you require a random number. The algorithm depends on a static variable called the “random seed”; starting with a given value of the random seed always produces the same sequence of numbers in successive calls to **rand**.

You can set the random seed using **srand**; it does nothing beyond storing its argument in the static variable used by **rand**. You can exploit this to make the pseudo-random sequence less predictable, if you wish, by using some other unpredictable value (often the least significant parts of a time-varying value) as the random seed before beginning a sequence of calls to **rand**; or, if you wish to ensure (for example, while debugging) that successive runs of your program use the same “random” numbers, you can use **srand** to set the same random seed at the outset.

Returns

rand returns the next pseudo-random integer in sequence; it is a number between 0 and **RAND_MAX** (inclusive).

srand does not return a result.

Portability

rand is required by ANSI, but the algorithm for pseudo-random number generation is not specified; therefore, even if you use the same random seed, you cannot expect the same sequence of results on two different systems.

rand requires no supporting OS subroutines.

1.30 rand48, drand48, erand48, lrand48, nrand48, mrand48, jrand48, srand48, seed48, lcong48 —pseudo random number generators and initialization routines

Synopsis

```
#include <stdlib.h>
double drand48(void);
double erand48(unsigned short xseed[3]);
long lrand48(void);
long nrand48(unsigned short xseed[3]);
long mrand48(void);
long jrand48(unsigned short xseed[3]);
void srand48(long seed);
unsigned short *seed48(unsigned short xseed[3]);
void lcong48(unsigned short p[7]);
```

Description

The **rand48** family of functions generates pseudo-random numbers using a linear congruential algorithm working on integers 48 bits in size. The particular formula employed is $r(n+1) = (a * r(n) + c) \bmod m$ where the default values are for the multiplicand $a = 0xfdece66d = 25214903917$ and the addend $c = 0xb = 11$. The modulo is always fixed at $m = 2^{**} 48$. $r(n)$ is called the seed of the random number generator.

For all the six generator routines described next, the first computational step is to perform a single iteration of the algorithm.

drand48 and **erand48** return values of type `double`. The full 48 bits of $r(n+1)$ are loaded into the mantissa of the returned value, with the exponent set such that the values produced lie in the interval $[0.0, 1.0]$.

lrand48 and **nrand48** return values of type `long` in the range $[0, 2^{**}31-1]$. The high-order (31) bits of $r(n+1)$ are loaded into the lower bits of the returned value, with the topmost (sign) bit set to zero.

mrnd48 and **jrand48** return values of type `long` in the range $[-2^{**}31, 2^{**}31-1]$. The high-order (32) bits of $r(n+1)$ are loaded into the returned value.

drand48, **lrand48**, and **mrnd48** use an internal buffer to store $r(n)$. For these functions the initial value of $r(0) = 0x1234abcd330e = 20017429951246$.

On the other hand, **erand48**, **nrand48**, and **jrand48** use a user-supplied buffer to store the seed $r(n)$, which consists of an array of 3 shorts, where the zeroth member holds the least significant bits.

All functions share the same multiplicand and addend.

srand48 is used to initialize the internal buffer $r(n)$ of **drand48**, **lrand48**, and **mrnd48** such that the 32 bits of the seed value are copied into the upper 32 bits of $r(n)$, with the lower 16 bits of $r(n)$ arbitrarily being set to `0x330e`. Additionally, the constant multiplicand and addend of the algorithm are reset to the default values given above.

seed48 also initializes the internal buffer $r(n)$ of **drand48**, **lrand48**, and **mrnd48**, but here all 48 bits of the seed can be specified in an array of 3 shorts, where the zeroth member specifies the lowest bits. Again, the constant multiplicand and addend of the algorithm are reset to the default values given above. **seed48** returns a pointer to an array of 3 shorts

which contains the old seed. This array is statically allocated, thus its contents are lost after each new call to `seed48`.

Finally, `lcong48` allows full control over the multiplicand and addend used in `drand48`, `erand48`, `lrand48`, `rand48`, `mrnd48`, and `jrand48`, and the seed used in `drand48`, `lrand48`, and `mrnd48`. An array of 7 shorts is passed as parameter; the first three shorts are used to initialize the seed; the second three are used to initialize the multiplicand; and the last short is used to initialize the addend. It is thus not possible to use values greater than 0xffff as the addend.

Note that all three methods of seeding the random number generator always also set the multiplicand and addend for any of the six generator calls.

For a more powerful random number generator, see `random`.

Portability

SUS requires these functions.

No supporting OS subroutines are required.

1.31 strtod, strtodf—string to double or float

Synopsis

```
#include <stdlib.h>
double strtod(const char *str, char **tail);
float strtodf(const char *str, char **tail);

double _strtod_r(void *reent,
                 const char *str, char **tail);
```

Description

The function `strtod` parses the character string `str`, producing a substring which can be converted to a double value. The substring converted is the longest initial subsequence of `str`, beginning with the first non-whitespace character, that has the format:

```
[+|-]digits[.][digits][(e|E)[+|-]digits]
```

The substring contains no characters if `str` is empty, consists entirely of whitespace, or if the first non-whitespace character is something other than `+`, `-`, `.`, or a digit. If the substring is empty, no conversion is done, and the value of `str` is stored in `*tail`. Otherwise, the substring is converted, and a pointer to the final string (which will contain at least the terminating null character of `str`) is stored in `*tail`. If you want no assignment to `*tail`, pass a null pointer as `tail`. `strtodf` is identical to `strtod` except for its return type.

This implementation returns the nearest machine number to the input decimal string. Ties are broken by using the IEEE round-even rule.

The alternate function `_strtod_r` is a reentrant version. The extra argument `reent` is a pointer to a reentrancy structure.

Returns

`strtod` returns the converted substring value, if any. If no conversion could be performed, 0 is returned. If the correct value is out of the range of representable values, plus or minus `HUGE_VAL` is returned, and `ERANGE` is stored in `errno`. If the correct value would cause underflow, 0 is returned and `ERANGE` is stored in `errno`.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

1.32 `strtol`—string to long

Synopsis

```
#include <stdlib.h>
long strtol(const char *s, char **ptr, int base);

long _strtol_r(void *reent,
               const char *s, char **ptr, int base);
```

Description

The function `strtol` converts the string `*s` to a `long`. First, it breaks down the string into three parts: leading whitespace, which is ignored; a subject string consisting of characters resembling an integer in the radix specified by `base`; and a trailing portion consisting of zero or more unparseable characters, and always including the terminating null character. Then, it attempts to convert the subject string into a `long` and returns the result.

If the value of `base` is 0, the subject string is expected to look like a normal C integer constant: an optional sign, a possible '0x' indicating a hexadecimal base, and a number. If `base` is between 2 and 36, the expected form of the subject is a sequence of letters and digits representing an integer in the radix specified by `base`, with an optional plus or minus sign. The letters `a–z` (or, equivalently, `A–Z`) are used to signify values from 10 to 35; only letters whose ascribed values are less than `base` are permitted. If `base` is 16, a leading `0x` is permitted.

The subject sequence is the longest initial sequence of the input string that has the expected form, starting with the first non-whitespace character. If the string is empty or consists entirely of whitespace, or if the first non-whitespace character is not a permissible letter or digit, the subject string is empty.

If the subject string is acceptable, and the value of `base` is zero, `strtol` attempts to determine the radix from the input string. A string with a leading `0x` is treated as a hexadecimal value; a string with a leading 0 and no `x` is treated as octal; all other strings are treated as decimal. If `base` is between 2 and 36, it is used as the conversion radix, as described above. If the subject string begins with a minus sign, the value is negated. Finally, a pointer to the first character past the converted subject string is stored in `ptr`, if `ptr` is not NULL.

If the subject string is empty (or not in acceptable form), no conversion is performed and the value of `s` is stored in `ptr` (if `ptr` is not NULL).

The alternate function `_strtol_r` is a reentrant version. The extra argument `reent` is a pointer to a reentrancy structure.

Returns

`strtol` returns the converted value, if any. If no conversion was made, 0 is returned.

`strtol` returns `LONG_MAX` or `LONG_MIN` if the magnitude of the converted value is too large, and sets `errno` to `ERANGE`.

Portability

`strtol` is ANSI.

No supporting OS subroutines are required.

1.33 strtoll—string to long long

Synopsis

```
#include <stdlib.h>

long long strtoll(const char *s, char **ptr,int base);

long long _strtoll_r(void *reent,
    const char *s, char **ptr,int base);
```

Description

The function `strtoll` converts the string `*s` to a `long long`. First, it breaks down the string into three parts: leading whitespace, which is ignored; a subject string consisting of characters resembling an integer in the radix specified by `base`; and a trailing portion consisting of zero or more unparseable characters, and always including the terminating null character. Then, it attempts to convert the subject string into a `long long` and returns the result.

If the value of `base` is 0, the subject string is expected to look like a normal C integer constant: an optional sign, a possible '0x' indicating a hexadecimal base, and a number. If `base` is between 2 and 36, the expected form of the subject is a sequence of letters and digits representing an integer in the radix specified by `base`, with an optional plus or minus sign. The letters `a–z` (or, equivalently, `A–Z`) are used to signify values from 10 to 35; only letters whose ascribed values are less than `base` are permitted. If `base` is 16, a leading 0x is permitted.

The subject sequence is the longest initial sequence of the input string that has the expected form, starting with the first non-whitespace character. If the string is empty or consists entirely of whitespace, or if the first non-whitespace character is not a permissible letter or digit, the subject string is empty.

If the subject string is acceptable, and the value of `base` is zero, `strtoll` attempts to determine the radix from the input string. A string with a leading 0x is treated as a hexadecimal value; a string with a leading 0 and no x is treated as octal; all other strings are treated as decimal. If `base` is between 2 and 36, it is used as the conversion radix, as described above. If the subject string begins with a minus sign, the value is negated. Finally, a pointer to the first character past the converted subject string is stored in `ptr`, if `ptr` is not NULL.

If the subject string is empty (or not in acceptable form), no conversion is performed and the value of `s` is stored in `ptr` (if `ptr` is not NULL).

The alternate function `_strtoll_r` is a reentrant version. The extra argument `reent` is a pointer to a reentrancy structure.

Returns

`strtoll` returns the converted value, if any. If no conversion was made, 0 is returned.

`strtoll` returns `LONG_LONG_MAX` or `LONG_LONG_MIN` if the magnitude of the converted value is too large, and sets `errno` to `ERANGE`.

Portability

`strtoll` is nonstandard.

No supporting OS subroutines are required.

1.34 strtoul—string to unsigned long

Synopsis

```
#include <stdlib.h>

unsigned long strtoul(const char *s, char **ptr,
                     int base);

unsigned long _strtoul_r(void *reent, const char *s,
                        char **ptr, int base);
```

Description

The function `strtoul` converts the string `*s` to an `unsigned long`. First, it breaks down the string into three parts: leading whitespace, which is ignored; a subject string consisting of the digits meaningful in the radix specified by `base` (for example, 0 through 7 if the value of `base` is 8); and a trailing portion consisting of one or more unparseable characters, which always includes the terminating null character. Then, it attempts to convert the subject string into an unsigned long integer, and returns the result.

If the value of `base` is zero, the subject string is expected to look like a normal C integer constant (save that no optional sign is permitted): a possible `0x` indicating hexadecimal radix, and a number. If `base` is between 2 and 36, the expected form of the subject is a sequence of digits (which may include letters, depending on the base) representing an integer in the radix specified by `base`. The letters `a–z` (or `A–Z`) are used as digits valued from 10 to 35. If `base` is 16, a leading `0x` is permitted.

The subject sequence is the longest initial sequence of the input string that has the expected form, starting with the first non-whitespace character. If the string is empty or consists entirely of whitespace, or if the first non-whitespace character is not a permissible digit, the subject string is empty.

If the subject string is acceptable, and the value of `base` is zero, `strtoul` attempts to determine the radix from the input string. A string with a leading `0x` is treated as a hexadecimal value; a string with a leading 0 and no `x` is treated as octal; all other strings are treated as decimal. If `base` is between 2 and 36, it is used as the conversion radix, as described above. Finally, a pointer to the first character past the converted subject string is stored in `ptr`, if `ptr` is not `NULL`.

If the subject string is empty (that is, if `*s` does not start with a substring in acceptable form), no conversion is performed and the value of `s` is stored in `ptr` (if `ptr` is not `NULL`).

The alternate function `_strtoul_r` is a reentrant version. The extra argument `reent` is a pointer to a reentrancy structure.

Returns

`strtoul` returns the converted value, if any. If no conversion was made, 0 is returned.

`strtoul` returns `ULONG_MAX` if the magnitude of the converted value is too large, and sets `errno` to `ERANGE`.

Portability

`strtoul` is ANSI.

`strtoul` requires no supporting OS subroutines.

1.35 strtoull—string to unsigned long long

Synopsis

```
#include <stdlib.h>

unsigned long long strtoull(const char *s, char **ptr,
                           int base);

unsigned long long _strtoull_r(void *reent, const char *s,
                              char **ptr, int base);
```

Description

The function `strtoull` converts the string `*s` to an `unsigned long long`. First, it breaks down the string into three parts: leading whitespace, which is ignored; a subject string consisting of the digits meaningful in the radix specified by `base` (for example, 0 through 7 if the value of `base` is 8); and a trailing portion consisting of one or more unparseable characters, which always includes the terminating null character. Then, it attempts to convert the subject string into an unsigned long long integer, and returns the result.

If the value of `base` is zero, the subject string is expected to look like a normal C integer constant (save that no optional sign is permitted): a possible `0x` indicating hexadecimal radix, and a number. If `base` is between 2 and 36, the expected form of the subject is a sequence of digits (which may include letters, depending on the base) representing an integer in the radix specified by `base`. The letters `a–z` (or `A–Z`) are used as digits valued from 10 to 35. If `base` is 16, a leading `0x` is permitted.

The subject sequence is the longest initial sequence of the input string that has the expected form, starting with the first non-whitespace character. If the string is empty or consists entirely of whitespace, or if the first non-whitespace character is not a permissible digit, the subject string is empty.

If the subject string is acceptable, and the value of `base` is zero, `strtoull` attempts to determine the radix from the input string. A string with a leading `0x` is treated as a hexadecimal value; a string with a leading 0 and no `x` is treated as octal; all other strings are treated as decimal. If `base` is between 2 and 36, it is used as the conversion radix, as described above. Finally, a pointer to the first character past the converted subject string is stored in `ptr`, if `ptr` is not NULL.

If the subject string is empty (that is, if `*s` does not start with a substring in acceptable form), no conversion is performed and the value of `s` is stored in `ptr` (if `ptr` is not NULL).

The alternate function `_strtoull_r` is a reentrant version. The extra argument `reent` is a pointer to a reentrancy structure.

Returns

`strtoull` returns the converted value, if any. If no conversion was made, 0 is returned.

`strtoull` returns `ULONG_LONG_MAX` if the magnitude of the converted value is too large, and sets `errno` to `ERANGE`.

Portability

`strtoull` is nonstandard.

`strtoull` requires no supporting OS subroutines.

1.36 `system`—execute command string

Synopsis

```
#include <stdlib.h>
int system(char *s);

int _system_r(void *reent, char *s);
```

Description

Use `system` to pass a command string `*s` to `/bin/sh` on your system, and wait for it to finish executing.

Use “`system(NULL)`” to test whether your system has `/bin/sh` available.

The alternate function `_system_r` is a reentrant version. The extra argument `reent` is a pointer to a reentrancy structure.

Returns

`system(NULL)` returns a non-zero value if `/bin/sh` is available, and 0 if it is not.

With a command argument, the result of `system` is the exit status returned by `/bin/sh`.

Portability

ANSI C requires `system`, but leaves the nature and effects of a command processor undefined. ANSI C does, however, specify that `system(NULL)` return zero or nonzero to report on the existence of a command processor.

POSIX.2 requires `system`, and requires that it invoke a `sh`. Where `sh` is found is left unspecified.

Supporting OS subroutines required: `_exit`, `_execve`, `_fork_r`, `_wait_r`.

1.37 `wcstombs`—minimal wide char string to multibyte string converter

Synopsis

```
#include <stdlib.h>
int wcstombs(const char *s, wchar_t *pwc, size_t n);
```

Description

When `_MB_CAPABLE` is not defined, this is a minimal ANSI-conforming implementation of `wcstombs`. In this case, all wide-characters are expected to represent single bytes and so are converted simply by casting to `char`.

When `_MB_CAPABLE` is defined, this routine calls `_wcstombs_r` to perform the conversion, passing a state variable to allow state dependent decoding. The result is based on the locale setting which may be restricted to a defined set of locales.

Returns

This implementation of `wcstombs` returns 0 if `s` is `NULL` or is the empty string; it returns -1 if `_MB_CAPABLE` and one of the wide-char characters does not represent a valid multi-byte character; otherwise it returns the minimum of: `n` or the number of bytes that are transferred to `s`, not including the nul terminator.

If the return value is -1, the state of the `pwc` string is indeterminate. If the input has a length of 0, the output string will be modified to contain a `wchar_t` nul terminator if `n > 0`.

Portability

`wcstombs` is required in the ANSI C standard. However, the precise effects vary with the locale.

`wcstombs` requires no supporting OS subroutines.

1.38 wctomb—minimal wide char to multibyte converter

Synopsis

```
#include <stdlib.h>
int wctomb(char *s, wchar_t wchar);
```

Description

When `_MB_CAPABLE` is not defined, this is a minimal ANSI-conforming implementation of `wctomb`. The only “wide characters” recognized are single bytes, and they are “converted” to themselves.

When `_MB_CAPABLE` is defined, this routine calls `_wctomb_r` to perform the conversion, passing a state variable to allow state dependent decoding. The result is based on the locale setting which may be restricted to a defined set of locales.

Each call to `wctomb` modifies `*s` unless `s` is a null pointer or `_MB_CAPABLE` is defined and `wchar` is invalid.

Returns

This implementation of `wctomb` returns 0 if `s` is NULL; it returns -1 if `_MB_CAPABLE` is enabled and the `wchar` is not a valid multi-byte character, it returns 1 if `_MB_CAPABLE` is not defined or the `wchar` is in reality a single byte character, otherwise it returns the number of bytes in the multi-byte character.

Portability

`wctomb` is required in the ANSI C standard. However, the precise effects vary with the locale.

`wctomb` requires no supporting OS subroutines.

2 Character Type Macros and Functions (`'ctype.h'`)

This chapter groups macros (which are also available as subroutines) to classify characters into several categories (alphabetic, numeric, control characters, whitespace, and so on), or to perform simple character mappings.

The header file `'ctype.h'` defines the macros.

2.1 isalnum—alphanumeric character predicate

Synopsis

```
#include <ctype.h>
int isalnum(int c);
```

Description

`isalnum` is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for alphabetic or numeric ASCII characters, and 0 for other arguments. It is defined for all integer values.

You can use a compiled subroutine instead of the macro definition by undefining the macro using `#undef isalnum`.

Returns

`isalnum` returns non-zero if `c` is a letter (`a–z` or `A–Z`) or a digit (`0–9`).

Portability

`isalnum` is ANSI C.

No OS subroutines are required.

2.2 isalpha—alphabetic character predicate

Synopsis

```
#include <ctype.h>
int isalpha(int c);
```

Description

isalpha is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero when *c* represents an alphabetic ASCII character, and 0 otherwise. It is defined only when **isascii**(*c*) is true or *c* is EOF.

You can use a compiled subroutine instead of the macro definition by undefining the macro using '#undef isalpha'.

Returns

isalpha returns non-zero if *c* is a letter (A–Z or a–z).

Portability

isalpha is ANSI C.

No supporting OS subroutines are required.

2.3 `isascii`—ASCII character predicate

Synopsis

```
#include <ctype.h>
int isascii(int c);
```

Description

`isascii` is a macro which returns non-zero when *c* is an ASCII character, and 0 otherwise. It is defined for all integer values.

You can use a compiled subroutine instead of the macro definition by undefining the macro using ‘`#undef isascii`’.

Returns

`isascii` returns non-zero if the low order byte of *c* is in the range 0 to 127 (0x00–0x7F).

Portability

`isascii` is ANSI C.

No supporting OS subroutines are required.

2.4 iscntrl—control character predicate

Synopsis

```
#include <ctype.h>
int iscntrl(int c);
```

Description

`iscntrl` is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for control characters, and 0 for other characters. It is defined only when `isascii(c)` is true or `c` is EOF.

You can use a compiled subroutine instead of the macro definition by undefining the macro using `#undef iscntrl`.

Returns

`iscntrl` returns non-zero if `c` is a delete character or ordinary control character (0x7F or 0x00–0x1F).

Portability

`iscntrl` is ANSI C.

No supporting OS subroutines are required.

2.5 isdigit—decimal digit predicate

Synopsis

```
#include <ctype.h>
int isdigit(int c);
```

Description

`isdigit` is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for decimal digits, and 0 for other characters. It is defined only when `isascii(c)` is true or `c` is EOF.

You can use a compiled subroutine instead of the macro definition by undefining the macro using `#undef isdigit`.

Returns

`isdigit` returns non-zero if `c` is a decimal digit (0–9).

Portability

`isdigit` is ANSI C.

No supporting OS subroutines are required.

2.6 islower—lower-case character predicate

Synopsis

```
#include <ctype.h>
int islower(int c);
```

Description

islower is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for minuscules (lower-case alphabetic characters), and 0 for other characters. It is defined only when **isascii**(*c*) is true or *c* is EOF.

You can use a compiled subroutine instead of the macro definition by undefining the macro using '#undef **islower**'.

Returns

islower returns non-zero if *c* is a lower case letter (a–z).

Portability

islower is ANSI C.

No supporting OS subroutines are required.

2.7 isprint, isgraph—printable character predicates

Synopsis

```
#include <ctype.h>
int isprint(int c);
int isgraph(int c);
```

Description

isprint is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for printable characters, and 0 for other character arguments. It is defined only when **isascii**(*c*) is true or *c* is EOF.

You can use a compiled subroutine instead of the macro definition by undefining either macro using `#undef isprint` or `#undef isgraph`.

Returns

isprint returns non-zero if *c* is a printing character, (0x20–0x7E). **isgraph** behaves identically to **isprint**, except that the space character (0x20) is excluded.

Portability

isprint and **isgraph** are ANSI C.

No supporting OS subroutines are required.

2.8 ispunct—punctuation character predicate

Synopsis

```
#include <ctype.h>
int ispunct(int c);
```

Description

`ispunct` is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for printable punctuation characters, and 0 for other characters. It is defined only when `isascii(c)` is true or `c` is EOF.

You can use a compiled subroutine instead of the macro definition by undefining the macro using `#undef ispunct`.

Returns

`ispunct` returns non-zero if `c` is a printable punctuation character (`isgraph(c) && !isalnum(c)`).

Portability

`ispunct` is ANSI C.

No supporting OS subroutines are required.

2.9 isspace—whitespace character predicate

Synopsis

```
#include <ctype.h>
int isspace(int c);
```

Description

`isspace` is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for whitespace characters, and 0 for other characters. It is defined only when `isascii(c)` is true or *c* is EOF.

You can use a compiled subroutine instead of the macro definition by undefining the macro using `#undef isspace`.

Returns

`isspace` returns non-zero if *c* is a space, tab, carriage return, new line, vertical tab, or formfeed (0x09–0x0D, 0x20).

Portability

`isspace` is ANSI C.

No supporting OS subroutines are required.

2.10 isupper—uppercase character predicate

Synopsis

```
#include <ctype.h>
int isupper(int c);
```

Description

isupper is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for upper-case letters (A–Z), and 0 for other characters. It is defined only when **isascii**(*c*) is true or *c* is EOF.

You can use a compiled subroutine instead of the macro definition by undefining the macro using '#undef isupper'.

Returns

isupper returns non-zero if *c* is a upper case letter (A–Z).

Portability

isupper is ANSI C.

No supporting OS subroutines are required.

2.11 isxdigit—hexadecimal digit predicate

Synopsis

```
#include <ctype.h>
int isxdigit(int c);
```

Description

`isxdigit` is a macro which classifies ASCII integer values by table lookup. It is a predicate returning non-zero for hexadecimal digits, and 0 for other characters. It is defined only when `isascii(c)` is true or *c* is EOF.

You can use a compiled subroutine instead of the macro definition by undefining the macro using `#undef isxdigit`.

Returns

`isxdigit` returns non-zero if *c* is a hexadecimal digit (0–9, a–f, or A–F).

Portability

`isxdigit` is ANSI C.

No supporting OS subroutines are required.

2.12 toascii—force integers to ASCII range

Synopsis

```
#include <ctype.h>
int toascii(int c);
```

Description

`toascii` is a macro which coerces integers to the ASCII range (0–127) by zeroing any higher-order bits.

You can use a compiled subroutine instead of the macro definition by undefining this macro using `#undef toascii`.

Returns

`toascii` returns integers between 0 and 127.

Portability

`toascii` is not ANSI C.

No supporting OS subroutines are required.

2.13 tolower—translate characters to lower case

Synopsis

```
#include <ctype.h>
int tolower(int c);
int _tolower(int c);
```

Description

`tolower` is a macro which converts upper-case characters to lower case, leaving all other characters unchanged. It is only defined when `c` is an integer in the range EOF to 255.

You can use a compiled subroutine instead of the macro definition by undefining this macro using `#undef tolower`.

`_tolower` performs the same conversion as `tolower`, but should only be used when `c` is known to be an uppercase character (A–Z).

Returns

`tolower` returns the lower-case equivalent of `c` when it is a character between A and Z, and `c` otherwise.

`_tolower` returns the lower-case equivalent of `c` when it is a character between A and Z. If `c` is not one of these characters, the behaviour of `_tolower` is undefined.

Portability

`tolower` is ANSI C. `_tolower` is not recommended for portable programs.

No supporting OS subroutines are required.

2.14 toupper—translate characters to upper case

Synopsis

```
#include <ctype.h>
int toupper(int c);
int _toupper(int c);
```

Description

toupper is a macro which converts lower-case characters to upper case, leaving all other characters unchanged. It is only defined when *c* is an integer in the range EOF to 255.

You can use a compiled subroutine instead of the macro definition by undefining this macro using '#undef toupper'.

_toupper performs the same conversion as **toupper**, but should only be used when *c* is known to be a lowercase character (a–z).

Returns

toupper returns the upper-case equivalent of *c* when it is a character between **a** and **z**, and *c* otherwise.

_toupper returns the upper-case equivalent of *c* when it is a character between **a** and **z**. If *c* is not one of these characters, the behaviour of **_toupper** is undefined.

Portability

toupper is ANSI C. **_toupper** is not recommended for portable programs.

No supporting OS subroutines are required.

2.15 iswalnum—alpha-numeric wide-character test

Synopsis

```
#include <wctype.h>
int iswalnum(wint_t c);
```

Description

`iswalnum` is a function which classifies wide-character values that are alpha-numeric.

Returns

`iswalnum` returns non-zero if `c` is a alpha-numeric wide-character.

Portability

`iswalnum` is C99.

No supporting OS subroutines are required.

2.16 iswalpha—alphabetic wide-character test

Synopsis

```
#include <wctype.h>
int iswalpha(wint_t c);
```

Description

`iswalpha` is a function which classifies wide-character values that are alphabetic.

Returns

`iswalpha` returns non-zero if *c* is an alphabetic wide-character.

Portability

`iswalpha` is C99.

No supporting OS subroutines are required.

2.17 iswcntrl—wide-character cntrl test

Synopsis

```
#include <wctype.h>
int iswcntrl(wint_t c);
```

Description

`iswcntrl` is a function which classifies wide-character values that are categorized as a control character.

Returns

`iswcntrl` returns non-zero if *c* is a control wide-character.

Portability

`iswcntrl` is C99.

No supporting OS subroutines are required.

2.18 `iswblank`—wide-character blank test

Synopsis

```
#include <wctype.h>
int iswblank(wint_t c);
```

Description

`iswblank` is a function which classifies wide-character values that are categorized as blank.

Returns

`iswblank` returns non-zero if *c* is a blank wide-character.

Portability

`iswblank` is C99.

No supporting OS subroutines are required.

2.19 iswdigit—decimal digit wide-character test

Synopsis

```
#include <wctype.h>
int iswdigit(wint_t c);
```

Description

`iswdigit` is a function which classifies wide-character values that are decimal digits.

Returns

`iswdigit` returns non-zero if *c* is a decimal digit wide-character.

Portability

`iswdigit` is C99.

No supporting OS subroutines are required.

2.20 iswgraph—graphic wide-character test

Synopsis

```
#include <wctype.h>
int iswgraph(wint_t c);
```

Description

`iswgraph` is a function which classifies wide-character values that are graphic.

Returns

`iswgraph` returns non-zero if *c* is a graphic wide-character.

Portability

`iswgraph` is C99.

No supporting OS subroutines are required.

2.21 iswlower—lower-case wide-character test

Synopsis

```
#include <wctype.h>
int iswlower(wint_t c);
```

Description

`iswlower` is a function which classifies wide-character values that have an upper-case translation.

Returns

`iswlower` returns non-zero if *c* is a lower-case wide-character.

Portability

`iswlower` is C99.

No supporting OS subroutines are required.

2.22 iswprint—printable wide-character test

Synopsis

```
#include <wctype.h>
int iswprint(wint_t c);
```

Description

`iswprint` is a function which classifies wide-character values that are printable.

Returns

`iswprint` returns non-zero if *c* is a printable wide-character.

Portability

`iswprint` is C99.

No supporting OS subroutines are required.

2.23 iswpunct—punctuation wide-character test

Synopsis

```
#include <wctype.h>
int iswpunct(wint_t c);
```

Description

`iswpunct` is a function which classifies wide-character values that are punctuation.

Returns

`iswpunct` returns non-zero if *c* is a punctuation wide-character.

Portability

`iswpunct` is C99.

No supporting OS subroutines are required.

2.24 iswspace—wide-character space test

Synopsis

```
#include <wctype.h>
int iswspace(wint_t c);
```

Description

`iswspace` is a function which classifies wide-character values that are categorized as white-space.

Returns

`iswspace` returns non-zero if *c* is a white-space wide-character.

Portability

`iswspace` is C99.

No supporting OS subroutines are required.

2.25 iswupper—upper-case wide-character test

Synopsis

```
#include <wctype.h>
int iswupper(wint_t c);
```

Description

`iswupper` is a function which classifies wide-character values that have an upper-case translation.

Returns

`iswupper` returns non-zero if *c* is a upper-case wide-character.

Portability

`iswupper` is C99.

No supporting OS subroutines are required.

2.26 iswxdigit—hexadecimal digit wide-character test

Synopsis

```
#include <wctype.h>
int iswxdigit(wint_t c);
```

Description

`iswxdigit` is a function which classifies wide-character values that are hexadecimal digits.

Returns

`iswxdigit` returns non-zero if *c* is a hexadecimal digit wide-character.

Portability

`iswxdigit` is C99.

No supporting OS subroutines are required.

2.27 iswctype—extensible wide-character test

Synopsis

```
#include <wctype.h>
int iswctype(wint_t c, wctype_t desc);
```

Description

iswctype is a function which classifies wide-character values using the wide-character test specified by *desc*.

Returns

iswctype returns non-zero if and only if *c* matches the test specified by *desc*. If *desc* is unknown, zero is returned.

Portability

iswctype is C99.

No supporting OS subroutines are required.

2.28 wctype—get wide-character classification type

Synopsis

```
#include <wctype.h>
wctype_t wctype(const char *c);
```

Description

wctype is a function which takes a string *c* and gives back the appropriate `wctype_t` type value associated with the string, if one exists. The following values are guaranteed to be recognized: "alnum", "alpha", "blank", "cntrl", "digit", "graph", "lower", "print", "punct", "space", "upper", and "xdigit".

Returns

wctype returns 0 and sets `errno` to `EINVAL` if the given name is invalid. Otherwise, it returns a valid non-zero `wctype_t` value.

Portability

wctype is C99.

No supporting OS subroutines are required.

2.29 `towlower`—translate wide-characters to lower case

Synopsis

```
#include <wctype.h>
wint_t tolower(wint_t c);
```

Description

`towlower` is a function which converts upper-case wide-characters to lower case, leaving all other characters unchanged.

Returns

`towlower` returns the lower-case equivalent of `c` when it is a upper-case wide-character, otherwise, it returns the input character.

Portability

`towlower` is C99.

No supporting OS subroutines are required.

2.30 towupper—translate wide-characters to upper case

Synopsis

```
#include <wctype.h>
wint_t towupper(wint_t c);
```

Description

towupper is a function which converts lower-case wide-characters to upper case, leaving all other characters unchanged.

Returns

towupper returns the upper-case equivalent of *c* when it is a lower-case wide-character, otherwise, it returns the input character.

Portability

towupper is C99.

No supporting OS subroutines are required.

2.31 towctrans—extensible wide-character case mapping

Synopsis

```
#include <wctype.h>
wint_t towctrans(wint_t c, wctrans_t w);
```

Description

towctrans is a function which converts wide-characters based on a specified translation type *w*. If the translation type is invalid or cannot be applied to the current character, no change to the character is made.

Returns

towctrans returns the translated equivalent of *c* when it is a valid for the given translation, otherwise, it returns the input character. When the translation type is invalid, **errno** is set **EINVAL**.

Portability

towctrans is C99.

No supporting OS subroutines are required.

2.32 wctrans—get wide-character translation type

Synopsis

```
#include <wctype.h>
wctrans_t wctrans(const char *c);
```

Description

wctrans is a function which takes a string *c* and gives back the appropriate `wctrans_t` type value associated with the string, if one exists. The following values are guaranteed to be recognized: "tolower" and "toupper".

Returns

wctrans returns 0 and sets `errno` to `EINVAL` if the given name is invalid. Otherwise, it returns a valid non-zero `wctrans_t` value.

Portability

wctrans is C99.

No supporting OS subroutines are required.

3 Input and Output (`'stdio.h'`)

This chapter comprises functions to manage files or other input/output streams. Among these functions are subroutines to generate or scan strings according to specifications from a format string.

The underlying facilities for input and output depend on the host system, but these functions provide a uniform interface.

The corresponding declarations are in `'stdio.h'`.

3.1 `clearerr`—clear file or stream error indicator

Synopsis

```
#include <stdio.h>
void clearerr(FILE *fp);
```

Description

The `stdio` functions maintain an error indicator with each file pointer *fp*, to record whether any read or write errors have occurred on the associated file or stream. Similarly, it maintains an end-of-file indicator to record whether there is no more data in the file.

Use `clearerr` to reset both of these indicators.

See `ferror` and `feof` to query the two indicators.

Returns

`clearerr` does not return a result.

Portability

ANSI C requires `clearerr`.

No supporting OS subroutines are required.

3.2 fclose—close a file

Synopsis

```
#include <stdio.h>
int fclose(FILE *fp);
int _fclose_r(struct _reent *reent, FILE *fp);
```

Description

If the file or stream identified by *fp* is open, **fclose** closes it, after first ensuring that any pending data is written (by calling **fflush**(*fp*)).

The alternate function **_fclose_r** is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

fclose returns 0 if successful (including when *fp* is NULL or not an open file); otherwise, it returns EOF.

Portability

fclose is required by ANSI C.

Required OS subroutines: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

3.3 `fcloseall`—close all files

Synopsis

```
#include <stdio.h>
int fcloseall(void);
int _fcloseall_r (struct _reent *ptr);
```

Description

`fcloseall` closes all files in the current reentrancy struct's domain. The function `_fcloseall_r` is the same function, except the reentrancy struct is passed in as the *ptr* argument.

This function is not recommended as it closes all streams, including the std streams.

Returns

`fclose` returns 0 if all closes are successful. Otherwise, EOF is returned.

Portability

`fcloseall` is a glibc extension.

Required OS subroutines: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.4 feof—test for end of file

Synopsis

```
#include <stdio.h>
int feof(FILE *fp);
```

Description

`feof` tests whether or not the end of the file identified by *fp* has been reached.

Returns

`feof` returns 0 if the end of file has not yet been reached; if at end of file, the result is nonzero.

Portability

`feof` is required by ANSI C.

No supporting OS subroutines are required.

3.5 `ferror`—test whether read/write error has occurred

Synopsis

```
#include <stdio.h>
int ferror(FILE *fp);
```

Description

The `stdio` functions maintain an error indicator with each file pointer *fp*, to record whether any read or write errors have occurred on the associated file or stream. Use `ferror` to query this indicator.

See `clearerr` to reset the error indicator.

Returns

`ferror` returns 0 if no errors have occurred; it returns a nonzero value otherwise.

Portability

ANSI C requires `ferror`.

No supporting OS subroutines are required.

3.6 fflush—flush buffered file output

Synopsis

```
#include <stdio.h>
int fflush(FILE *fp);
```

Description

The `stdio` output functions can buffer output before delivering it to the host system, in order to minimize the overhead of system calls.

Use `fflush` to deliver any such pending output (for the file or stream identified by *fp*) to the host system.

If *fp* is `NULL`, `fflush` delivers pending output from all open files.

Returns

`fflush` returns 0 unless it encounters a write error; in that situation, it returns `EOF`.

Portability

ANSI C requires `fflush`.

No supporting OS subroutines are required.

3.7 flockfile, funlockfile, ftrylockfile—lock and unlock a stream

Synopsis

```
#include <stdio.h>
void flockfile(FILE *);
int ftrylockfile(FILE *);
void funlockfile(FILE *);
```

Description

flockfile locks a stream so that only one thread can write to it. **funlockfile** unlocks a locked stream. **ftrylockfile** attempts to lock a stream and returns zero for success and non-zero for failure. Most stdio routines lock the file descriptor internally; these routines are provided so that users can group together a sequence of I/O operations so that another thread can't interleave with it, or to increase efficiency, in which case the user can use **getc_unlocked** or **putc_unlocked** instead of the usual locking **getc** and **putc** routines.

Portability

flockfile et al conform to POSIX.1.

3.8 fgetc—get a character from a file or stream

Synopsis

```
#include <stdio.h>
int fgetc(FILE *fp);
```

Description

Use **fgetc** to get the next single character from the file or stream identified by *fp*. As a side effect, **fgetc** advances the file's current position indicator.

For a macro version of this function, see **getc**.

Returns

The next character (read as an **unsigned char**, and cast to **int**), unless there is no more data, or the host system reports a read error; in either of these situations, **fgetc** returns EOF.

You can distinguish the two situations that cause an EOF result by using the **ferror** and **feof** functions.

Portability

ANSI C requires **fgetc**.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

3.9 fgetpos—record position in a stream or file

Synopsis

```
#include <stdio.h>
int fgetpos(FILE *fp, fpos_t *pos);
int _fgetpos_r(struct _reent *ptr, FILE *fp, fpos_t *pos);
```

Description

Objects of type `FILE` can have a “position” that records how much of the file your program has already read. Many of the `stdio` functions depend on this position, and many change it as a side effect.

You can use `fgetpos` to report on the current position for a file identified by *fp*; `fgetpos` will write a value representing that position at **pos*. Later, you can use this value with `fsetpos` to return the file to this position.

In the current implementation, `fgetpos` simply uses a character count to represent the file position; this is the same number that would be returned by `ftell`.

Returns

`fgetpos` returns 0 when successful. If `fgetpos` fails, the result is 1. Failure occurs on streams that do not support positioning; the global `errno` indicates this condition with the value `ESPIPE`.

Portability

`fgetpos` is required by the ANSI C standard, but the meaning of the value it records is not specified beyond requiring that it be acceptable as an argument to `fsetpos`. In particular, other conforming C implementations may return a different result from `ftell` than what `fgetpos` writes at **pos*.

No supporting OS subroutines are required.

3.10 fgets—get character string from a file or stream

Synopsis

```
#include <stdio.h>
char *fgets(char *buf, int n, FILE *fp);
```

Description

Reads at most $n-1$ characters from *fp* until a newline is found. The characters including to the newline are stored in *buf*. The buffer is terminated with a 0.

Returns

fgets returns the buffer passed to it, with the data filled in. If end of file occurs with some data already accumulated, the data is returned with no other indication. If no data are read, NULL is returned instead.

Portability

fgets should replace all uses of **gets**. Note however that **fgets** returns all of the data, while **gets** removes the trailing newline (with no indication that it has done so.)

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

3.11 `fileno`—return file descriptor associated with stream

Synopsis

```
#include <stdio.h>
int fileno(FILE *fp);
```

Description

You can use `fileno` to return the file descriptor identified by *fp*.

Returns

`fileno` returns a non-negative integer when successful. If *fp* is not an open stream, `fileno` returns -1.

Portability

`fileno` is not part of ANSI C. POSIX requires `fileno`.

Supporting OS subroutines required: none.

3.12 fopen—open a file

Synopsis

```
#include <stdio.h>
FILE *fopen(const char *file, const char *mode);

FILE *_fopen_r(struct _reent *reent,
               const char *file, const char *mode);
```

Description

fopen initializes the data structures needed to read or write a file. Specify the file's name as the string at *file*, and the kind of access you need to the file with the string at *mode*.

The alternate function **_fopen_r** is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Three fundamental kinds of access are available: read, write, and append. **mode* must begin with one of the three characters 'r', 'w', or 'a', to select one of these:

- r** Open the file for reading; the operation will fail if the file does not exist, or if the host system does not permit you to read it.
- w** Open the file for writing *from the beginning* of the file: effectively, this always creates a new file. If the file whose name you specified already existed, its old contents are discarded.
- a** Open the file for appending data, that is writing from the end of file. When you open a file this way, all data always goes to the current end of file; you cannot change this using **fseek**.

Some host systems distinguish between “binary” and “text” files. Such systems may perform data transformations on data written to, or read from, files opened as “text”. If your system is one of these, then you can append a 'b' to any of the three modes above, to specify that you are opening the file as a binary file (the default is to open the file as a text file).

'rb', then, means “read binary”; 'wb', “write binary”; and 'ab', “append binary”.

To make C programs more portable, the 'b' is accepted on all systems, whether or not it makes a difference.

Finally, you might need to both read and write from the same file. You can also append a '+' to any of the three modes, to permit this. (If you want to append both 'b' and '+', you can do it in either order: for example, "rb+" means the same thing as "r+b" when used as a mode string.)

Use "r+" (or "rb+") to permit reading and writing anywhere in an existing file, without discarding any data; "w+" (or "wb+") to create a new file (or begin by discarding all data from an old one) that permits reading and writing anywhere in it; and "a+" (or "ab+") to permit reading anywhere in an existing file, but writing only at the end.

Returns

fopen returns a file pointer which you can use for other file operations, unless the file you requested could not be opened; in that situation, the result is NULL. If the reason for failure was an invalid string at *mode*, **errno** is set to EINVAL.

Portability

`fopen` is required by ANSI C.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `open`, `read`, `sbrk`, `write`.

3.13 fdopen—turn open file into a stream

Synopsis

```
#include <stdio.h>
FILE *fdopen(int fd, const char *mode);
FILE *_fdopen_r(struct _reent *reent,
                int fd, const char *mode);
```

Description

fdopen produces a file descriptor of type `FILE *`, from a descriptor for an already-open file (returned, for example, by the system subroutine **open** rather than by **fopen**). The *mode* argument has the same meanings as in **fopen**.

Returns

File pointer or `NULL`, as for **fopen**.

Portability

fdopen is ANSI.

3.14 `fputc`—write a character on a stream or file

Synopsis

```
#include <stdio.h>
int fputc(int ch, FILE *fp);
```

Description

`fputc` converts the argument *ch* from an `int` to an unsigned `char`, then writes it to the file or stream identified by *fp*.

If the file was opened with append mode (or if the stream cannot support positioning), then the new character goes at the end of the file or stream. Otherwise, the new character is written at the current value of the position indicator, and the position indicator advances by one.

For a macro version of this function, see `putc`.

Returns

If successful, `fputc` returns its argument *ch*. If an error intervenes, the result is `EOF`. You can use ‘`ferror(fp)`’ to query for errors.

Portability

`fputc` is required by ANSI C.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.15 fputs—write a character string in a file or stream

Synopsis

```
#include <stdio.h>
int fputs(const char *s, FILE *fp);
```

Description

fputs writes the string at *s* (but without the trailing null) to the file or stream identified by *fp*.

Returns

If successful, the result is 0; otherwise, the result is EOF.

Portability

ANSI C requires **fputs**, but does not specify that the result on success must be 0; any non-negative value is permitted.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

3.16 fread—read array elements from a file

Synopsis

```
#include <stdio.h>
size_t fread(void *buf, size_t size, size_t count,
             FILE *fp);
```

Description

fread attempts to copy, from the file or stream identified by *fp*, *count* elements (each of size *size*) into memory, starting at *buf*. **fread** may copy fewer elements than *count* if an error, or end of file, intervenes.

fread also advances the file position indicator (if any) for *fp* by the number of *characters* actually read.

Returns

The result of **fread** is the number of elements it succeeded in reading.

Portability

ANSI C requires **fread**.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

3.17 freopen—open a file using an existing file descriptor

Synopsis

```
#include <stdio.h>
FILE *freopen(const char *file, const char *mode,
              FILE *fp);
FILE *_freopen_r(struct _reent *ptr, const char *file,
                 const char *mode, FILE *fp);
```

Description

Use this variant of **fopen** if you wish to specify a particular file descriptor *fp* (notably **stdin**, **stdout**, or **stderr**) for the file.

If *fp* was associated with another file or stream, **freopen** closes that other file or stream (but ignores any errors while closing it).

file and *mode* are used just as in **fopen**.

Returns

If successful, the result is the same as the argument *fp*. If the file cannot be opened as specified, the result is **NULL**.

Portability

ANSI C requires **freopen**.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **open**, **read**, **sbrk**, **write**.

3.18 fseek, fseeko—set file position

Synopsis

```
#include <stdio.h>
int fseek(FILE *fp, long offset, int whence)
int fseeko(FILE *fp, off_t offset, int whence)
int _fseek_r(struct _reent *ptr, FILE *fp,
             long offset, int whence)
int _fseeko_r(struct _reent *ptr, FILE *fp,
              off_t offset, int whence)
```

Description

Objects of type `FILE` can have a “position” that records how much of the file your program has already read. Many of the `stdio` functions depend on this position, and many change it as a side effect.

You can use `fseek/fseeko` to set the position for the file identified by *fp*. The value of *offset* determines the new position, in one of three ways selected by the value of *whence* (defined as macros in ‘`stdio.h`’):

`SEEK_SET`—*offset* is the absolute file position (an offset from the beginning of the file) desired. *offset* must be positive.

`SEEK_CUR`—*offset* is relative to the current file position. *offset* can meaningfully be either positive or negative.

`SEEK_END`—*offset* is relative to the current end of file. *offset* can meaningfully be either positive (to increase the size of the file) or negative.

See `ftell/ftello` to determine the current file position.

Returns

`fseek/fseeko` return 0 when successful. On failure, the result is `EOF`. The reason for failure is indicated in `errno`: either `ESPIPE` (the stream identified by *fp* doesn’t support repositioning) or `EINVAL` (invalid file position).

Portability

ANSI C requires `fseek`.

`fseeko` is defined by the Single Unix specification.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.19 fsetpos—restore position of a stream or file

Synopsis

```
#include <stdio.h>
int fsetpos(FILE *fp, const fpos_t *pos);
int _fsetpos_r(struct _reent *ptr, FILE *fp, 1
               const fpos_t *pos);
```

Description

Objects of type `FILE` can have a “position” that records how much of the file your program has already read. Many of the `stdio` functions depend on this position, and many change it as a side effect.

You can use `fsetpos` to return the file identified by `fp` to a previous position `*pos` (after first recording it with `fgetpos`).

See `fseek` for a similar facility.

Returns

`fgetpos` returns 0 when successful. If `fgetpos` fails, the result is 1. The reason for failure is indicated in `errno`: either `ESPIPE` (the stream identified by `fp` doesn't support repositioning) or `EINVAL` (invalid file position).

Portability

ANSI C requires `fsetpos`, but does not specify the nature of `*pos` beyond identifying it as written by `fgetpos`.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.20 `ftell`, `ftello`—return position in a stream or file

Synopsis

```
#include <stdio.h>
long ftell(FILE *fp);
off_t ftello(FILE *fp);
long _ftell_r(struct _reent *ptr, FILE *fp);
off_t _ftello_r(struct _reent *ptr, FILE *fp);
```

Description

Objects of type `FILE` can have a “position” that records how much of the file your program has already read. Many of the `stdio` functions depend on this position, and many change it as a side effect.

The result of `ftell/ftello` is the current position for a file identified by `fp`. If you record this result, you can later use it with `fseek/fseeko` to return the file to this position. The difference between `ftell` and `ftello` is that `ftell` returns `long` and `ftello` returns `off_t`.

In the current implementation, `ftell/ftello` simply uses a character count to represent the file position; this is the same number that would be recorded by `fgetpos`.

Returns

`ftell/ftello` return the file position, if possible. If they cannot do this, they return `-1L`. Failure occurs on streams that do not support positioning; the global `errno` indicates this condition with the value `ESPIPE`.

Portability

`ftell` is required by the ANSI C standard, but the meaning of its result (when successful) is not specified beyond requiring that it be acceptable as an argument to `fseek`. In particular, other conforming C implementations may return a different result from `ftell` than what `fgetpos` records.

`ftello` is defined by the Single Unix specification.

No supporting OS subroutines are required.

3.21 fwrite—write array elements

Synopsis

```
#include <stdio.h>
size_t fwrite(const void *buf, size_t size,
              size_t count, FILE *fp);
```

Description

fwrite attempts to copy, starting from the memory location *buf*, *count* elements (each of size *size*) into the file or stream identified by *fp*. **fwrite** may copy fewer elements than *count* if an error intervenes.

fwrite also advances the file position indicator (if any) for *fp* by the number of *characters* actually written.

Returns

If **fwrite** succeeds in writing all the elements you specify, the result is the same as the argument *count*. In any event, the result is the number of complete elements that **fwrite** copied to the file.

Portability

ANSI C requires **fwrite**.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

3.22 `getc`—read a character (macro)

Synopsis

```
#include <stdio.h>
int getc(FILE *fp);
```

Description

`getc` is a macro, defined in `stdio.h`. You can use `getc` to get the next single character from the file or stream identified by *fp*. As a side effect, `getc` advances the file's current position indicator.

For a subroutine version of this macro, see `fgetc`.

Returns

The next character (read as an **unsigned char**, and cast to **int**), unless there is no more data, or the host system reports a read error; in either of these situations, `getc` returns `EOF`.

You can distinguish the two situations that cause an `EOF` result by using the `ferror` and `feof` functions.

Portability

ANSI C requires `getc`; it suggests, but does not require, that `getc` be implemented as a macro. The standard explicitly permits macro implementations of `getc` to use the argument more than once; therefore, in a portable program, you should not use an expression with side effects as the `getc` argument.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.23 `getc_unlocked`—non-thread-safe version of `getc` (macro)

Description

`getc_unlocked` is a non-thread-safe version of `getc` declared in `stdio.h`. `getc_unlocked` may only safely be used within a scope protected by `flockfile()` (or `ftrylockfile()`) and `funlockfile()`. These functions may safely be used in a multi-threaded program if and only if they are called while the invoking thread owns the (`FILE *`) object, as is the case after a successful call to the `flockfile()` or `ftrylockfile()` functions. If threads are disabled, then `getc_unlocked` is equivalent to `getc`.

Returns

See `getc`.

Portability

POSIX 1003.1 requires `getc_unlocked`. `getc_unlocked` may be implemented as a macro, so arguments should not have side-effects.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.24 `getchar`—read a character (macro)

Synopsis

```
#include <stdio.h>
int getchar(void);

int _getchar_r(struct _reent *reent);
```

Description

`getchar` is a macro, defined in `stdio.h`. You can use `getchar` to get the next single character from the standard input stream. As a side effect, `getchar` advances the standard input's current position indicator.

The alternate function `_getchar_r` is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

The next character (read as an `unsigned char`, and cast to `int`), unless there is no more data, or the host system reports a read error; in either of these situations, `getchar` returns EOF.

You can distinguish the two situations that cause an EOF result by using `'ferror(stdin)'` and `'feof(stdin)'`.

Portability

ANSI C requires `getchar`; it suggests, but does not require, that `getchar` be implemented as a macro.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.25 `getchar_unlocked`—non-thread-safe version of `getchar` (macro)

Description

`getchar_unlocked` is a non-thread-safe version of `getchar` declared in `stdio.h`. `getchar_unlocked` may only safely be used within a scope protected by `flockfile()` (or `ftrylockfile()`) and `funlockfile()`. These functions may safely be used in a multi-threaded program if and only if they are called while the invoking thread owns the (`FILE *`) object, as is the case after a successful call to the `flockfile()` or `ftrylockfile()` functions. If threads are disabled, then `getchar_unlocked` is equivalent to `getchar`.

Returns

See `getchar`.

Portability

POSIX 1003.1 requires `getchar_unlocked`. `getchar_unlocked` may be implemented as a macro.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.26 getdelim—read a line up to a specified line delimiter

Synopsis

```
#include <stdio.h>
int getdelim(char **bufptr, size_t *n,
             int delim, FILE *fp);
```

Description

`getdelim` reads a file *fp* up to and possibly including a specified delimiter *delim*. The line is read into a buffer pointed to by *bufptr* and designated with size **n*. If the buffer is not large enough, it will be dynamically grown by `getdelim`. As the buffer is grown, the pointer to the size *n* will be updated.

Returns

`getdelim` returns `-1` if no characters were successfully read, otherwise, it returns the number of bytes successfully read. at end of file, the result is nonzero.

Portability

`getdelim` is a glibc extension.

No supporting OS subroutines are directly required.

3.27 getline—read a line from a file

Synopsis

```
#include <stdio.h>
ssize_t getline(char **bufptr, size_t *n, FILE *fp);
```

Description

getline reads a file *fp* up to and possibly including the newline character. The line is read into a buffer pointed to by *bufptr* and designated with size **n*. If the buffer is not large enough, it will be dynamically grown by **getdelim**. As the buffer is grown, the pointer to the size *n* will be updated.

getline is equivalent to `getdelim(bufptr, n, '\n', fp);`

Returns

getline returns `-1` if no characters were successfully read, otherwise, it returns the number of bytes successfully read. at end of file, the result is nonzero.

Portability

getline is a glibc extension.

No supporting OS subroutines are directly required.

3.28 `gets`—get character string (obsolete, use `fgets` instead)

Synopsis

```
#include <stdio.h>

char *gets(char *buf);

char *_gets_r(struct _reent *reent, char *buf);
```

Description

Reads characters from standard input until a newline is found. The characters up to the newline are stored in *buf*. The newline is discarded, and the buffer is terminated with a 0. This is a *dangerous* function, as it has no way of checking the amount of space available in *buf*. One of the attacks used by the Internet Worm of 1988 used this to overrun a buffer allocated on the stack of the finger daemon and overwrite the return address, causing the daemon to execute code downloaded into it over the connection.

The alternate function `_gets_r` is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

`gets` returns the buffer passed to it, with the data filled in. If end of file occurs with some data already accumulated, the data is returned with no other indication. If end of file occurs with no data in the buffer, NULL is returned.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.29 `getw`—read a word (`int`)

Synopsis

```
#include <stdio.h>
int getw(FILE *fp);
```

Description

`getw` is a function, defined in `stdio.h`. You can use `getw` to get the next word from the file or stream identified by `fp`. As a side effect, `getw` advances the file's current position indicator.

RETURNS The next word (read as an `int`), unless there is no more data, or the host system reports a read error; in either of these situations, `getw` returns `EOF`. Since `EOF` is a valid `int`, you must use `ferror` or `feof` to distinguish these situations.

Portability

`getw` is a remnant of K&R C, it is not part of any ISO C Standard. `fread` should be used instead. In fact, this implementation of `getw` is based upon `fread`.

Supporting OS subroutines required: `fread`.

3.30 mktemp, mkstemp—generate unused file name

Synopsis

```
#include <stdio.h>
char *mktemp(char *path);
int mkstemp(char *path);

char *_mktemp_r(struct _reent *reent, char *path);
int *_mkstemp_r(struct _reent *reent, char *path);
```

Description

mktemp and **mkstemp** attempt to generate a file name that is not yet in use for any existing file. **mkstemp** creates the file and opens it for reading and writing; **mktemp** simply generates the file name.

You supply a simple pattern for the generated file name, as the string at *path*. The pattern should be a valid filename (including path information if you wish) ending with some number of ‘X’ characters. The generated filename will match the leading part of the name you supply, with the trailing ‘X’ characters replaced by some combination of digits and letters.

The alternate functions **_mktemp_r** and **_mkstemp_r** are reentrant versions. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

mktemp returns the pointer *path* to the modified string representing an unused filename, unless it could not generate one, or the pattern you provided is not suitable for a filename; in that case, it returns **NULL**.

mkstemp returns a file descriptor to the newly created file, unless it could not generate an unused filename, or the pattern you provided is not suitable for a filename; in that case, it returns **-1**.

Portability

ANSI C does not require either **mktemp** or **mkstemp**; the System V Interface Definition requires **mktemp** as of Issue 2.

Supporting OS subroutines required: **getpid**, **open**, **stat**.

3.31 perror—print an error message on standard error

Synopsis

```
#include <stdio.h>
void perror(char *prefix);

void _perror_r(struct _reent *reent, char *prefix);
```

Description

Use **perror** to print (on standard error) an error message corresponding to the current value of the global variable **errno**. Unless you use **NULL** as the value of the argument *prefix*, the error message will begin with the string at *prefix*, followed by a colon and a space (:). The remainder of the error message is one of the strings described for **strerror**.

The alternate function **_perror_r** is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

perror returns no result.

Portability

ANSI C requires **perror**, but the strings issued vary from one implementation to another.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

3.32 `putc`—write a character (macro)

Synopsis

```
#include <stdio.h>
int putc(int ch, FILE *fp);
```

Description

`putc` is a macro, defined in `stdio.h`. `putc` writes the argument *ch* to the file or stream identified by *fp*, after converting it from an `int` to an `unsigned char`.

If the file was opened with append mode (or if the stream cannot support positioning), then the new character goes at the end of the file or stream. Otherwise, the new character is written at the current value of the position indicator, and the position indicator advances by one.

For a subroutine version of this macro, see `fputc`.

Returns

If successful, `putc` returns its argument *ch*. If an error intervenes, the result is `EOF`. You can use ‘`ferror(fp)`’ to query for errors.

Portability

ANSI C requires `putc`; it suggests, but does not require, that `putc` be implemented as a macro. The standard explicitly permits macro implementations of `putc` to use the *fp* argument more than once; therefore, in a portable program, you should not use an expression with side effects as this argument.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.33 `putc_unlocked`—non-thread-safe version of `putc` (macro)

Description

`putc_unlocked` is a non-thread-safe version of `putc` declared in `stdio.h`. `putc_unlocked` may only safely be used within a scope protected by `flockfile()` (or `ftrylockfile()`) and `funlockfile()`. These functions may safely be used in a multi-threaded program if and only if they are called while the invoking thread owns the (`FILE *`) object, as is the case after a successful call to the `flockfile()` or `ftrylockfile()` functions. If threads are disabled, then `putc_unlocked` is equivalent to `putc`.

Returns

See `putc`.

Portability

POSIX 1003.1 requires `putc_unlocked`. `putc_unlocked` may be implemented as a macro, so arguments should not have side-effects.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.34 putchar—write a character (macro)

Synopsis

```
#include <stdio.h>
int putchar(int ch);

int _putchar_r(struct _reent *reent, int ch);
```

Description

`putchar` is a macro, defined in `stdio.h`. `putchar` writes its argument to the standard output stream, after converting it from an `int` to an `unsigned char`.

The alternate function `_putchar_r` is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

If successful, `putchar` returns its argument *ch*. If an error intervenes, the result is `EOF`. You can use `'ferror(stdin)'` to query for errors.

Portability

ANSI C requires `putchar`; it suggests, but does not require, that `putchar` be implemented as a macro.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.35 `putchar_unlocked`—non-thread-safe version of `putchar` (macro)

Description

`putchar_unlocked` is a non-thread-safe version of `putchar` declared in `stdio.h`. `putchar_unlocked` may only safely be used within a scope protected by `flockfile()` (or `ftrylockfile()`) and `funlockfile()`. These functions may safely be used in a multi-threaded program if and only if they are called while the invoking thread owns the (`FILE *`) object, as is the case after a successful call to the `flockfile()` or `ftrylockfile()` functions. If threads are disabled, then `putchar_unlocked` is equivalent to `putchar`.

Returns

See `putchar`.

Portability

POSIX 1003.1 requires `putchar_unlocked`. `putchar_unlocked` may be implemented as a macro.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.36 puts—write a character string

Synopsis

```
#include <stdio.h>
int puts(const char *s);

int _puts_r(struct _reent *reent, const char *s);
```

Description

puts writes the string at *s* (followed by a newline, instead of the trailing null) to the standard output stream.

The alternate function **_puts_r** is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

If successful, the result is a nonnegative integer; otherwise, the result is EOF.

Portability

ANSI C requires **puts**, but does not specify that the result on success must be 0; any non-negative value is permitted.

Supporting OS subroutines required: **close**, **fstat**, **isatty**, **lseek**, **read**, **sbrk**, **write**.

3.37 putw—write a word (int)

Synopsis

```
#include <stdio.h>
int putw(int w, FILE *fp);
```

Description

putw is a function, defined in **stdio.h**. You can use **putw** to write a word to the file or stream identified by *fp*. As a side effect, **putw** advances the file's current position indicator. RETURNS Zero on success, EOF on failure.

Portability

putw is a remnant of K&R C, it is not part of any ISO C Standard. **fwrite** should be used instead. In fact, this implementation of **putw** is based upon **fwrite**.

Supporting OS subroutines required: **fwrite**.

3.38 `remove`—delete a file’s name

Synopsis

```
#include <stdio.h>
int remove(char *filename);

int _remove_r(struct _reent *reent, char *filename);
```

Description

Use `remove` to dissolve the association between a particular filename (the string at *filename*) and the file it represents. After calling `remove` with a particular filename, you will no longer be able to open the file by that name.

In this implementation, you may use `remove` on an open file without error; existing file descriptors for the file will continue to access the file’s data until the program using them closes the file.

The alternate function `_remove_r` is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

`remove` returns 0 if it succeeds, -1 if it fails.

Portability

ANSI C requires `remove`, but only specifies that the result on failure be nonzero. The behavior of `remove` when you call it on an open file may vary among implementations.

Supporting OS subroutine required: `unlink`.

3.39 rename—rename a file

Synopsis

```
#include <stdio.h>
int rename(const char *old, const char *new);

int _rename_r(struct _reent *reent,
               const char *old, const char *new);
```

Description

Use **rename** to establish a new name (the string at *new*) for a file now known by the string at *old*. After a successful **rename**, the file is no longer accessible by the string at *old*.

If **rename** fails, the file named **old* is unaffected. The conditions for failure depend on the host operating system.

The alternate function **_rename_r** is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

The result is either 0 (when successful) or -1 (when the file could not be renamed).

Portability

ANSI C requires **rename**, but only specifies that the result on failure be nonzero. The effects of using the name of an existing file as **new* may vary from one implementation to another.

Supporting OS subroutines required: **link**, **unlink**, or **rename**.

3.40 `rewind`—reinitialize a file or stream

Synopsis

```
#include <stdio.h>
void rewind(FILE *fp);
void _rewind_r(struct _reent *ptr, FILE *fp);
```

Description

`rewind` returns the file position indicator (if any) for the file or stream identified by *fp* to the beginning of the file. It also clears any error indicator and flushes any pending output.

Returns

`rewind` does not return a result.

Portability

ANSI C requires `rewind`.

No supporting OS subroutines are required.

3.41 `setbuf`—specify full buffering for a file or stream

Synopsis

```
#include <stdio.h>
void setbuf(FILE *fp, char *buf);
```

Description

`setbuf` specifies that output to the file or stream identified by *fp* should be fully buffered. All output for this file will go to a buffer (of size `BUFSIZ`, specified in 'stdio.h'). Output will be passed on to the host system only when the buffer is full, or when an input operation intervenes.

You may, if you wish, supply your own buffer by passing a pointer to it as the argument *buf*. It must have size `BUFSIZ`. You can also use `NULL` as the value of *buf*, to signal that the `setbuf` function is to allocate the buffer.

Warnings

You may only use `setbuf` before performing any file operation other than opening the file. If you supply a non-null *buf*, you must ensure that the associated storage continues to be available until you close the stream identified by *fp*.

Returns

`setbuf` does not return a result.

Portability

Both ANSI C and the System V Interface Definition (Issue 2) require `setbuf`. However, they differ on the meaning of a `NULL` buffer pointer: the SVID issue 2 specification says that a `NULL` buffer pointer requests unbuffered output. For maximum portability, avoid `NULL` buffer pointers.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.42 `setbuffer`—specify full buffering for a file or stream with size

Synopsis

```
#include <stdio.h>
void setbuffer(FILE *fp, char *buf, int size);
```

Description

`setbuffer` specifies that output to the file or stream identified by *fp* should be fully buffered. All output for this file will go to a buffer (of size *size*). Output will be passed on to the host system only when the buffer is full, or when an input operation intervenes.

You may, if you wish, supply your own buffer by passing a pointer to it as the argument *buf*. It must have size *size*. You can also use `NULL` as the value of *buf*, to signal that the `setbuffer` function is to allocate the buffer.

Warnings

You may only use `setbuffer` before performing any file operation other than opening the file.

If you supply a non-null *buf*, you must ensure that the associated storage continues to be available until you close the stream identified by *fp*.

Returns

`setbuffer` does not return a result.

Portability

This function comes from BSD not ANSI or POSIX.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.43 `setlinebuf`—specify line buffering for a file or stream

Synopsis

```
#include <stdio.h>
void setlinebuf(FILE *fp);
```

Description

`setlinebuf` specifies that output to the file or stream identified by *fp* should be line buffered. This causes the file or stream to pass on output to the host system at every newline, as well as when the buffer is full, or when an input operation intervenes.

Warnings

You may only use `setlinebuf` before performing any file operation other than opening the file.

Returns

`setlinebuf` returns as per `setvbuf`.

Portability

This function comes from BSD not ANSI or POSIX.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.44 setvbuf—specify file or stream buffering

Synopsis

```
#include <stdio.h>
int setvbuf(FILE *fp, char *buf,
            int mode, size_t size);
```

Description

Use `setvbuf` to specify what kind of buffering you want for the file or stream identified by *fp*, by using one of the following values (from `stdio.h`) as the *mode* argument:

- `_IONBF` Do not use a buffer: send output directly to the host system for the file or stream identified by *fp*.
- `_IOFBF` Use full output buffering: output will be passed on to the host system only when the buffer is full, or when an input operation intervenes.
- `_IOLBF` Use line buffering: pass on output to the host system at every newline, as well as when the buffer is full, or when an input operation intervenes.

Use the *size* argument to specify how large a buffer you wish. You can supply the buffer itself, if you wish, by passing a pointer to a suitable area of memory as *buf*. Otherwise, you may pass `NULL` as the *buf* argument, and `setvbuf` will allocate the buffer.

Warnings

You may only use `setvbuf` before performing any file operation other than opening the file. If you supply a non-null *buf*, you must ensure that the associated storage continues to be available until you close the stream identified by *fp*.

Returns

A 0 result indicates success, EOF failure (invalid *mode* or *size* can cause failure).

Portability

Both ANSI C and the System V Interface Definition (Issue 2) require `setvbuf`. However, they differ on the meaning of a `NULL` buffer pointer: the SVID issue 2 specification says that a `NULL` buffer pointer requests unbuffered output. For maximum portability, avoid `NULL` buffer pointers.

Both specifications describe the result on failure only as a nonzero value.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.45 printf, fprintf, asprintf, sprintf, snprintf—format output

Synopsis

```
#include <stdio.h>

int printf(const char *format [, arg, ...]);
int fprintf(FILE *fd, const char *format [, arg, ...]);
int sprintf(char *str, const char *format [, arg, ...]);
int asprintf(char **strp, const char *format [, arg, ...]);
int snprintf(char *str, size_t size, const char *format [, arg, ...]);
```

Description

printf accepts a series of arguments, applies to each a format specifier from **format*, and writes the formatted data to **stdout**, terminated with a null character. The behavior of **printf** is undefined if there are not enough arguments for the format. **printf** returns when it reaches the end of the format string. If there are more arguments than the format requires, excess arguments are ignored.

fprintf, **asprintf**, **sprintf** and **snprintf** are identical to **printf**, other than the destination of the formatted output: **fprintf** sends the output to a specified file *fd*, while **asprintf** stores the output in a dynamically allocated buffer, while **sprintf** stores the output in the specified char array *str* and **snprintf** limits number of characters written to *str* to at most *size* (including terminating 0). For **sprintf** and **snprintf**, the behavior is undefined if the output **str* overlaps with one of the arguments. For **asprintf**, *strp* points to a pointer to char which is filled in with the dynamically allocated buffer. *format* is a pointer to a character string containing two types of objects: ordinary characters (other than %), which are copied unchanged to the output, and conversion specifications, each of which is introduced by %. (To include % in the output, use %% in the format string.) A conversion specification has the following form:

```
%[flags][width][.prec][size][type]
```

The fields of the conversion specification have the following meanings:

- *flags*

an optional sequence of characters which control output justification, numeric signs, decimal points, trailing zeroes, and octal and hex prefixes. The flag characters are minus (-), plus (+), space (), zero (0), and sharp (#). They can appear in any combination.

- The result of the conversion is left justified, and the right is padded with blanks. If you do not use this flag, the result is right justified, and padded on the left.
- + The result of a signed conversion (as determined by *type*) will always begin with a plus or minus sign. (If you do not use this flag, positive values do not begin with a plus sign.)

- " " (space)

If the first character of a signed conversion specification is not a sign, or if a signed conversion results in no characters, the result will begin with a space. If the space () flag and the plus (+) flag both appear, the space flag is ignored.

0	If the <i>type</i> character is d , i , o , u , x , X , e , E , f , g , or G : leading zeroes, are used to pad the field width (following any indication of sign or base); no spaces are used for padding. If the zero (0) and minus (-) flags both appear, the zero (0) flag will be ignored. For d , i , o , u , x , and X conversions, if a precision <i>prec</i> is specified, the zero (0) flag is ignored. Note that 0 is interpreted as a flag, not as the beginning of a field width.
#	The result is to be converted to an alternative form, according to the next character:
0	increases precision to force the first digit of the result to be a zero.
x	a non-zero result will have a 0x prefix.
X	a non-zero result will have a 0X prefix.
e, E or f	The result will always contain a decimal point even if no digits follow the point. (Normally, a decimal point appears only if a digit follows it.) Trailing zeroes are removed.
g or G	same as e or E , but trailing zeroes are not removed.
all others	undefined.

- *width*

width is an optional minimum field width. You can either specify it directly as a decimal integer, or indirectly by using instead an asterisk (*), in which case an **int** argument is used as the field width. Negative field widths are not supported; if you attempt to specify a negative field width, it is interpreted as a minus (-) flag followed by a positive field width.

- *prec*

an optional field; if present, it is introduced with '.' (a period). This field gives the maximum number of characters to print in a conversion; the minimum number of digits of an integer to print, for conversions with *type* **d**, **i**, **o**, **u**, **x**, and **X**; the maximum number of significant digits, for the **g** and **G** conversions; or the number of digits to print after the decimal point, for **e**, **E**, and **f** conversions. You can specify the precision either directly as a decimal integer or indirectly by using an asterisk (*), in which case an **int** argument is used as the precision. Supplying a negative precision is equivalent to omitting the precision. If only a period is specified the precision is zero. If a precision appears with any other conversion *type* than those listed here, the behavior is undefined.

- *size*

h, **l**, and **L** are optional size characters which override the default way that **printf** interprets the data type of the corresponding argument. **h** forces the following **d**, **i**, **o**, **u**, **x** or **X** conversion *type* to apply to a **short** or **unsigned short**. **h** also forces a following **n** *type* to apply to a pointer to a **short**. Similarly, an **l** forces the following **d**, **i**, **o**, **u**, **x** or **X** conversion *type* to apply to a **long** or **unsigned long**. **l** also forces a following **n** *type* to apply to a pointer to a **long**. **l** with **c**, **s** is equivalent to **C**, **S**

respectively. If an `h` or an `l` appears with another conversion specifier, the behavior is undefined. `L` or `ll` forces the following `d`, `i`, `o`, `u`, `x` or `X` conversion *type* to apply to a `long long` or `unsigned long long`. `L` forces a following `e`, `E`, `f`, `g` or `G` conversion *type* to apply to a `long double` argument. If `L` appears with any other conversion *type*, the behavior is undefined.

- *type*

type specifies what kind of conversion `printf` performs. Here is a table of these:

<code>%</code>	prints the percent character (<code>%</code>)
<code>c</code>	prints <i>arg</i> as single character
<code>C</code>	prints <code>wchar_t arg</code> as single multibyte character
<code>s</code>	prints characters until precision is reached or a null terminator is encountered; takes a string pointer
<code>S</code>	converts <code>wchar_t</code> characters to multibyte output characters until precision is reached or a null <code>wchar_t</code> terminator is encountered; takes a <code>wchar_t</code> pointer
<code>d</code>	prints a signed decimal integer; takes an <code>int</code> (same as <code>i</code>)
<code>i</code>	prints a signed decimal integer; takes an <code>int</code> (same as <code>d</code>)
<code>o</code>	prints a signed octal integer; takes an <code>int</code>
<code>u</code>	prints an unsigned decimal integer; takes an <code>int</code>
<code>x</code>	prints an unsigned hexadecimal integer (using <code>abcdef</code> as digits beyond 9); takes an <code>int</code>
<code>X</code>	prints an unsigned hexadecimal integer (using <code>ABCDEF</code> as digits beyond 9); takes an <code>int</code>
<code>f</code>	prints a signed value of the form <code>[-]9999.9999</code> ; takes a floating-point number
<code>e</code>	prints a signed value of the form <code>[-]9.9999e[+ -]999</code> ; takes a floating-point number
<code>E</code>	prints the same way as <code>e</code> , but using <code>E</code> to introduce the exponent; takes a floating-point number
<code>g</code>	prints a signed value in either <code>f</code> or <code>e</code> form, based on given value and precision—trailing zeros and the decimal point are printed only if necessary; takes a floating-point number
<code>G</code>	prints the same way as <code>g</code> , but using <code>E</code> for the exponent if an exponent is needed; takes a floating-point number
<code>n</code>	stores (in the same object) a count of the characters written; takes a pointer to <code>int</code>
<code>p</code>	prints a pointer in an implementation-defined format. This implementation treats the pointer as an <code>unsigned long</code> (same as <code>Lu</code>).

Returns

`sprintf` and `asprintf` return the number of bytes in the output string, save that the concluding NULL is not counted. `printf` and `fprintf` return the number of characters transmitted. If an error occurs, `printf` and `fprintf` return EOF and `asprintf` returns -1. No error returns occur for `sprintf`.

Portability

The ANSI C standard specifies that implementations must support at least formatted output of up to 509 characters.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.46 scanf, fscanf, sscanf—scan and format input

Synopsis

```
#include <stdio.h>

int scanf(const char *format [, arg, ...]);
int fscanf(FILE *fd, const char *format [, arg, ...]);
int sscanf(const char *str, const char *format
           [, arg, ...]);

int _scanf_r(struct _reent *ptr, const char *format [, arg, ...]);
int _fscanf_r(struct _reent *ptr, FILE *fd, const char *format [, arg, ...]);
int _sscanf_r(struct _reent *ptr, const char *str, const char *format
             [, arg, ...]);
```

Description

scanf scans a series of input fields from standard input, one character at a time. Each field is interpreted according to a format specifier passed to **scanf** in the format string at **format*. **scanf** stores the interpreted input from each field at the address passed to it as the corresponding argument following *format*. You must supply the same number of format specifiers and address arguments as there are input fields.

There must be sufficient address arguments for the given format specifiers; if not the results are unpredictable and likely disastrous. Excess address arguments are merely ignored.

scanf often produces unexpected results if the input diverges from an expected pattern. Since the combination of **gets** or **fgets** followed by **sscanf** is safe and easy, that is the preferred way to be certain that a program is synchronized with input at the end of a line.

fscanf and **sscanf** are identical to **scanf**, other than the source of input: **fscanf** reads from a file, and **sscanf** from a string.

The routines **_scanf_r**, **_fscanf_r**, and **_sscanf_r** are reentrant versions of **scanf**, **fscanf**, and **sscanf** that take an additional first argument pointing to a reentrancy structure.

The string at **format* is a character sequence composed of zero or more directives. Directives are composed of one or more whitespace characters, non-whitespace characters, and format specifications.

Whitespace characters are blank (), tab (\t), or newline (\n). When **scanf** encounters a whitespace character in the format string it will read (but not store) all consecutive whitespace characters up to the next non-whitespace character in the input.

Non-whitespace characters are all other ASCII characters except the percent sign (%). When **scanf** encounters a non-whitespace character in the format string it will read, but not store a matching non-whitespace character.

Format specifications tell **scanf** to read and convert characters from the input field into specific types of values, and store them in the locations specified by the address arguments.

Trailing whitespace is left unread unless explicitly matched in the format string.

The format specifiers must begin with a percent sign (%) and have the following form:

```
%[*][width][size]type
```

Each format specification begins with the percent character (%). The other fields are:

***** an optional marker; if present, it suppresses interpretation and assignment of this input field.

width an optional maximum field width: a decimal integer, which controls the maximum number of characters that will be read before converting the current input field. If the input field has fewer than *width* characters, `scanf` reads all the characters in the field, and then proceeds with the next field and its format specification.

If a whitespace or a non-convertable character occurs before *width* character are read, the characters up to that character are read, converted, and stored. Then `scanf` proceeds to the next format specification.

size `h`, `l`, and `L` are optional size characters which override the default way that `scanf` interprets the data type of the corresponding argument.

Modifier	Type(s)	
<code>hh</code>	<code>d, i, o, u, x, n</code>	convert input to char, store in char object
<code>h</code>	<code>d, i, o, u, x, n</code>	convert input to short, store in short object
<code>h</code>	<code>D, I, O, U, X</code> <code>e, f, c, s, p</code>	no effect
<code>l</code>	<code>d, i, o, u, x, n</code>	convert input to long, store in long object
<code>l</code>	<code>e, f, g</code>	convert input to double store in a double object
<code>l</code>	<code>D, I, O, U, X</code> <code>c, s, p</code>	no effect
<code>ll</code>	<code>d, i, o, u, x, n</code>	convert to long long, store in long long
<code>L</code>	<code>d, i, o, u, x, n</code>	convert to long long, store in long long
<code>L</code>	<code>e, f, g, E, G</code>	convert to long double, store in long double
<code>L</code>	all others	no effect

type

A character to specify what kind of conversion `scanf` performs. Here is a table of the conversion characters:

<code>%</code>	No conversion is done; the percent character (<code>%</code>) is stored.
<code>c</code>	Scans one character. Corresponding <i>arg</i> : <code>(char *arg)</code> .
<code>s</code>	Reads a character string into the array supplied. Corresponding <i>arg</i> : <code>(char arg[])</code> .

<code>[pattern]</code>	Reads a non-empty character string into memory starting at <i>arg</i> . This area must be large enough to accept the sequence and a terminating null character which will be added automatically. (<i>pattern</i> is discussed in the paragraph following this table). Corresponding <i>arg</i> : <code>(char *arg)</code> .
<code>d</code>	Reads a decimal integer into the corresponding <i>arg</i> : <code>(int *arg)</code> .
<code>D</code>	Reads a decimal integer into the corresponding <i>arg</i> : <code>(long *arg)</code> .
<code>o</code>	Reads an octal integer into the corresponding <i>arg</i> : <code>(int *arg)</code> .
<code>O</code>	Reads an octal integer into the corresponding <i>arg</i> : <code>(long *arg)</code> .
<code>u</code>	Reads an unsigned decimal integer into the corresponding <i>arg</i> : <code>(unsigned int *arg)</code> .
<code>U</code>	Reads an unsigned decimal integer into the corresponding <i>arg</i> : <code>(unsigned long *arg)</code> .
<code>x,X</code>	Read a hexadecimal integer into the corresponding <i>arg</i> : <code>(int *arg)</code> .
<code>e, f, g</code>	Read a floating-point number into the corresponding <i>arg</i> : <code>(float *arg)</code> .
<code>E, F, G</code>	Read a floating-point number into the corresponding <i>arg</i> : <code>(double *arg)</code> .
<code>i</code>	Reads a decimal, octal or hexadecimal integer into the corresponding <i>arg</i> : <code>(int *arg)</code> .
<code>I</code>	Reads a decimal, octal or hexadecimal integer into the corresponding <i>arg</i> : <code>(long *arg)</code> .
<code>n</code>	Stores the number of characters read in the corresponding <i>arg</i> : <code>(int *arg)</code> .
<code>p</code>	Stores a scanned pointer. ANSI C leaves the details to each implementation; this implementation treats <code>%p</code> exactly the same as <code>%U</code> . Corresponding <i>arg</i> : <code>(void **arg)</code> .

A *pattern* of characters surrounded by square brackets can be used instead of the `s` type character. *pattern* is a set of characters which define a search set of possible characters making up the `scanf` input field. If the first character in the brackets is a caret (^), the search set is inverted to include all ASCII characters except those between the brackets. There is also a range facility which you can use as a shortcut. `%[0-9]` matches all decimal digits. The hyphen must not be the first or last character in the set. The character prior to the hyphen must be lexically less than the character after it.

Here are some *pattern* examples:

`%[abcd]` matches strings containing only `a`, `b`, `c`, and `d`.

`%[^abcd]` matches strings containing any characters except `a`, `b`, `c`, or `d`

`%[A-DW-Z]`

matches strings containing A, B, C, D, W, X, Y, Z

`%[z-a]` matches the characters z, -, and a

Floating point numbers (for field types `e`, `f`, `g`, `E`, `F`, `G`) must correspond to the following general form:

`[+/-] dddd[.]ddd [E|e[+|-]ddd]`

where objects inclosed in square brackets are optional, and `ddd` represents decimal, octal, or hexadecimal digits.

Returns

`scanf` returns the number of input fields successfully scanned, converted and stored; the return value does not include scanned fields which were not stored.

If `scanf` attempts to read at end-of-file, the return value is `EOF`.

If no fields were stored, the return value is 0.

`scanf` might stop scanning a particular field before reaching the normal field end character, or may terminate entirely.

`scanf` stops scanning and storing the current field and moves to the next input field (if any) in any of the following situations:

- The assignment suppressing character (`*`) appears after the `%` in the format specification; the current input field is scanned but not stored.
- *width* characters have been read (*width* is a width specification, a positive decimal integer).
- The next character read cannot be converted under the the current format (for example, if a `Z` is read when the format is decimal).
- The next character in the input field does not appear in the search set (or does appear in the inverted search set).

When `scanf` stops scanning the current input field for one of these reasons, the next character is considered unread and used as the first character of the following input field, or the first character in a subsequent read operation on the input.

`scanf` will terminate under the following circumstances:

- The next character in the input field conflicts with a corresponding non-whitespace character in the format string.
- The next character in the input field is `EOF`.
- The format string has been exhausted.

When the format string contains a character sequence that is not part of a format specification, the same character sequence must appear in the input; `scanf` will scan but not store the matched characters. If a conflict occurs, the first conflicting character remains in the input as if it had never been read.

Portability

`scanf` is ANSI C.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.47 `iprintf`, `fiprintf`, `asiprintf`, `siprintf`, `sniprintf`— format output

Synopsis

```
#include <stdio.h>

int iprintf(const char *format [, arg, ...]);
int fiprintf(FILE *fd, const char *format [, arg, ...]);
int siprintf(char *str, const char *format [, arg, ...]);
int asiprintf(char **strp, const char *format [, arg, ...]);
int sniprintf(char *str, size_t size, const char *format [, arg, ...]);
```

Description

`iprintf`, `fiprintf`, `siprintf`, `sniprintf`, `asiprintf`, are the same as `printf`, `fprintf`, `sprintf`, `snprintf`, and `asprintf`, respectively, only that they restrict usage to non-floating-point format specifiers.

Returns

`siprintf` and `asiprintf` return the number of bytes in the output string, save that the concluding NULL is not counted. `iprintf` and `fiprintf` return the number of characters transmitted. If an error occurs, `iprintf` and `fiprintf` return EOF and `asiprintf` returns -1. No error returns occur for `siprintf`.

Portability

`iprintf`, `fiprintf`, `siprintf`, `sniprintf`, and `asprintf` are newlib extensions.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.48 `iscanf`, `fiscanf`, `siscanf`—scan and format non-floating input

Synopsis

```
#include <stdio.h>

int iscanf(const char *format [, arg, ...]);
int fscanf(FILE *fd, const char *format [, arg, ...]);
int siscanf(const char *str, const char *format
            [, arg, ...]);

int _iscanf_r(struct _reent *ptr, const char *format [, arg, ...]);
int _fiscanf_r(struct _reent *ptr, FILE *fd, const char *format [, arg, ...]);
int _siscanf_r(struct _reent *ptr, const char *str, const char *format
              [, arg, ...]);
```

Description

`iscanf`, `fiscanf`, and `siscanf` are the same as `scanf`, `fscanf`, and `sscanf` respectively, only that they restrict the available formats to non-floating-point format specifiers.

The routines `_iscanf_r`, `_fiscanf_r`, and `_siscanf_r` are reentrant versions of `iscanf`, `fiscanf`, and `siscanf` that take an additional first argument pointing to a reentrancy structure.

Returns

`iscanf` returns the number of input fields successfully scanned, converted and stored; the return value does not include scanned fields which were not stored.

If `iscanf` attempts to read at end-of-file, the return value is EOF.

If no fields were stored, the return value is 0.

Portability

`iscanf`, `fiscanf`, and `siscanf` are newlib extensions.

Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.49 tmpfile—create a temporary file

Synopsis

```
#include <stdio.h>
FILE *tmpfile(void);

FILE *_tmpfile_r(struct _reent *reent);
```

Description

Create a temporary file (a file which will be deleted automatically), using a name generated by `tmpnam`. The temporary file is opened with the mode "`wb+`", permitting you to read and write anywhere in it as a binary file (without any data transformations the host system may perform for text files).

The alternate function `_tmpfile_r` is a reentrant version. The argument *reent* is a pointer to a reentrancy structure.

Returns

`tmpfile` normally returns a pointer to the temporary file. If no temporary file could be created, the result is `NULL`, and `errno` records the reason for failure.

Portability

Both ANSI C and the System V Interface Definition (Issue 2) require `tmpfile`.

Supporting OS subroutines required: `close`, `fstat`, `getpid`, `isatty`, `lseek`, `open`, `read`, `sbrk`, `write`.

`tmpfile` also requires the global pointer `environ`.

3.50 tmpnam, tempnam—name for a temporary file

Synopsis

```
#include <stdio.h>
char *tmpnam(char *s);
char *tempnam(char *dir, char *pfx);
char *_tmpnam_r(struct _reent *reent, char *s);
char *_tempnam_r(struct _reent *reent, char *dir, char *pfx);
```

Description

Use either of these functions to generate a name for a temporary file. The generated name is guaranteed to avoid collision with other files (for up to TMP_MAX calls of either function). **tmpnam** generates file names with the value of P_tmpdir (defined in 'stdio.h') as the leading directory component of the path.

You can use the **tmpnam** argument *s* to specify a suitable area of memory for the generated filename; otherwise, you can call **tmpnam**(NULL) to use an internal static buffer.

tempnam allows you more control over the generated filename: you can use the argument *dir* to specify the path to a directory for temporary files, and you can use the argument *pfx* to specify a prefix for the base filename.

If *dir* is NULL, **tempnam** will attempt to use the value of environment variable TMPDIR instead; if there is no such value, **tempnam** uses the value of P_tmpdir (defined in 'stdio.h').

If you don't need any particular prefix to the basename of temporary files, you can pass NULL as the *pfx* argument to **tempnam**.

_tmpnam_r and **_tempnam_r** are reentrant versions of **tmpnam** and **tempnam** respectively. The extra argument *reent* is a pointer to a reentrancy structure.

Warnings

The generated filenames are suitable for temporary files, but do not in themselves make files temporary. Files with these names must still be explicitly removed when you no longer want them.

If you supply your own data area *s* for **tmpnam**, you must ensure that it has room for at least L_tmpnam elements of type **char**.

Returns

Both **tmpnam** and **tempnam** return a pointer to the newly generated filename.

Portability

ANSI C requires **tmpnam**, but does not specify the use of P_tmpdir. The System V Interface Definition (Issue 2) requires both **tmpnam** and **tempnam**.

Supporting OS subroutines required: **close**, **fstat**, **getpid**, **isatty**, **lseek**, **open**, **read**, **sbrk**, **write**.

The global pointer **environ** is also required.

3.51 vprintf, vfprintf, vsprintf—format argument list

Synopsis

```
#include <stdio.h>
#include <stdarg.h>
int vprintf(const char *fmt, va_list list);
int vfprintf(FILE *fp, const char *fmt, va_list list);
int vsprintf(char *str, const char *fmt, va_list list);
int vasprintf(char **strp, const char *fmt, va_list list);
int vsnprintf(char *str, size_t size, const char *fmt, va_list list);

int _vprintf_r(struct _reent *reent, const char *fmt,
               va_list list);
int _vfprintf_r(struct _reent *reent, FILE *fp, const char *fmt,
               va_list list);
int _vasprintf_r(struct _reent *reent, char **str, const char *fmt,
               va_list list);
int _vsprintf_r(struct _reent *reent, char *str, const char *fmt,
               va_list list);
int _vsnprintf_r(struct _reent *reent, char *str, size_t size, const char *fmt,
               va_list list);
```

Description

vprintf, vfprintf, vasprintf, vsprintf and vsnprintf are (respectively) variants of printf, fprintf, asprintf, sprintf, and snprintf. They differ only in allowing their caller to pass the variable argument list as a va_list object (initialized by va_start) rather than directly accepting a variable number of arguments.

Returns

The return values are consistent with the corresponding functions: vasprintf/vsprintf returns the number of bytes in the output string, save that the concluding NULL is not counted. vprintf and vfprintf return the number of characters transmitted. If an error occurs, vprintf and vfprintf return EOF and vasprintf returns -1. No error returns occur for vsprintf.

Portability

ANSI C requires all three functions.

Supporting OS subroutines required: close, fstat, isatty, lseek, read, sbrk, write.

3.52 vscanf, vfscanf, vsscanf—format argument list

Synopsis

```
#include <stdio.h>
#include <stdarg.h>
int vscanf(const char *fmt, va_list list);
int vfscanf(FILE *fp, const char *fmt, va_list list);
int vsscanf(const char *str, const char *fmt, va_list list);

int _vscanf_r(struct _reent *reent, const char *fmt,
              va_list list);
int _vfscanf_r(struct _reent *reent, FILE *fp, const char *fmt,
              va_list list);
int _vsscanf_r(struct _reent *reent, const char *str, const char *fmt,
              va_list list);
```

Description

vsscanf, **vfscanf**, and **vsscanf** are (respectively) variants of **scanf**, **fscanf**, and **sscanf**. They differ only in allowing their caller to pass the variable argument list as a **va_list** object (initialized by **va_start**) rather than directly accepting a variable number of arguments.

Returns

The return values are consistent with the corresponding functions: **vsscanf** returns the number of input fields successfully scanned, converted, and stored; the return value does not include scanned fields which were not stored.

If **vsscanf** attempts to read at end-of-file, the return value is **EOF**.

If no fields were stored, the return value is 0.

The routines **_vscanf_r**, **_vfscanf_r**, and **_vsscanf_r** are reentrant versions which take an additional first parameter which points to the reentrancy structure.

Portability

These are GNU extensions.

Supporting OS subroutines required:

3.53 `viprintf`, `vfiprintf`, `vasiprintf`—format argument list

Synopsis

```
#include <stdio.h>
#include <stdarg.h>
int viprintf(const char *fmt, va_list list);
int vfiprintf(FILE *fp, const char *fmt, va_list list);
int vsiprintf(char *str, const char *fmt, va_list list);
int vasiprintf(char **strp, const char *fmt, va_list list);
int vsniprintf(char *str, size_t size, const char *fmt, va_list list);

int _viprintf_r(struct _reent *reent, const char *fmt,
               va_list list);
int _vfiprintf_r(struct _reent *reent, FILE *fp, const char *fmt,
               va_list list);
int _vasiprintf_r(struct _reent *reent, char **str, const char *fmt,
               va_list list);
int _vsiprintf_r(struct _reent *reent, char *str, const char *fmt,
               va_list list);
int _vsniprintf_r(struct _reent *reent, char *str, size_t size, const char *fmt,
               va_list list);
```

Description

`viprintf`, `vfiprintf`, `vasiprintf`, `vsiprintf` and `vsniprintf` are (respectively) variants of `iprintf`, `fiprintf`, `asiprintf`, `siprintf`, and `sniprintf`. They differ only in restricting the caller to use non-floating-point format specifiers.

Returns

The return values are consistent with the corresponding functions: `vasiprintf`/`vsiprintf` returns the number of bytes in the output string, save that the concluding NULL is not counted. `viprintf` and `vfiprintf` return the number of characters transmitted. If an error occurs, `viprintf` and `vfiprintf` return EOF and `vasiprintf` returns -1. No error returns occur for `vsiprintf`.

Portability

`viprintf`, `vfiprintf`, `vasiprintf`, `vsiprintf` and `vsniprintf` are newlib extensions. Supporting OS subroutines required: `close`, `fstat`, `isatty`, `lseek`, `read`, `sbrk`, `write`.

3.54 viscanf, vfiscanf, vsiscanf—format argument list

Synopsis

```
#include <stdio.h>
#include <stdarg.h>
int viscanf(const char *fmt, va_list list);
int vfiscanf(FILE *fp, const char *fmt, va_list list);
int vsiscanf(const char *str, const char *fmt, va_list list);

int _viscanf_r(struct _reent *reent, const char *fmt,
               va_list list);
int _vfiscanf_r(struct _reent *reent, FILE *fp, const char *fmt,
               va_list list);
int _vsiscanf_r(struct _reent *reent, const char *str, const char *fmt,
               va_list list);
```

Description

`viscanf`, `vfiscanf`, and `vsiscanf` are (respectively) variants of `iscanf`, `fiscanf`, and `siscanf`. They differ only in allowing their caller to pass the variable argument list as a `va_list` object (initialized by `va_start`) rather than directly accepting a variable number of arguments.

Returns

The return values are consistent with the corresponding functions: `viscanf` returns the number of input fields successfully scanned, converted, and stored; the return value does not include scanned fields which were not stored.

If `viscanf` attempts to read at end-of-file, the return value is EOF.

If no fields were stored, the return value is 0.

The routines `_viscanf_r`, `_vfiscanf_r`, and `_vsiscanf_r` are reentrant versions which take an additional first parameter which points to the reentrancy structure.

Portability

These are newlib extensions.

Supporting OS subroutines required:

4 Strings and Memory (`'string.h'`)

This chapter describes string-handling functions and functions for managing areas of memory. The corresponding declarations are in `'string.h'`.

4.1 bcmp—compare two memory areas

Synopsis

```
#include <string.h>
int bcmp(const void *s1, const void *s2, size_t n);
```

Description

This function compares not more than *n* bytes of the object pointed to by *s1* with the object pointed to by *s2*.

This function is identical to `memcmp`.

Returns

The function returns an integer greater than, equal to or less than zero according to whether the object pointed to by *s1* is greater than, equal to or less than the object pointed to by *s2*.

Portability

`bcmp` requires no supporting OS subroutines.

4.2 bcopy—copy memory regions

Synopsis

```
#include <string.h>
void bcopy(const void *in, void *out, size_t n);
```

Description

This function copies *n* bytes from the memory region pointed to by *in* to the memory region pointed to by *out*.

This function is implemented in term of `memmove`.

Portability

`bcopy` requires no supporting OS subroutines.

4.3 bzero—initialize memory to zero

Synopsis

```
#include <string.h>
void bzero(void *b, size_t length);
```

Description

`bzero` initializes *length* bytes of memory, starting at address *b*, to zero.

Returns

`bzero` does not return a result.

Portability

`bzero` is in the Berkeley Software Distribution. Neither ANSI C nor the System V Interface Definition (Issue 2) require `bzero`.

`bzero` requires no supporting OS subroutines.

4.4 `index`—search for character in string

Synopsis

```
#include <string.h>
char * index(const char *string, int c);
```

Description

This function finds the first occurrence of *c* (converted to a `char`) in the string pointed to by *string* (including the terminating null character).

This function is identical to `strchr`.

Returns

Returns a pointer to the located character, or a null pointer if *c* does not occur in *string*.

Portability

`index` requires no supporting OS subroutines.

4.5 memccpy—copy memory regions with end-token check

Synopsis

```
#include <string.h>
void* memccpy(void *out, const void *in,
               int endchar, size_t n);
```

Description

This function copies up to n bytes from the memory region pointed to by *in* to the memory region pointed to by *out*. If a byte matching the *endchar* is encountered, the byte is copied and copying stops.

If the regions overlap, the behavior is undefined.

Returns

`memccpy` returns a pointer to the first byte following the *endchar* in the *out* region. If no byte matching *endchar* was copied, then `NULL` is returned.

Portability

`memccpy` is a GNU extension.

`memccpy` requires no supporting OS subroutines.

4.6 `memchr`—find character in memory

Synopsis

```
#include <string.h>
void *memchr(const void *src, int c, size_t length);
```

Description

This function searches memory starting at **src* for the character *c*. The search only ends with the first occurrence of *c*, or after *length* characters; in particular, `NULL` does not terminate the search.

Returns

If the character *c* is found within *length* characters of **src*, a pointer to the character is returned. If *c* is not found, then `NULL` is returned.

Portability

`memchr` is ANSI C.

`memchr` requires no supporting OS subroutines.

4.7 memcmp—compare two memory areas

Synopsis

```
#include <string.h>
int memcmp(const void *s1, const void *s2, size_t n);
```

Description

This function compares not more than n characters of the object pointed to by $s1$ with the object pointed to by $s2$.

Returns

The function returns an integer greater than, equal to or less than zero according to whether the object pointed to by $s1$ is greater than, equal to or less than the object pointed to by $s2$.

Portability

`memcmp` is ANSI C.

`memcmp` requires no supporting OS subroutines.

4.8 memcpy—copy memory regions

Synopsis

```
#include <string.h>
void* memcpy(void *out, const void *in, size_t n);
```

Description

This function copies *n* bytes from the memory region pointed to by *in* to the memory region pointed to by *out*.

If the regions overlap, the behavior is undefined.

Returns

`memcpy` returns a pointer to the first byte of the *out* region.

Portability

`memcpy` is ANSI C.

`memcpy` requires no supporting OS subroutines.

4.9 memmove—move possibly overlapping memory

Synopsis

```
#include <string.h>
void *memmove(void *dst, const void *src, size_t length);
```

Description

This function moves *length* characters from the block of memory starting at **src* to the memory starting at **dst*. `memmove` reproduces the characters correctly at **dst* even if the two areas overlap.

Returns

The function returns *dst* as passed.

Portability

`memmove` is ANSI C.

`memmove` requires no supporting OS subroutines.

4.10 mempcpy—copy memory regions and return end pointer

Synopsis

```
#include <string.h>
void* mempcpy(void *out, const void *in, size_t n);
```

Description

This function copies *n* bytes from the memory region pointed to by *in* to the memory region pointed to by *out*.

If the regions overlap, the behavior is undefined.

Returns

`mempcpy` returns a pointer to the byte following the last byte copied to the *out* region.

Portability

`mempcpy` is a GNU extension.

`mempcpy` requires no supporting OS subroutines.

4.11 `memset`—set an area of memory

Synopsis

```
#include <string.h>
void *memset(const void *dst, int c, size_t length);
```

Description

This function converts the argument *c* into an unsigned char and fills the first *length* characters of the array pointed to by *dst* to the value.

Returns

`memset` returns the value of *m*.

Portability

`memset` is ANSI C.

`memset` requires no supporting OS subroutines.

4.12 rindex—reverse search for character in string

Synopsis

```
#include <string.h>
char * rindex(const char *string, int c);
```

Description

This function finds the last occurrence of *c* (converted to a char) in the string pointed to by *string* (including the terminating null character).

This function is identical to `strrchr`.

Returns

Returns a pointer to the located character, or a null pointer if *c* does not occur in *string*.

Portability

`rindex` requires no supporting OS subroutines.

4.13 strcasecmp—case insensitive character string compare

Synopsis

```
#include <string.h>
int strcasecmp(const char *a, const char *b);
```

Description

`strcasecmp` compares the string at *a* to the string at *b* in a case-insensitive manner.

Returns

If **a* sorts lexicographically after **b* (after both are converted to upper case), `strcasecmp` returns a number greater than zero. If the two strings match, `strcasecmp` returns zero. If **a* sorts lexicographically before **b*, `strcasecmp` returns a number less than zero.

Portability

`strcasecmp` is in the Berkeley Software Distribution.

`strcasecmp` requires no supporting OS subroutines. It uses `tolower()` from elsewhere in this library.

4.14 `strcat`—concatenate strings

Synopsis

```
#include <string.h>
char *strcat(char *dst, const char *src);
```

Description

`strcat` appends a copy of the string pointed to by *src* (including the terminating null character) to the end of the string pointed to by *dst*. The initial character of *src* overwrites the null character at the end of *dst*.

Returns

This function returns the initial value of *dst*.

Portability

`strcat` is ANSI C.

`strcat` requires no supporting OS subroutines.

4.15 strchr—search for character in string

Synopsis

```
#include <string.h>
char * strchr(const char *string, int c);
```

Description

This function finds the first occurrence of *c* (converted to a char) in the string pointed to by *string* (including the terminating null character).

Returns

Returns a pointer to the located character, or a null pointer if *c* does not occur in *string*.

Portability

`strchr` is ANSI C.

`strchr` requires no supporting OS subroutines.

4.16 `strcmp`—character string compare

Synopsis

```
#include <string.h>
int strcmp(const char *a, const char *b);
```

Description

`strcmp` compares the string at *a* to the string at *b*.

Returns

If **a* sorts lexicographically after **b*, `strcmp` returns a number greater than zero. If the two strings match, `strcmp` returns zero. If **a* sorts lexicographically before **b*, `strcmp` returns a number less than zero.

Portability

`strcmp` is ANSI C.

`strcmp` requires no supporting OS subroutines.

4.17 strcoll—locale specific character string compare

Synopsis

```
#include <string.h>
int strcoll(const char *stra, const char * strb);
```

Description

strcoll compares the string pointed to by *stra* to the string pointed to by *strb*, using an interpretation appropriate to the current LC_COLLATE state.

Returns

If the first string is greater than the second string, **strcoll** returns a number greater than zero. If the two strings are equivalent, **strcoll** returns zero. If the first string is less than the second string, **strcoll** returns a number less than zero.

Portability

strcoll is ANSI C.

strcoll requires no supporting OS subroutines.

4.18 strcpy—copy string

Synopsis

```
#include <string.h>
char *strcpy(char *dst, const char *src);
```

Description

`strcpy` copies the string pointed to by *src* (including the terminating null character) to the array pointed to by *dst*.

Returns

This function returns the initial value of *dst*.

Portability

`strcpy` is ANSI C.

`strcpy` requires no supporting OS subroutines.

4.19 `strcspn`—count chars not in string

Synopsis

```
size_t strcspn(const char *s1, const char *s2);
```

Description

This function computes the length of the initial part of the string pointed to by *s1* which consists entirely of characters *NOT* from the string pointed to by *s2* (excluding the terminating null character).

Returns

`strcspn` returns the length of the substring found.

Portability

`strcspn` is ANSI C.

`strcspn` requires no supporting OS subroutines.

4.20 `strerror`—convert error number to string

Synopsis

```
#include <string.h>
char *strerror(int errnum);
```

Description

`strerror` converts the error number *errnum* into a string. The value of *errnum* is usually a copy of `errno`. If *errnum* is not a known error number, the result points to an empty string. This implementation of `strerror` prints out the following strings for each of the values defined in `'errno.h'`:

E2BIG	Arg list too long
EACCES	Permission denied
EADDRINUSE	Address already in use
EADV	Advertise error
EAFNOSUPPORT	Address family not supported by protocol family
EAGAIN	No more processes
EALREADY	Socket already connected
EBADF	Bad file number
EBADMSG	Bad message
EBUSY	Device or resource busy
ECHILD	No children
ECOMM	Communication error
ECONNABORTED	Software caused connection abort
ECONNREFUSED	Connection refused
EDEADLK	Deadlock
EDESTADDRREQ	Destination address required
EEXIST	File exists
EDOM	Math argument
EFAULT	Bad address
EFBIG	File too large
EHOSTDOWN	Host is down

EHOSTUNREACH

Host is unreachable

EIDRM Identifier removed**EINPROGRESS**

Connection already in progress

EINTR Interrupted system call**EINVAL** Invalid argument**EIO** I/O error**EISCONN** Socket is already connected**EISDIR** Is a directory**ELIBACC** Cannot access a needed shared library**ELIBBAD** Accessing a corrupted shared library**ELIBEXEC** Cannot exec a shared library directly**ELIBMAX** Attempting to link in more shared libraries than system limit**ELIBSCN** .lib section in a.out corrupted**EMFILE** Too many open files**EMLINK** Too many links**EMSGSIZE** Message too long**EMULTIHOP**

Multihop attempted

ENAMETOOLONG

File or path name too long

ENETDOWN Network interface not configured**ENETUNREACH**

Network is unreachable

ENFILE Too many open files in system**ENODEV** No such device**ENOENT** No such file or directory**ENOEXEC** Exec format error**ENOLCK** No lock**ENOLINK** Virtual circuit is gone**ENOMEM** Not enough space**ENOMSG** No message of desired type**ENONET** Machine is not on the network

ENOPKG	No package
ENOPROTOOPT	Protocol not available
ENOSPC	No space left on device
ENOSR	No stream resources
ENOSTR	Not a stream
ENOSYS	Function not implemented
ENOTBLK	Block device required
ENOTCONN	Socket is not connected
ENOTDIR	Not a directory
ENOTEMPTY	Directory not empty
ENOTSOCK	Socket operation on non-socket
ENOTSUP	Not supported
ENOTTY	Not a character device
ENXIO	No such device or address
EPERM	Not owner
EPIPE	Broken pipe
EPROTO	Protocol error
EPROTOTYPE	Protocol wrong type for socket
EPROTONOSUPPORT	Unknown protocol
ERANGE	Result too large
EREMOTE	Resource is remote
EROFS	Read-only file system
ESHUTDOWN	Can't send after socket shutdown
ESOCKTNOSUPPORT	Socket type not supported
ESPIPE	Illegal seek
ESRCH	No such process
ESRMNT	Srmount error
ETIME	Stream ioctl timeout

ETIMEDOUT

Connection timed out

ETXTBSY Text file busy

EXDEV Cross-device link

Returns

This function returns a pointer to a string. Your application must not modify that string.

Portability

ANSI C requires `strerror`, but does not specify the strings used for each error number.

Although this implementation of `strerror` is reentrant, ANSI C declares that subsequent calls to `strerror` may overwrite the result string; therefore portable code cannot depend on the reentrancy of this subroutine.

This implementation of `strerror` provides for user-defined extensibility. `errno.h` defines `__ELASTERROR`, which can be used as a base for user-defined error values. If the user supplies a routine named `_user_strerror`, and `errnum` passed to `strerror` does not match any of the supported values, `_user_strerror` is called with `errnum` as its argument.

`_user_strerror` takes one argument of type `int`, and returns a character pointer. If `errnum` is unknown to `_user_strerror`, `_user_strerror` returns `NULL`. The default `_user_strerror` returns `NULL` for all input values.

`strerror` requires no supporting OS subroutines.

4.21 `strerror_r`—convert error number to string and copy to buffer

Synopsis

```
#include <string.h>
char *strerror_r(int errnum, char *buffer, size_t n);
```

Description

`strerror_r` converts the error number *errnum* into a string and copies the result into the supplied *buffer* for a length up to *n*, including the NUL terminator. The value of *errnum* is usually a copy of `errno`. If *errnum* is not a known error number, the result is the empty string.

See `strerror` for how strings are mapped to *errnum*.

Returns

This function returns a pointer to a string. Your application must not modify that string.

Portability

`strerror_r` is a gnu extension.

`strerror_r` requires no supporting OS subroutines.

4.22 strlen—character string length

Synopsis

```
#include <string.h>
size_t strlen(const char *str);
```

Description

The `strlen` function works out the length of the string starting at **str* by counting characters until it reaches a `NULL` character.

Returns

`strlen` returns the character count.

Portability

`strlen` is ANSI C.

`strlen` requires no supporting OS subroutines.

4.23 `strlwr`—force string to lower case

Synopsis

```
#include <string.h>
char *strlwr(char *a);
```

Description

`strlwr` converts each characters in the string at *a* to lower case.

Returns

`strlwr` returns its argument, *a*.

Portability

`strlwr` is not widely portable.

`strlwr` requires no supporting OS subroutines.

4.24 strncasecmp—case insensitive character string compare

Synopsis

```
#include <string.h>
int strncasecmp(const char *a, const char * b, size_t length);
```

Description

strncasecmp compares up to *length* characters from the string at *a* to the string at *b* in a case-insensitive manner.

Returns

If **a* sorts lexicographically after **b* (after both are converted to upper case), **strncasecmp** returns a number greater than zero. If the two strings are equivalent, **strncasecmp** returns zero. If **a* sorts lexicographically before **b*, **strncasecmp** returns a number less than zero.

Portability

strncasecmp is in the Berkeley Software Distribution.

strncasecmp requires no supporting OS subroutines. It uses `tolower()` from elsewhere in this library.

4.25 strncat—concatenate strings

Synopsis

```
#include <string.h>
char *strncat(char *dst, const char *src, size_t length);
```

Description

`strncat` appends not more than *length* characters from the string pointed to by *src* (including the terminating null character) to the end of the string pointed to by *dst*. The initial character of *src* overwrites the null character at the end of *dst*. A terminating null character is always appended to the result

Warnings

Note that a null is always appended, so that if the copy is limited by the *length* argument, the number of characters appended to *dst* is `n + 1`.

Returns

This function returns the initial value of *dst*

Portability

`strncat` is ANSI C.

`strncat` requires no supporting OS subroutines.

4.26 strncmp—character string compare

Synopsis

```
#include <string.h>
int strncmp(const char *a, const char * b, size_t length);
```

Description

`strncmp` compares up to *length* characters from the string at *a* to the string at *b*.

Returns

If **a* sorts lexicographically after **b*, `strncmp` returns a number greater than zero. If the two strings are equivalent, `strncmp` returns zero. If **a* sorts lexicographically before **b*, `strncmp` returns a number less than zero.

Portability

`strncmp` is ANSI C.

`strncmp` requires no supporting OS subroutines.

4.27 strncpy—counted copy string

Synopsis

```
#include <string.h>
char *strncpy(char *dst, const char *src, size_t length);
```

Description

strncpy copies not more than *length* characters from the the string pointed to by *src* (including the terminating null character) to the array pointed to by *dst*. If the string pointed to by *src* is shorter than *length* characters, null characters are appended to the destination array until a total of *length* characters have been written.

Returns

This function returns the initial value of *dst*.

Portability

strncpy is ANSI C.

strncpy requires no supporting OS subroutines.

4.28 strlen—character string length

Synopsis

```
#include <string.h>
size_t strlen(const char *str, size_t n);
```

Description

The `strlen` function works out the length of the string starting at `*str` by counting characters until it reaches a NUL character or the maximum: `n` number of characters have been inspected.

Returns

`strlen` returns the character count or `n`.

Portability

`strlen` is a Gnu extension.

`strlen` requires no supporting OS subroutines.

4.29 `strpbrk`—find chars in string

Synopsis

```
#include <string.h>
char *strpbrk(const char *s1, const char *s2);
```

Description

This function locates the first occurrence in the string pointed to by *s1* of any character in string pointed to by *s2* (excluding the terminating null character).

Returns

`strpbrk` returns a pointer to the character found in *s1*, or a null pointer if no character from *s2* occurs in *s1*.

Portability

`strpbrk` requires no supporting OS subroutines.

4.30 strchr—reverse search for character in string

Synopsis

```
#include <string.h>
char * strchr(const char *string, int c);
```

Description

This function finds the last occurrence of *c* (converted to a char) in the string pointed to by *string* (including the terminating null character).

Returns

Returns a pointer to the located character, or a null pointer if *c* does not occur in *string*.

Portability

`strchr` is ANSI C.

`strchr` requires no supporting OS subroutines.

4.31 strsignal—return string describing a signal

Synopsis

```
#include <string.h>
const char * strsignal(int sig);
```

Description

strsignal returns a string describing a particular signal.

Returns

For signals whose value is defined in **<signal.h>**, a pointer to a constant string is returned. Otherwise, a pointer to a static buffer is returned, holding the string "Unknown signal **n**"; this pointer is therefore not thread-safe, and should also not be held across subsequent calls to **strsignal**.

Portability

This function is found on Linux, Solaris, and some other systems.

No supporting OS subroutines are required to link with **strsignal**.

4.32 `strspn`—find initial match

Synopsis

```
#include <string.h>
size_t strspn(const char *s1, const char *s2);
```

Description

This function computes the length of the initial segment of the string pointed to by *s1* which consists entirely of characters from the string pointed to by *s2* (excluding the terminating null character).

Returns

`strspn` returns the length of the segment found.

Portability

`strspn` is ANSI C.

`strspn` requires no supporting OS subroutines.

4.33 `strstr`—find string segment

Synopsis

```
#include <string.h>
char *strstr(const char *s1, const char *s2);
```

Description

Locates the first occurrence in the string pointed to by *s1* of the sequence of characters in the string pointed to by *s2* (excluding the terminating null character).

Returns

Returns a pointer to the located string segment, or a null pointer if the string *s2* is not found. If *s2* points to a string with zero length, the *s1* is returned.

Portability

`strstr` is ANSI C.

`strstr` requires no supporting OS subroutines.

4.34 strtok, strtok_r, strsep—get next token from a string

Synopsis

```
#include <string.h>
char *strtok(char *source, const char *delimiters)
char *strtok_r(char *source, const char *delimiters,
               char **lasts)
char *strsep(char **source_ptr, const char *delimiters)
```

Description

The **strtok** function is used to isolate sequential tokens in a null-terminated string, **source*. These tokens are delimited in the string by at least one of the characters in **delimiters*. The first time that **strtok** is called, **source* should be specified; subsequent calls, wishing to obtain further tokens from the same string, should pass a null pointer instead. The separator string, **delimiters*, must be supplied each time, and may change between calls.

The **strtok** function returns a pointer to the beginning of each subsequent token in the string, after replacing the separator character itself with a NUL character. When no more tokens remain, a null pointer is returned.

The **strtok_r** function has the same behavior as **strtok**, except a pointer to placeholder **[lasts]* must be supplied by the caller.

The **strsep** function is similar in behavior to **strtok**, except a pointer to the string pointer must be supplied [*source_ptr*] and the function does not skip leading delimiters. When the string starts with a delimiter, the delimiter is changed to the NUL character and the empty string is returned. Like **strtok_r** and **strtok**, the **[source_ptr]* is updated to the next character following the last delimiter found or NULL if the end of string is reached with no more delimiters.

Returns

strtok, **strtok_r**, and **strsep** all return a pointer to the next token, or NULL if no more tokens can be found. For **strsep**, a token may be the empty string.

Portability

strtok is ANSI C. **strtok_r** is POSIX. **strsep** is a BSD-extension.

strtok, **strtok_r**, and **strsep** require no supporting OS subroutines.

4.35 `strupr`—force string to uppercase

Synopsis

```
#include <string.h>
char *strupr(char *a);
```

Description

`strupr` converts each characters in the string at *a* to upper case.

Returns

`strupr` returns its argument, *a*.

Portability

`strupr` is not widely portable.

`strupr` requires no supporting OS subroutines.

4.36 strxfrm—transform string

Synopsis

```
#include <string.h>
size_t strxfrm(char *s1, const char *s2, size_t n);
```

Description

This function transforms the string pointed to by *s2* and places the resulting string into the array pointed to by *s1*. The transformation is such that if the **strcmp** function is applied to the two transformed strings, it returns a value greater than, equal to, or less than zero, corresponding to the result of a **strcoll** function applied to the same two original strings.

No more than *n* characters are placed into the resulting array pointed to by *s1*, including the terminating null character. If *n* is zero, *s1* may be a null pointer. If copying takes place between objects that overlap, the behavior is undefined.

With a C locale, this function just copies.

Returns

The **strxfrm** function returns the length of the transformed string (not including the terminating null character). If the value returned is *n* or more, the contents of the array pointed to by *s1* are indeterminate.

Portability

strxfrm is ANSI C.

strxfrm requires no supporting OS subroutines.

4.37 swab—swap adjacent bytes

Synopsis

```
#include <unistd.h>
void swab(const void *in, void *out, ssize_t n);
```

Description

This function copies *n* bytes from the memory region pointed to by *in* to the memory region pointed to by *out*, exchanging adjacent even and odd bytes.

Portability

`swab` requires no supporting OS subroutines.

5 Wide Character Strings (`wchar.h`)

This chapter describes wide-character string-handling functions and managing areas of memory containing wide-characters. The corresponding declarations are in `wchar.h`.

5.1 wmemchr—find a wide-character in memory

Synopsis

```
#include <wchar.h>
wchar_t *wmemchr(const wchar_t *s, wchar_t c, size_t n);
```

Description

The `wmemchr` function locates the first occurrence of `c` in the initial `n` wide-characters of the object pointed to be `s`. This function is not affected by locale and all `wchar_t` values are treated identically. The null wide-character and `wchar_t` values not corresponding to valid characters are not treated specially.

If `n` is zero, `s` must be a valid pointer and the function behaves as if no valid occurrence of `c` is found.

Returns

The `wmemchr` function returns a pointer to the located wide-character, or a null pointer if the wide-character does not occur in the object.

Portability

`wmemchr` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.2 wmemcmp—compare wide-characters in memory

Synopsis

```
#include <wchar.h>
int wmemcmp(const wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

The `wmemcmp` function compares the first n wide-characters of the object pointed to by $s1$ to the first n wide-characters of the object pointed to by $s2$. This function is not affected by locale and all `wchar_t` values are treated identically. The null wide-character and `wchar_t` values not corresponding to valid characters are not treated specially.

If n is zero, $s1$ and $s2$ must be a valid pointers and the function behaves as if the two objects compare equal.

Returns

The `wmemcmp` function returns an integer greater than, equal to, or less than zero, accordingly as the object pointed to by $s1$ is greater than, equal to, or less than the object pointed to by $s2$.

Portability

`wmemcmp` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.3 wmemcpy—copy wide-characters in memory

Synopsis

```
#include <wchar.h>
wchar_t *wmemcpy(wchar_t *d, const wchar_t *s, size_t n);
```

Description

The `wmemcpy` function copies *n* wide-characters from the object pointed to by *s* to the object pointed to be *d*. This function is not affected by locale and all `wchar_t` values are treated identically. The null wide-character and `wchar_t` values not corresponding to valid characters are not treated specially.

If *n* is zero, *d* and *s* must be a valid pointers, and the function copies zero wide-characters.

Returns

The `wmemcpy` function returns the value of *d*.

Portability

`wmemcpy` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.4 wmemmove—wmemmove - copy wide-characters in memory with overlapping areas

Synopsis

```
#include <wchar.h>
wchar_t *wmemmove(wchar_t *d, const wchar_t *s, size_t n);
```

Description

The `wmemmove` function copies n wide-characters from the object pointed to by s to the object pointed to by d . Copying takes place as if the n wide-characters from the object pointed to by s are first copied into a temporary array of n wide-characters that does not overlap the objects pointed to by d or s , and then the n wide-characters from the temporary array are copied into the object pointed to by d .

This function is not affected by locale and all `wchar_t` values are treated identically. The null wide-character and `wchar_t` values not corresponding to valid characters are not treated specially.

If n is zero, d and s must be a valid pointers, and the function copies zero wide-characters.

Returns

The `wmemmove` function returns the value of d .

Portability

`wmemmove` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.5 `wmemset`—set wide-characters in memory

Synopsis

```
#include <wchar.h>
wchar_t *wmemset(wchar_t *s, wchar_t c, size_t n);
```

Description

The `wmemset` function copies the value of `c` into each of the first `n` wide-characters of the object pointed to by `s`. This function is not affected by locale and all `wchar_t` values are treated identically. The null wide-character and `wchar_t` values not corresponding to valid characters are not treated specially.

If `n` is zero, `s` must be a valid pointer and the function copies zero wide-characters.

Returns

The `wmemset` functions returns the value of `s`.

Portability

`wmemset` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.6 wcscat—concatenate two wide-character strings

Synopsis

```
#include <wchar.h>
wchar_t *wcscat(wchar_t *s1, const wchar_t *s2);
```

Description

The `wcscat` function appends a copy of the wide-character string pointed to by `s2` (including the terminating null wide-character code) to the end of the wide-character string pointed to by `s1`. The initial wide-character code of `s2` overwrites the null wide-character code at the end of `s1`. If copying takes place between objects that overlap, the behaviour is undefined.

Returns

The `wcscat` function returns `s1`; no return value is reserved to indicate an error.

Portability

`wcscat` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.7 wcschr—wide-character string scanning operation

Synopsis

```
#include <wchar.h>
wchar_t *wcschr(const wchar_t *s, wchar_t c);
```

Description

The `wcschr` function locates the first occurrence of `c` in the wide-character string pointed to by `s`. The value of `c` must be a character representable as a type `wchar_t` and must be a wide-character code corresponding to a valid character in the current locale. The terminating null wide-character string.

Returns

Upon completion, `wcschr` returns a pointer to the wide-character code, or a null pointer if the wide-character code is not found.

Portability

`wcschr` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.8 `wscmp`—compare two wide-character strings

Synopsis

```
#include <wchar.h>
int wscmp(const wchar_t *s1, *s2);
```

Description

The `wscmp` function compares the wide-character string pointed to by *s1* to the wide-character string pointed to by *s2*.

The sign of a non-zero return value is determined by the sign of the difference between the values of the first pair of wide-character codes that differ in the objects being compared.

Returns

Upon completion, `wscmp` returns an integer greater than, equal to or less than 0, if the wide-character string pointed to by *s1* is greater than, equal to or less than the wide-character string pointed to by *s2* respectively.

Portability

`wscmp` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.9 wcscoll—locale specific wide-character string compare

Synopsis

```
#include <wchar.h>
int wcscoll(const wchar_t *stra, const wchar_t * strb);
```

Description

`wcscoll` compares the wide-character string pointed to by *stra* to the wide-character string pointed to by *strb*, using an interpretation appropriate to the current `LC_COLLATE` state.

The current implementation of `wcscoll` simply uses `wscmp` and does not support any language-specific sorting.

Returns

If the first string is greater than the second string, `wcscoll` returns a number greater than zero. If the two strings are equivalent, `wcscoll` returns zero. If the first string is less than the second string, `wcscoll` returns a number less than zero.

Portability

`wcscoll` is ISO/IEC 9899/AMD1:1995 (ISO C).

5.10 `wcscpy`—copy a wide-character string

Synopsis

```
#include <wchar.h>
wchar_t *wcscpy(wchar_t *s1, const wchar_t *,s2);
```

Description

The `wcscpy` function copies the wide-character string pointed to by *s2* (including the terminating null wide-character code) into the array pointed to by *s1*. If copying takes place between objects that overlap, the behaviour is undefined.

Returns

The `wcscpy` function returns *s1*; no return value is reserved to indicate an error.

Portability

`wcscpy` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.11 `wcscspn`—get length of a complementary wide substring

Synopsis

```
#include <wchar.h>
size_t wcscspn(const wchar_t *s, wchar_t *set);
```

Description

The `wcscspn` function computes the length of the maximum initial segment of the wide-character string pointed to by *s* which consists entirely of wide-character codes not from the wide-character string pointed to by *set*.

Returns

The `wcscspn` function returns the length of the initial substring of *s1*; no return value is reserved to indicate an error.

Portability

`wcscspn` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.12 wcslcat—concatenate wide-character strings to specified length

Synopsis

```
#include <wchar.h>
size_t wcslcat(wchar_t *dst, const wchar_t *src, size_t siz);
```

Description

The `wcslcat` function appends wide-characters from *src* to end of the *dst* wide-character string so that the resultant wide-character string is not more than *siz* wide-characters including terminating null wide-character code. A terminating null wide-character is always added unless *siz* is 0. Thus, the maximum number of wide-characters that can be appended from *src* is *siz* - 1. If copying takes place between objects that overlap, the behaviour is undefined.

Returns

Wide-character string length of initial *dst* plus the wide-character string length of *src* (does not include terminating null wide-characters). If the return value is greater than or equal to *siz*, then truncation occurred and not all wide-characters from *src* were appended.

Portability

No supporting OS subroutines are required.

5.13 wcsncpy—copy a wide-character string to specified length

Synopsis

```
#include <wchar.h>
size_t wcsncpy(wchar_t *dst, const wchar_t *src, size_t siz);
```

Description

`wcsncpy` copies wide-characters from *src* to *dst* such that up to *siz* - 1 characters are copied. A terminating null is appended to the result, unless *siz* is zero.

Returns

`wcsncpy` returns the number of wide-characters in *src*, not including the terminating null wide-character. If the return value is greater than or equal to *siz*, then not all wide-characters were copied from *src* and truncation occurred.

Portability

No supporting OS subroutines are required.

5.14 `wcslen`—get wide-character string length

Synopsis

```
#include <wchar.h>
size_t wcslen(const wchar_t *s);
```

Description

The `wcslen` function computes the number of wide-character codes in the wide-character string to which *s* points, not including the terminating null wide-character code.

Returns

The `wcslen` function returns the length of *s*; no return value is reserved to indicate an error.

Portability

`wcslen` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.15 `wcsncat`—concatenate part of two wide-character strings

Synopsis

```
#include <wchar.h>
wchar_t *wcsncat(wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

The `wcsncat` function appends not more than *n* wide-character codes (a null wide-character code and wide-character codes that follow it are not appended) from the array pointed to by *s2* to the end of the wide-character string pointed to by *s1*. The initial wide-character code of *s2* overwrites the null wide-character code at the end of *s1*. A terminating null wide-character code is always appended to the result. If copying takes place between objects that overlap, the behaviour is undefined.

Returns

The `wcsncat` function returns *s1*; no return value is reserved to indicate an error.

Portability

`wcsncat` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.16 wcsncmp—compare part of two wide-character strings

Synopsis

```
#include <wchar.h>
int wcsncmp(const wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

The `wcsncmp` function compares not more than *n* wide-character codes (wide-character codes that follow a null wide-character code are not compared) from the array pointed to by *s1* to the array pointed to by *s2*.

The sign of a non-zero return value is determined by the sign of the difference between the values of the first pair of wide-character codes that differ in the objects being compared.

Returns

Upon successful completion, `wcsncmp` returns an integer greater than, equal to or less than 0, if the possibly null-terminated array pointed to by *s1* is greater than, equal to or less than the possibly null-terminated array pointed to by *s2* respectively.

Portability

`wcsncmp` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.17 wcsncpy—copy part of a wide-character string

Synopsis

```
#include <wchar.h>
wchar_t *wcsncpy(wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

The `wcsncpy` function copies not more than `n` wide-character codes (wide-character codes that follow a null wide-character code are not copied) from the array pointed to by `s2` to the array pointed to by `s1`. If copying takes place between objects that overlap, the behaviour is undefined.

If the array pointed to by `s2` is a wide-character string that is shorter than `n` wide-character codes, null wide-character codes are appended to the copy in the array pointed to by `s1`, until `n` wide-character codes in all are written.

Returns

The `wcsncpy` function returns `s1`; no return value is reserved to indicate an error.

Portability

`wcsncpy` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.18 `wcsnlen`—get fixed-size wide character string length

Synopsis

```
#include <wchar.h>
size_t wcsnlen(const wchar_t *s, size_t maxlen);
```

Description

The `wcsnlen` function computes the number of wide character codes in the wide character string pointed to by *s* not including the terminating `L'\0'` wide character but at most *maxlen* wide characters.

Returns

`wcsnlen` returns the length of *s* if it is less than *maxlen*, or *maxlen* if there is no `L'\0'` wide character in first *maxlen* characters.

Portability

`wcsnlen` is GNU extension.. `wcsnlen` requires no supporting OS subroutines.

5.19 `wcspbrk`—scan wide-character string for a wide-character code

Synopsis

```
#include <wchar.h>
wchar_t *wcspbrk(const wchar_t *s, const wchar_t *set);
```

Description

The `wcspbrk` function locates the first occurrence in the wide-character string pointed to by *s* of any wide-character code from the wide-character string pointed to by *set*.

Returns

Upon successful completion, `wcspbrk` returns a pointer to the wide-character code or a null pointer if no wide-character code from *set* occurs in *s*.

Portability

`wcspbrk` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.20 `wcsrchr`—wide-character string scanning operation

Synopsis

```
#include <wchar.h>
wchar_t *wcsrchr(const wchar_t *s, wchar_t c);
```

Description

The `wcsrchr` function locates the last occurrence of `c` in the wide-character string pointed to by `s`. The value of `c` must be a character representable as a type `wchar_t` and must be a wide-character code corresponding to a valid character in the current locale. The terminating null wide-character code is considered to be part of the wide-character string.

Returns

Upon successful completion, `wcsrchr` returns a pointer to the wide-character code or a null pointer if `c` does not occur in the wide-character string.

Portability

`wcsrchr` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.21 wcssp—get length of a wide substring

Synopsis

```
#include <wchar.h>
size_t wcssp(const wchar_t *s, const wchar_t *set);
```

Description

The `wcssp` function computes the length of the maximum initial segment of the wide-character string pointed to by *s* which consists entirely of wide-character codes from the wide-character string pointed to by *set*.

Returns

The `wcssp()` function returns the length *sl*; no return value is reserved to indicate an error.

Portability

`wcssp` is ISO/IEC 9899/AMD1:1995 (ISO C). No supporting OS subroutines are required.

5.22 wcsstr—find a wide-character substring

Synopsis

```
#include <wchar.h>
wchar_t *wcsstr(const wchar_t *big, const wchar_t *little);
```

Description

The `wcsstr` function locates the first occurrence in the wide-character string pointed to by *big* of the sequence of wide-characters (excluding the terminating null wide-character) in the wide-character string pointed to by *little*.

Returns

On successful completion, `wcsstr` returns a pointer to the located wide-character string, or a null pointer if the wide-character string is not found.

If *little* points to a wide-character string with zero length, the function returns *big*.

Portability

`wcsstr` is ISO/IEC 9899/AMD1:1995 (ISO C).

5.23 `wcswidth`—number of column positions of a wide-character string

Synopsis

```
#include <wchar.h>
int wcswidth(const wchar_t *pwcs, size_t n);
```

Description

The `wcswidth` function shall determine the number of column positions required for `n` wide-character codes (or fewer than `n` wide-character codes if a null wide-character code is encountered before `n` wide-character codes are exhausted) in the string pointed to by `pwcs`.

Returns

The `wcswidth` function either shall return 0 (if `pwcs` points to a null wide-character code), or return the number of column positions to be occupied by the wide-character string pointed to by `pwcs`, or return -1 (if any of the first `n` wide-character codes in the wide-character string pointed to by `pwcs` is not a printable wide-character code).

Portability

`wcswidth` has been introduced in the Single UNIX Specification Volume 2 `wcswidth` has been marked as extension in Single UNIX Specification Volume 3

5.24 `wcwidth`—number of column positions of a wide-character code

Synopsis

```
#include <wchar.h>
int wcwidth(const wchar_t wc);
```

Description

The `wcwidth` function shall determine the number of column positions required for the wide character `wc`. The application shall ensure that the value of `wc` is a character representable as a `wchar_t`, and is a wide-character code corresponding to a valid character in the current locale.

Returns

The `wcwidth` function shall either return 0 (if `wc` is a null wide-character code), or return the number of column positions to be occupied by the wide-character code `wc`, or return -1 (if `wc` does not correspond to a printable wide-character code).

The current implementation of `wcwidth` simply sets the width of all printable characters to 1 since newlib has no character tables around.

Portability

`wcwidth` has been introduced in the Single UNIX Specification Volume 2 `wcwidth` has been marked as extension in Single UNIX Specification Volume 3

6 Signal Handling ('signal.h')

A *signal* is an event that interrupts the normal flow of control in your program. Your operating environment normally defines the full set of signals available (see '`sys/signal.h`'), as well as the default means of dealing with them—typically, either printing an error message and aborting your program, or ignoring the signal.

All systems support at least the following signals:

SIGABRT	Abnormal termination of a program; raised by the <code><<abort>></code> function.
SIGFPE	A domain error in arithmetic, such as overflow, or division by zero.
SIGILL	Attempt to execute as a function data that is not executable.
SIGINT	Interrupt; an interactive attention signal.
SIGSEGV	An attempt to access a memory location that is not available.
SIGTERM	A request that your program end execution.

Two functions are available for dealing with asynchronous signals—one to allow your program to send signals to itself (this is called *raising* a signal), and one to specify subroutines (called *handlers* to handle particular signals that you anticipate may occur—whether raised by your own program or the operating environment).

To support these functions, '`signal.h`' defines three macros:

SIG_DFL	Used with the <code>signal</code> function in place of a pointer to a handler subroutine, to select the operating environment's default handling of a signal.
SIG_IGN	Used with the <code>signal</code> function in place of a pointer to a handler, to ignore a particular signal.
SIG_ERR	Returned by the <code>signal</code> function in place of a pointer to a handler, to indicate that your request to set up a handler could not be honored for some reason.

'`signal.h`' also defines an integral type, `sig_atomic_t`. This type is not used in any function declarations; it exists only to allow your signal handlers to declare a static storage location where they may store a signal value. (Static storage is not otherwise reliable from signal handlers.)

6.1 `raise`—send a signal

Synopsis

```
#include <signal.h>
int raise(int sig);

int _raise_r(void *reent, int sig);
```

Description

Send the signal *sig* (one of the macros from ‘`sys/signal.h`’). This interrupts your program’s normal flow of execution, and allows a signal handler (if you’ve defined one, using `signal`) to take control.

The alternate function `_raise_r` is a reentrant version. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

The result is 0 if *sig* was successfully raised, 1 otherwise. However, the return value (since it depends on the normal flow of execution) may not be visible, unless the signal handler for *sig* terminates with a `return` or unless `SIG_IGN` is in effect for this signal.

Portability

ANSI C requires `raise`, but allows the full set of signal numbers to vary from one implementation to another.

Required OS subroutines: `getpid`, `kill`.

6.2 signal—specify handler subroutine for a signal

Synopsis

```
#include <signal.h>
void ( * signal(int sig, void(*func)(int)) )(int);

void ( * _signal_r(void *reent,
    int sig, void(*func)(int)) )(int);

int raise (int sig);

int _raise_r (void *reent, int sig);
```

Description

signal, **raise** provide a simple signal/raise implementation for embedded targets.

signal allows you to request changed treatment for a particular signal *sig*. You can use one of the predefined macros **SIG_DFL** (select system default handling) or **SIG_IGN** (ignore this signal) as the value of *func*; otherwise, *func* is a function pointer that identifies a subroutine in your program as the handler for this signal.

Some of the execution environment for signal handlers is unpredictable; notably, the only library function required to work correctly from within a signal handler is **signal** itself, and only when used to redefine the handler for the current signal value.

Static storage is likewise unreliable for signal handlers, with one exception: if you declare a static storage location as 'volatile sig_atomic_t', then you may use that location in a signal handler to store signal values.

If your signal handler terminates using **return** (or implicit return), your program's execution continues at the point where it was when the signal was raised (whether by your program itself, or by an external event). Signal handlers can also use functions such as **exit** and **abort** to avoid returning.

raise sends the signal *sig* to the executing program. It returns zero if successful, non-zero if unsuccessful.

The alternate functions **_signal_r**, **_raise_r** are the reentrant versions. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

If your request for a signal handler cannot be honored, the result is **SIG_ERR**; a specific error number is also recorded in **errno**.

Otherwise, the result is the previous handler (a function pointer or one of the predefined macros).

Portability

ANSI C requires **raise**, **signal**.

No supporting OS subroutines are required to link with **signal**, but it will not have any useful effects, except for software generated signals, without an operating system that can actually raise exceptions.

7 Time Functions ('time.h')

This chapter groups functions used either for reporting on time (elapsed, current, or compute time) or to perform calculations based on time.

The header file 'time.h' defines three types. `clock_t` and `time_t` are both used for representations of time particularly suitable for arithmetic. (In this implementation, quantities of type `clock_t` have the highest resolution possible on your machine, and quantities of type `time_t` resolve to seconds.) `size_t` is also defined if necessary for quantities representing sizes.

'time.h' also defines the structure `tm` for the traditional representation of Gregorian calendar time as a series of numbers, with the following fields:

<code>tm_sec</code>	Seconds.
<code>tm_min</code>	Minutes.
<code>tm_hour</code>	Hours.
<code>tm_mday</code>	Day.
<code>tm_mon</code>	Month.
<code>tm_year</code>	Year (since 1900).
<code>tm_wday</code>	Day of week: the number of days since Sunday.
<code>tm_yday</code>	Number of days elapsed since last January 1.
<code>tm_isdst</code>	Daylight Savings Time flag: positive means DST in effect, zero means DST not in effect, negative means no information about DST is available.

7.1 `asctime`—format time as string

Synopsis

```
#include <time.h>
char *asctime(const struct tm *clock);
char *asctime_r(const struct tm *clock, char *buf);
```

Description

Format the time value at *clock* into a string of the form

```
Wed Jun 15 11:38:07 1988\n\0
```

The string is generated in a static buffer; each call to `asctime` overwrites the string generated by previous calls.

Returns

A pointer to the string containing a formatted timestamp.

Portability

ANSI C requires `asctime`.

`asctime` requires no supporting OS subroutines.

7.2 clock—cumulative processor time

Synopsis

```
#include <time.h>
clock_t clock(void);
```

Description

Calculates the best available approximation of the cumulative amount of time used by your program since it started. To convert the result into seconds, divide by the macro `CLOCKS_PER_SEC`.

Returns

The amount of processor time used so far by your program, in units defined by the machine-dependent macro `CLOCKS_PER_SEC`. If no measurement is available, the result is `(clock_t)-1`.

Portability

ANSI C requires `clock` and `CLOCKS_PER_SEC`.

Supporting OS subroutine required: `times`.

7.3 `ctime`—convert time to local and format as string

Synopsis

```
#include <time.h>
char *ctime(const time_t *clock);
char *ctime_r(const time_t *clock, char *buf);
```

Description

Convert the time value at *clock* to local time (like `localtime`) and format it into a string of the form

```
Wed Jun 15 11:38:07 1988\n\0
```

(like `asctime`).

Returns

A pointer to the string containing a formatted timestamp.

Portability

ANSI C requires `ctime`.

`ctime` requires no supporting OS subroutines.

7.4 difftime—subtract two times

Synopsis

```
#include <time.h>
double difftime(time_t tim1, time_t tim2);
```

Description

Subtracts the two times in the arguments: '*tim1* - *tim2*'.

Returns

The difference (in seconds) between *tim2* and *tim1*, as a double.

Portability

ANSI C requires `difftime`, and defines its result to be in seconds in all implementations.

`difftime` requires no supporting OS subroutines.

7.5 `gmtime`—convert time to UTC traditional form

Synopsis

```
#include <time.h>
struct tm *gmtime(const time_t *clock);
struct tm *gmtime_r(const time_t *clock, struct tm *res);
```

Description

`gmtime` assumes the time at *clock* represents a local time. `gmtime` converts it to UTC (Universal Coordinated Time, also known in some countries as GMT, Greenwich Mean time), then converts the representation from the arithmetic representation to the traditional representation defined by `struct tm`.

`gmtime` constructs the traditional time representation in static storage; each call to `gmtime` or `localtime` will overwrite the information generated by previous calls to either function.

Returns

A pointer to the traditional time representation (`struct tm`).

Portability

ANSI C requires `gmtime`.

`gmtime` requires no supporting OS subroutines.

7.6 localtime—convert time to local representation

Synopsis

```
#include <time.h>
struct tm *localtime(time_t *clock);
struct tm *localtime_r(time_t *clock, struct tm *res);
```

Description

localtime converts the time at *clock* into local time, then converts its representation from the arithmetic representation to the traditional representation defined by **struct tm**.

localtime constructs the traditional time representation in static storage; each call to **gmtime** or **localtime** will overwrite the information generated by previous calls to either function.

mktime is the inverse of **localtime**.

Returns

A pointer to the traditional time representation (**struct tm**).

Portability

ANSI C requires **localtime**.

localtime requires no supporting OS subroutines.

7.7 mktime—convert time to arithmetic representation

Synopsis

```
#include <time.h>
time_t mktime(struct tm *timp);
```

Description

`mktime` assumes the time at *timp* is a local time, and converts its representation from the traditional representation defined by `struct tm` into a representation suitable for arithmetic. `localtime` is the inverse of `mktime`.

Returns

If the contents of the structure at *timp* do not form a valid calendar time representation, the result is -1. Otherwise, the result is the time, converted to a `time_t` value.

Portability

ANSI C requires `mktime`.

`mktime` requires no supporting OS subroutines.

7.8 strftime—flexible calendar time formatter

Synopsis

```
#include <time.h>
size_t strftime(char *s, size_t maxsize,
    const char *format, const struct tm *timp);
```

Description

strftime converts a **struct tm** representation of the time (at *timp*) into a string, starting at *s* and occupying no more than *maxsize* characters.

You control the format of the output using the string at *format*. **format* can contain two kinds of specifications: text to be copied literally into the formatted string, and time conversion specifications. Time conversion specifications are two-character sequences beginning with '%' (use '%%' to include a percent sign in the output). Each defined conversion specification selects a field of calendar time data from **timp*, and converts it to a string in one of the following ways:

%a	An abbreviation for the day of the week.
%A	The full name for the day of the week.
%b	An abbreviation for the month name.
%B	The full name of the month.
%c	A string representing the complete date and time, in the form Mon Apr 01 13:13:13 1992
%d	The day of the month, formatted with two digits.
%e	The day of the month, formatted with leading space if single digit.
%H	The hour (on a 24-hour clock), formatted with two digits.
%I	The hour (on a 12-hour clock), formatted with two digits.
%j	The count of days in the year, formatted with three digits (from '001' to '366').
%m	The month number, formatted with two digits.
%M	The minute, formatted with two digits.
%p	Either 'AM' or 'PM' as appropriate.
%S	The second, formatted with two digits.
%U	The week number, formatted with two digits (from '00' to '53'; week number 1 is taken as beginning with the first Sunday in a year). See also %W.
%w	A single digit representing the day of the week: Sunday is day 0.
%W	Another version of the week number: like '%U', but counting week 1 as beginning with the first Monday in a year.
o %x	A string representing the complete date, in a format like Mon Apr 01 1992

%X	A string representing the full time of day (hours, minutes, and seconds), in a format like 13:13:13
%y	The last two digits of the year.
%Y	The full year, formatted with four digits to include the century.
%Z	The time zone name. If <code>tm_isdst</code> is -1, no output is generated. Otherwise, the time zone name based on the TZ environment variable is used.
%%	A single character, '%'.

Returns

When the formatted time takes up no more than *maxsize* characters, the result is the length of the formatted string. Otherwise, if the formatting operation was abandoned due to lack of room, the result is 0, and the string starting at *s* corresponds to just those parts of **format* that could be completely filled in within the *maxsize* limit.

Portability

ANSI C requires `strftime`, but does not specify the contents of **s* when the formatted string would require more than *maxsize* characters.

`strftime` requires no supporting OS subroutines.

7.9 time—get current calendar time (as single number)

Synopsis

```
#include <time.h>
time_t time(time_t *t);
```

Description

`time` looks up the best available representation of the current time and returns it, encoded as a `time_t`. It stores the same value at `t` unless the argument is `NULL`.

Returns

A `-1` result means the current time is not available; otherwise the result represents the current time.

Portability

ANSI C requires `time`.

Supporting OS subroutine required: Some implementations require `gettimeofday`.

7.10 `__tz_lock`, `__tz_unlock`—lock time zone global variables

Synopsis

```
#include "local.h"
void __tz_lock (void);
void __tz_unlock (void);
```

Description

The `tzset` facility functions call these functions when they need to ensure the values of global variables. The version of these routines supplied in the library use the lock API defined in `sys/lock.h`. If multiple threads of execution can call the time functions and give up scheduling in the middle, then you need to define your own versions of these functions in order to safely lock the time zone variables during a call. If you do not, the results of `localtime`, `mktime`, `ctime`, and `strftime` are undefined.

The lock `__tz_lock` may not be called recursively; that is, a call `__tz_lock` will always lock all subsequent `__tz_lock` calls until the corresponding `__tz_unlock` call on the same thread is made.

7.11 tzset—set timezone characteristics from TZ environment variable

Synopsis

```
#include <time.h>
void tzset(void);
void _tzset_r (struct _reent *);
```

Description

tzset examines the TZ environment variable and sets up the three external variables: `_timezone`, `_daylight`, and `tzname`. The value of `_timezone` shall be the offset from the current time zone to GMT. The value of `_daylight` shall be 0 if there is no daylight savings time for the current time zone, otherwise it will be non-zero. The `tzname` array has two entries: the first is the name of the standard time zone, the second is the name of the daylight-savings time zone.

The TZ environment variable is expected to be in the following POSIX format:

```
stdoffset1[dst[offset2][,start[/time1],end[/time2]]]
```

where: `std` is the name of the standard time-zone (minimum 3 chars) `offset1` is the value to add to local time to arrive at Universal time it has the form: `hh[:mm[:ss]]` `dst` is the name of the alternate (daylight-savings) time-zone (min 3 chars) `offset2` is the value to add to local time to arrive at Universal time it has the same format as the `std` `offset` `start` is the day that the alternate time-zone starts `time1` is the optional time that the alternate time-zone starts (this is in local time and defaults to 02:00:00 if not specified) `end` is the day that the alternate time-zone ends `time2` is the time that the alternate time-zone ends (it is in local time and defaults to 02:00:00 if not specified)

Note that there is no white-space padding between fields. Also note that if TZ is null, the default is Universal GMT which has no daylight-savings time. If TZ is empty, the default EST5EDT is used.

The function `_tzset_r` is identical to `tzset` only it is reentrant and is used for applications that use multiple threads.

Returns

There is no return value.

Portability

tzset is part of the POSIX standard.

Supporting OS subroutine required: None

8 Locale ('locale.h')

A *locale* is the name for a collection of parameters (affecting collating sequences and formatting conventions) that may be different depending on location or culture. The "C" locale is the only one defined in the ANSI C standard.

This is a minimal implementation, supporting only the required "C" value for locale; strings representing other locales are not honored. ("" is also accepted; it represents the default locale for an implementation, here equivalent to "C".

'locale.h' defines the structure `lconv` to collect the information on a locale, with the following fields:

`char *decimal_point`

The decimal point character used to format "ordinary" numbers (all numbers except those referring to amounts of money). "." in the C locale.

`char *thousands_sep`

The character (if any) used to separate groups of digits, when formatting ordinary numbers. "" in the C locale.

`char *grouping`

Specifications for how many digits to group (if any grouping is done at all) when formatting ordinary numbers. The *numeric value* of each character in the string represents the number of digits for the next group, and a value of 0 (that is, the string's trailing NULL) means to continue grouping digits using the last value specified. Use `CHAR_MAX` to indicate that no further grouping is desired. "" in the C locale.

`char *int_curr_symbol`

The international currency symbol (first three characters), if any, and the character used to separate it from numbers. "" in the C locale.

`char *currency_symbol`

The local currency symbol, if any. "" in the C locale.

`char *mon_decimal_point`

The symbol used to delimit fractions in amounts of money. "" in the C locale.

`char *mon_thousands_sep`

Similar to `thousands_sep`, but used for amounts of money. "" in the C locale.

`char *mon_grouping`

Similar to `grouping`, but used for amounts of money. "" in the C locale.

`char *positive_sign`

A string to flag positive amounts of money when formatting. "" in the C locale.

`char *negative_sign`

A string to flag negative amounts of money when formatting. "" in the C locale.

`char int_frac_digits`

The number of digits to display when formatting amounts of money to international conventions. `CHAR_MAX` (the largest number representable as a `char`) in the C locale.

`char frac_digits`

The number of digits to display when formatting amounts of money to local conventions. `CHAR_MAX` in the C locale.

`char p_cs_precedes`

1 indicates the local currency symbol is used before a *positive or zero* formatted amount of money; 0 indicates the currency symbol is placed after the formatted number. `CHAR_MAX` in the C locale.

`char p_sep_by_space`

1 indicates the local currency symbol must be separated from *positive or zero* numbers by a space; 0 indicates that it is immediately adjacent to numbers. `CHAR_MAX` in the C locale.

`char n_cs_precedes`

1 indicates the local currency symbol is used before a *negative* formatted amount of money; 0 indicates the currency symbol is placed after the formatted number. `CHAR_MAX` in the C locale.

`char n_sep_by_space`

1 indicates the local currency symbol must be separated from *negative* numbers by a space; 0 indicates that it is immediately adjacent to numbers. `CHAR_MAX` in the C locale.

`char p_sign_posn`

Controls the position of the *positive* sign for numbers representing money. 0 means parentheses surround the number; 1 means the sign is placed before both the number and the currency symbol; 2 means the sign is placed after both the number and the currency symbol; 3 means the sign is placed just before the currency symbol; and 4 means the sign is placed just after the currency symbol. `CHAR_MAX` in the C locale.

`char n_sign_posn`

Controls the position of the *negative* sign for numbers representing money, using the same rules as `p_sign_posn`. `CHAR_MAX` in the C locale.

8.1 setlocale, localeconv—select or query locale

Synopsis

```
#include <locale.h>
char *setlocale(int category, const char *locale);
lconv *localeconv(void);

char *_setlocale_r(void *reent,
                  int category, const char *locale);
lconv *_localeconv_r(void *reent);
```

Description

setlocale is the facility defined by ANSI C to condition the execution environment for international collating and formatting information; **localeconv** reports on the settings of the current locale.

This is a minimal implementation, supporting only the required "C" value for *locale*; strings representing other locales are not honored unless `_MB_CAPABLE` is defined in which case three new extensions are allowed for `LC_CTYPE` or `LC_MESSAGES` only: "C-JIS", "C-EUCJP", "C-SJIS", or "C-ISO-8859-1". ("" is also accepted; it represents the default locale for an implementation, here equivalent to "C".)

If you use `NULL` as the *locale* argument, **setlocale** returns a pointer to the string representing the current locale (always "C" in this implementation). The acceptable values for *category* are defined in 'locale.h' as macros beginning with "LC_", but this implementation does not check the values you pass in the *category* argument.

localeconv returns a pointer to a structure (also defined in 'locale.h') describing the locale-specific conventions currently in effect.

`_localeconv_r` and `_setlocale_r` are reentrant versions of **localeconv** and **setlocale** respectively. The extra argument *reent* is a pointer to a reentrancy structure.

Returns

setlocale returns either a pointer to a string naming the locale currently in effect (always "C" for this implementation, or, if the locale request cannot be honored, `NULL`).

localeconv returns a pointer to a structure of type `lconv`, which describes the formatting and collating conventions in effect (in this implementation, always those of the C locale).

Portability

ANSI C requires **setlocale**, but the only locale required across all implementations is the C locale.

No supporting OS subroutines are required.

9 Miscellaneous Macros and Functions

This chapter describes miscellaneous routines not covered elsewhere.

9.1 ffs—find first bit set in a word

Synopsis

```
#include <string.h>
int ffs(int word);
```

Description

ffs returns the first bit set in a word.

Returns

ffs returns 0 if *c* is 0, 1 if *c* is odd, 2 if *c* is a multiple of 2, etc.

Portability

ffs is not ANSI C.

No supporting OS subroutines are required.

9.2 **isatty**—report whether a given file descriptor is attached to a terminal

Synopsis

```
#include <unistd.h>
int isatty(int fd);
```

Description

isatty returns 1 if the given file descriptor references a terminal, or 0 otherwise. Within tile-sim's "magic-supervisor" mode, we always assume that the initial file descriptors assigned to stdin, stdout, and stderr reference a terminal.

Portability

isatty conforms to SVID and X/OPEN.

No supporting OS subroutines are required.

9.3 `setjmp`, `longjmp`—non-local jump support

Synopsis

```
#include <setjmp.h>
void longjmp(jmp_buf env, int val);
int setjmp(jmp_buf env);
```

Description

`setjmp` sets up a location which can be returned to later. The specific location (including register and stack) is saved to the specified `jmp_buf env`.

`longjmp` transfer control to the location described by the given buffer, and provides a return `val` to use as the return value from `setjmp`. `longjmp` must pass a non-zero value to use as the return value, or else 1 is returned.

Returns

`setjmp` returns zero if called directly, and non-zero if returning from a `longjmp` that returned control to it.

Portability

`longjmp` and `setjmp` conform to POSIX.

No supporting OS subroutines are required.

9.4 unctrl—translate characters to upper case

Synopsis

```
#include <unctrl.h>
char *unctrl(int c);
int unctrlllen(int c);
```

Description

unctrl is a macro which returns the printable representation of *c* as a string. **unctrlllen** is a macro which returns the length of the printable representation of *c*.

Returns

unctrl returns a string of the printable representation of *c*.

unctrlllen returns the length of the string which is the printable representation of *c*.

Portability

unctrl and **unctrlllen** are not ANSI C.

No supporting OS subroutines are required.

10 System Calls

10.1 Definitions for OS interface

This section is TBD. Supported Unix system calls include:

```
_exit  
brk  
close  
execve  
fcntl  
fstat  
getpid  
gettimeofday  
kill  
link  
lseek  
mmap  
mprotect  
mremap  
munmap  
open  
read  
select  
socketcall  
spawn  
stat  
times  
unlink  
write
```


11 Variable Argument Lists

The `printf` family of functions is defined to accept a variable number of arguments, rather than a fixed argument list. You can define your own functions with a variable argument list, by using macro definitions from `'stdarg.h'`.

11.1 ANSI-standard macros, `'stdarg.h'`

In ANSI C, a function has a variable number of arguments when its parameter list ends in an ellipsis (...). The parameter list must also include at least one explicitly named argument; that argument is used to initialize the variable list data structure.

ANSI C defines three macros (`va_start`, `va_arg`, and `va_end`) to operate on variable argument lists. `'stdarg.h'` also defines a special type to represent variable argument lists: this type is called `va_list`.

11.1.1 Initialize variable argument list

Synopsis

```
#include <stdarg.h>
void va_start(va_list ap, rightmost);
```

Description

Use `va_start` to initialize the variable argument list `ap`, so that `va_arg` can extract values from it. `rightmost` is the name of the last explicit argument in the parameter list (the argument immediately preceding the ellipsis ‘`...`’ that flags variable arguments in an ANSI C function header). You can only use `va_start` in a function declared using this ellipsis notation (not, for example, in one of its subfunctions).

Returns

`va_start` does not return a result.

Portability

ANSI C requires `va_start`.

11.1.2 Extract a value from argument list

Synopsis

```
#include <stdarg.h>
type va_arg(va_list ap, type);
```

Description

va_arg returns the next unprocessed value from a variable argument list *ap* (which you must previously create with *va_start*). Specify the type for the value as the second parameter to the macro, *type*.

You may pass a **va_list** object *ap* to a subfunction, and use **va_arg** from the subfunction rather than from the function actually declared with an ellipsis in the header; however, in that case you may *only* use **va_arg** from the subfunction. ANSI C does not permit extracting successive values from a single variable-argument list from different levels of the calling stack.

There is no mechanism for testing whether there is actually a next argument available; you might instead pass an argument count (or some other data that implies an argument count) as one of the fixed arguments in your function call.

Returns

va_arg returns the next argument, an object of type *type*.

Portability

ANSI C requires **va_arg**.

11.1.3 Abandon a variable argument list

Synopsis

```
#include <stdarg.h>
void va_end(va_list ap);
```

Description

Use `va_end` to declare that your program will not use the variable argument list *ap* any further.

Returns

`va_end` does not return a result.

Portability

ANSI C requires `va_end`.

Index

-	
__env_lock	16
__env_unlock	16
__malloc_lock	26
__malloc_unlock	26
__tz_lock	228
__tz_unlock	228
_asctime_r	218
_atoi_r	9
_atol_r	9
_atoll_r	10
_calloc_r	11
_Exit	2
_fclose_r	83
_fcloseall_r	84
_fdopen_r	95
_fgetpos_r	90
_fopen_r	93
_free_r	23
_freopen_r	99
_fseek_r	100
_fseeko_r	100
_fsetpos_r	101
_ftell_r	102
_ftello_r	102
_getchar_r	106
_gets_r	110
_localeconv_r	233
_mallinfo_r	25
_malloc_r	23
_malloc_stats_r	25
_malloc_usable_size_r	23
_mallopt_r	25
_memalign_r	23
_mkstemp_r	112
_mktemp_r	112
_perror_r	113
_putchar_r	116
_puts_r	118
_raise_r	214, 215
_realloc_r	23
_rename_r	121
_rewind_r	122
_setlocale_r	233
_signal_r	215
_strtod_r	34
_strtol_r	35
_strtoll_r	37
_strtoul_r	39
_strtoull_r	41
_system_r	43
_tempnam_r	139
_tmpfile_r	138
_tmpnam_r	139
_tolower	60
_toupper	61
A	
a64l	3
abort	4
abs	5
asctime	218
asiprintf	136
asprintf	127
assert	6
atexit	7
atof	8
atoff	8
atoi	9
atol	9
atoll	10
B	
bcmp	146
bzero	148
C	
calloc	11
clearerr	82
clock	219
ctime	220
D	
difftime	221
div	12
drand48	32
E	
ecvt	13
ecvtbuf	15
environ	18
erand48	32
exit	17
F	
fclose	83
fcloseall	84
fcvt	13
fcvtbuf	15
fdopen	95
feof	85
ferror	86
fflush	87

ffs	236
fgetc	89
fgetpos	90
fgets	91
fileno	92
fiprintf	136
fiscanf	137
flockfile	88
fopen	93
fprintf	127
fputc	96
fputs	97
fread	98
free	23
freopen	99
fscanf	131
fseek	100
fseeko	100
fsetpos	101
ftell	102
ftello	102
ftrylockfile	88
funlockfile	88
fwrite	103

G

gcvt	14
gcvtf	14
getc	104
getc_unlocked	105
getchar	106
getchar_unlocked	107
getdelim	108
getenv	18
getline	109
gets	110
getw	111
gmtime	222

I

index	149
iprintf	136
isalnum	48
isalpha	49
isascii	50
isatty	237
iscanf	137
iscntrl	51
isdigit	52
isgraph	54
islower	53
isprint	54
ispunct	55
isspace	56
isupper	57
iswalnum	62

iswalpha	63
iswblank	65
iswcntrl	64
iswctype	74
iswdigit	66
iswgraph	67
iswlower	68
iswprint	69
iswpunct	70
iswspace	71
iswupper	72
iswxdigit	73
isxdigit	58

J

jrand48	32
---------------	----

L

l64a	3
labs	19
lcong48	32
ldiv	20
llabs	21
lldiv	22
localeconv	233
localtime	223
longjmp	238
lrand48	32

M

mallinfo	25
malloc	23
malloc_stats	25
malloc_usable_size	23
mallopt	25
mblen	27
mbstowcs	28
mbtowc	29
memalign	23
memchr	151
memcmp	152
memmove	154
memset	156
mkstemp	112
mktemp	112
mktime	224
mrnd48	32

N

nrand48	32
---------------	----

O

on_exit	30
---------------	----

OS interface subroutines..... 241

P

perror..... 113
 printf..... 127
 putc..... 114
 putc_unlocked..... 115
 putchar..... 116
 putchar_unlocked..... 117
 puts..... 118
 putw..... 119

R

raise..... 214, 215
 rand..... 31
 rand_r..... 31
 rand48..... 32
 realloc..... 23
 remove..... 120
 rename..... 121
 rewind..... 122
 rindex..... 157

S

scanf..... 131
 seed48..... 32
 setbuf..... 123
 setbuffer..... 124
 setjmp..... 238
 setlinebuf..... 125
 setlocale..... 233
 setvbuf..... 126
 signal..... 215
 siprintf..... 136
 siscanf..... 137
 sniprintf..... 136
 snprintf..... 127
 sprintf..... 127
 srand..... 31
 srand48..... 32
 sscanf..... 131
 strcasecmp..... 158
 strcat..... 159
 strchr..... 160
 strcmp..... 161
 strcoll..... 162
 strcpy..... 163
 strcspn..... 164
 strerror..... 165
 strerror_r..... 169
 strftime..... 225
 strlen..... 170
 strlwr..... 171
 strncasecmp..... 172
 strncat..... 173

strncmp..... 174
 strncpy..... 175
 strnlen..... 176
 strpbrk..... 177
 strrchr..... 178
 strsep..... 182
 strsignal..... 179
 strspn..... 180
 strstr..... 181
 strtod..... 34
 strtodf..... 34
 strtok..... 182
 strtok_r..... 182
 strtol..... 35
 strtoll..... 37
 strtoul..... 39
 strtoull..... 41
 strupr..... 183
 strxfrm..... 184
 stubs..... 241
 subroutines for OS interface..... 241
 system..... 43

T

tempnam..... 139
 time..... 227
 tmpfile..... 138
 tmpnam..... 139
 toascii..... 59
 tolower..... 60
 toupper..... 61
 towctrans..... 78
 tolower..... 76
 towupper..... 77
 tzset..... 229

U

unctrl..... 239
 unctrlen..... 239

V

va_arg..... 245
 va_end..... 246
 va_start..... 244
 vfiprintf..... 142
 vfiscanf..... 143
 vfprintf..... 140
 vfscanf..... 141
 viprintf..... 142
 viscanf..... 143
 vprintf..... 140
 vscanf..... 141
 vsiprintf..... 142
 vsiscanf..... 143
 vsniprintf..... 142

`vsnprintf`..... 140
`vsprintf`..... 140
`vsscanf`..... 141

W

`wscat`..... 193
`wscoll`..... 196

`wcsnlen`..... 205
`wcstombs`..... 44
`wcswidth`..... 210
`wctomb`..... 45
`wctrans`..... 79
`wctype`..... 75
`wcwidth`..... 211

The body of this manual is set in
cmr10 at 10.95pt,
with headings in **cmb10 at 10.95pt**
and examples in cmtt10 at 10.95pt.
cmti10 at 10.95pt and
cmsl10 at 10.95pt
are used for emphasis.

Table of Contents

1	Standard Utility Functions ('stdlib.h')	1
1.1	_Exit—end program execution with no cleanup processing	2
1.2	a64l,l64a—convert between radix-64 ascii string and long	3
1.3	abort—abnormal termination of a program	4
1.4	abs—integer absolute value (magnitude)	5
1.5	assert—Macro for Debugging Diagnostics	6
1.6	atexit—request execution of functions at program exit	7
1.7	atof, atoff—string to double or float	8
1.8	atoi, atol—string to integer	9
1.9	atoll—convert a string to a long long integer	10
1.10	calloc—allocate space for arrays	11
1.11	div—divide two integers	12
1.12	ecvt,ecvtf,fcvt,fcvtf—double or float to string	13
1.13	gvcvt, gcvtf—format double or float as string	14
1.14	ecvtbuf, fcvtbuf—double or float to string	15
1.15	__env_lock, __env_unlock—lock environ variable	16
1.16	exit—end program execution	17
1.17	getenv—look up environment variable	18
1.18	labs—long integer absolute value	19
1.19	ldiv—divide two long integers	20
1.20	llabs—compute the absolute value of an long long integer	21
1.21	lldiv—divide two long long integers	22
1.22	malloc, realloc, free—manage memory	23
1.23	mallinfo, malloc_stats, mallopt—malloc support	25
1.24	__malloc_lock, __malloc_unlock—lock malloc pool	26
1.25	mblen—minimal multibyte length function	27
1.26	mbstowcs—minimal multibyte string to wide char converter	28
1.27	mbtowc—minimal multibyte to wide char converter	29
1.28	on_exit—request execution of function with argument at program exit	30
1.29	rand, srand—pseudo-random numbers	31
1.30	rand48, drand48, erand48, lrand48, nrand48, mrand48, jrand48, srand48, seed48, lcong48 —pseudo random number generators and initialization routines	32
1.31	strtod, strtodf—string to double or float	34
1.32	strtol—string to long	35
1.33	strtoll—string to long long	37
1.34	strtoul—string to unsigned long	39
1.35	strtoull—string to unsigned long long	41
1.36	system—execute command string	43
1.37	wcstombs—minimal wide char string to multibyte string converter	44
1.38	wctomb—minimal wide char to multibyte converter	45

2 Character Type Macros and Functions

(‘ctype.h’)	47
2.1 isalnum—alphanumeric character predicate	48
2.2 isalpha—alphabetic character predicate	49
2.3 isascii—ASCII character predicate	50
2.4 iscntrl—control character predicate	51
2.5 isdigit—decimal digit predicate	52
2.6 islower—lower-case character predicate	53
2.7 isprint, isgraph—printable character predicates	54
2.8 ispunct—punctuation character predicate	55
2.9 isspace—whitespace character predicate	56
2.10 isupper—uppercase character predicate	57
2.11 isxdigit—hexadecimal digit predicate	58
2.12 toascii—force integers to ASCII range	59
2.13 tolower—translate characters to lower case	60
2.14 toupper—translate characters to upper case	61
2.15 iswalnum—alpha-numeric wide-character test	62
2.16 iswalpha—alphabetic wide-character test	63
2.17 iswcntrl—wide-character cntrl test	64
2.18 iswblank—wide-character blank test	65
2.19 iswdigit—decimal digit wide-character test	66
2.20 iswgraph—graphic wide-character test	67
2.21 iswlower—lower-case wide-character test	68
2.22 iswprint—printable wide-character test	69
2.23 iswpunct—punctuation wide-character test	70
2.24 iswspace—wide-character space test	71
2.25 iswupper—upper-case wide-character test	72
2.26 iswxdigit—hexadecimal digit wide-character test	73
2.27 iswctype—extensible wide-character test	74
2.28 wctype—get wide-character classification type	75
2.29 towlower—translate wide-characters to lower case	76
2.30 towupper—translate wide-characters to upper case	77
2.31 towctrans—extensible wide-character case mapping	78
2.32 wctrans—get wide-character translation type	79

3 Input and Output (‘stdio.h’)

3.1 clearerr—clear file or stream error indicator	82
3.2 fclose—close a file	83
3.3 fcloseall—close all files	84
3.4 feof—test for end of file	85
3.5 ferror—test whether read/write error has occurred	86
3.6 fflush—flush buffered file output	87
3.7 flockfile, funlockfile, ftrylockfile—lock and unlock a stream	88
3.8 fgetc—get a character from a file or stream	89
3.9 fgetpos—record position in a stream or file	90
3.10 fgets—get character string from a file or stream	91
3.11 fileno—return file descriptor associated with stream	92

3.12	<code>fopen</code> —open a file	93
3.13	<code>fdopen</code> —turn open file into a stream	95
3.14	<code>fputc</code> —write a character on a stream or file	96
3.15	<code>fputs</code> —write a character string in a file or stream	97
3.16	<code>fread</code> —read array elements from a file	98
3.17	<code>freopen</code> —open a file using an existing file descriptor	99
3.18	<code>fseek</code> , <code>fseeko</code> —set file position	100
3.19	<code>fsetpos</code> —restore position of a stream or file	101
3.20	<code>ftell</code> , <code>ftello</code> —return position in a stream or file	102
3.21	<code>fwrite</code> —write array elements	103
3.22	<code>getc</code> —read a character (macro)	104
3.23	<code>getc_unlocked</code> —non-thread-safe version of <code>getc</code> (macro)	105
3.24	<code>getchar</code> —read a character (macro)	106
3.25	<code>getchar_unlocked</code> —non-thread-safe version of <code>getchar</code> (macro)	107
3.26	<code>getdelim</code> —read a line up to a specified line delimiter	108
3.27	<code>getline</code> —read a line from a file	109
3.28	<code>gets</code> —get character string (obsolete, use <code>fgets</code> instead)	110
3.29	<code>getw</code> —read a word (int)	111
3.30	<code>mktemp</code> , <code>mkstemp</code> —generate unused file name	112
3.31	<code>perror</code> —print an error message on standard error	113
3.32	<code>putc</code> —write a character (macro)	114
3.33	<code>putc_unlocked</code> —non-thread-safe version of <code>putc</code> (macro)	115
3.34	<code>putchar</code> —write a character (macro)	116
3.35	<code>putchar_unlocked</code> —non-thread-safe version of <code>putchar</code> (macro)	117
3.36	<code>puts</code> —write a character string	118
3.37	<code>putw</code> —write a word (int)	119
3.38	<code>remove</code> —delete a file’s name	120
3.39	<code>rename</code> —rename a file	121
3.40	<code>rewind</code> —reinitialize a file or stream	122
3.41	<code>setbuf</code> —specify full buffering for a file or stream	123
3.42	<code>setbuffer</code> —specify full buffering for a file or stream with size	124
3.43	<code>setlinebuf</code> —specify line buffering for a file or stream	125
3.44	<code>setvbuf</code> —specify file or stream buffering	126
3.45	<code>printf</code> , <code>fprintf</code> , <code>asprintf</code> , <code>sprintf</code> , <code>snprintf</code> —format output	127
3.46	<code>scanf</code> , <code>fscanf</code> , <code>sscanf</code> —scan and format input	131
3.47	<code>iprintf</code> , <code>fiprintf</code> , <code>asiprintf</code> , <code>siprintf</code> , <code>sniprintf</code> —format output	136
3.48	<code>iscanf</code> , <code>fiscanf</code> , <code>sscanf</code> —scan and format non-floating input	137
3.49	<code>tmpfile</code> —create a temporary file	138
3.50	<code>tmpnam</code> , <code>tempnam</code> —name for a temporary file	139
3.51	<code>vprintf</code> , <code>vfprintf</code> , <code>vsprintf</code> —format argument list	140
3.52	<code>vscanf</code> , <code>vfscanf</code> , <code>vsscanf</code> —format argument list	141
3.53	<code>viprintf</code> , <code>vfiprintf</code> , <code>vsiprintf</code> —format argument list	142

3.54	<code>viscanf, vfiscanf, vsiscanf</code> —format argument list.....	143
------	---	-----

4	Strings and Memory ('string.h')	145
4.1	<code>bcmp</code> —compare two memory areas	146
4.2	<code>bcopy</code> —copy memory regions	147
4.3	<code>bzero</code> —initialize memory to zero.....	148
4.4	<code>index</code> —search for character in string.....	149
4.5	<code>memccpy</code> —copy memory regions with end-token check	150
4.6	<code>memchr</code> —find character in memory	151
4.7	<code>memcmp</code> —compare two memory areas	152
4.8	<code>memcpy</code> —copy memory regions	153
4.9	<code>memmove</code> —move possibly overlapping memory	154
4.10	<code>mempcpy</code> —copy memory regions and return end pointer.....	155
4.11	<code>memset</code> —set an area of memory	156
4.12	<code>rindex</code> —reverse search for character in string.....	157
4.13	<code>strcasecmp</code> —case insensitive character string compare.....	158
4.14	<code>strcat</code> —concatenate strings.....	159
4.15	<code>strchr</code> —search for character in string.....	160
4.16	<code>strcmp</code> —character string compare	161
4.17	<code>strcoll</code> —locale specific character string compare.....	162
4.18	<code>strcpy</code> —copy string.....	163
4.19	<code>strcspn</code> —count chars not in string	164
4.20	<code>strerror</code> —convert error number to string.....	165
4.21	<code>strerror_r</code> —convert error number to string and copy to buffer	169
4.22	<code>strlen</code> —character string length.....	170
4.23	<code>strlwr</code> —force string to lower case	171
4.24	<code>strncasecmp</code> —case insensitive character string compare.....	172
4.25	<code>strncat</code> —concatenate strings.....	173
4.26	<code>strncmp</code> —character string compare	174
4.27	<code>strncpy</code> —counted copy string	175
4.28	<code>strnlen</code> —character string length	176
4.29	<code>strpbrk</code> —find chars in string.....	177
4.30	<code>strrchr</code> —reverse search for character in string.....	178
4.31	<code>strsignal</code> —return string describing a signal.....	179
4.32	<code>strspn</code> —find initial match.....	180
4.33	<code>strstr</code> —find string segment	181
4.34	<code>strtok, strtok_r, strsep</code> —get next token from a string	182
4.35	<code>strupr</code> —force string to uppercase.....	183
4.36	<code>strxfrm</code> —transform string.....	184
4.37	<code>swab</code> —swap adjacent bytes.....	185

5 Wide Character Strings ('wchar.h') 187

5.1	<code>wmemchr</code> —find a wide-character in memory	188
5.2	<code>wmemcmp</code> —compare wide-characters in memory	189
5.3	<code>wmemcpy</code> —copy wide-characters in memory	190
5.4	<code>wmemmove</code> — <code>wmemmove</code> - copy wide-characters in memory with overlapping areas	191
5.5	<code>wmemset</code> —set wide-characters in memory	192
5.6	<code>wscat</code> —concatenate two wide-character strings	193
5.7	<code>wscchr</code> —wide-character string scanning operation	194
5.8	<code>wscmp</code> —compare two wide-character strings	195
5.9	<code>wscoll</code> —locale specific wide-character string compare	196
5.10	<code>wscpy</code> —copy a wide-character string	197
5.11	<code>wscspn</code> —get length of a complementary wide substring	198
5.12	<code>wslcat</code> —concatenate wide-character strings to specified length	199
5.13	<code>wslcpy</code> —copy a wide-character string to specified length ...	200
5.14	<code>wslen</code> —get wide-character string length	201
5.15	<code>wscncat</code> —concatenate part of two wide-character strings	202
5.16	<code>wscncmp</code> —compare part of two wide-character strings	203
5.17	<code>wscncpy</code> —copy part of a wide-character string	204
5.18	<code>wcsnlen</code> —get fixed-size wide character string length	205
5.19	<code>wcspbrk</code> —scan wide-character string for a wide-character code	206
5.20	<code>wcsrchr</code> —wide-character string scanning operation	207
5.21	<code>wcsspn</code> —get length of a wide substring	208
5.22	<code>wcsstr</code> —find a wide-character substring	209
5.23	<code>wcswidth</code> —number of column positions of a wide-character string	210
5.24	<code>wcwidth</code> —number of column positions of a wide-character code	211

6 Signal Handling ('signal.h') 213

6.1	<code>raise</code> —send a signal	214
6.2	<code>signal</code> —specify handler subroutine for a signal	215

7 Time Functions ('time.h') 217

7.1	<code>asctime</code> —format time as string	218
7.2	<code>clock</code> —cumulative processor time	219
7.3	<code>ctime</code> —convert time to local and format as string	220
7.4	<code>difftime</code> —subtract two times	221
7.5	<code>gmtime</code> —convert time to UTC traditional form	222
7.6	<code>localtime</code> —convert time to local representation	223
7.7	<code>mktime</code> —convert time to arithmetic representation	224
7.8	<code>strftime</code> —flexible calendar time formatter	225
7.9	<code>time</code> —get current calendar time (as single number)	227
7.10	<code>__tz_lock</code> , <code>__tz_unlock</code> —lock time zone global variables	228
7.11	<code>tzset</code> —set timezone characteristics from TZ environment variable	229

8	Locale ('locale.h')	231
8.1	setlocale, localeconv—select or query locale.....	233
9	Miscellaneous Macros and Functions	235
9.1	ffs—find first bit set in a word	236
9.2	isatty—report whether a given file descriptor is attached to a terminal.....	237
9.3	setjmp, longjmp—non-local jump support	238
9.4	unctrl—translate characters to upper case.....	239
10	System Calls	241
10.1	Definitions for OS interface	241
11	Variable Argument Lists	243
11.1	ANSI-standard macros, 'stdarg.h'	243
11.1.1	Initialize variable argument list	244
11.1.2	Extract a value from argument list	245
11.1.3	Abandon a variable argument list	246
	Index	247